

Innehållsförteckning

Innehållsförteckning	1
Java för Jin23	11
Innehållsförteckning	12
Java för Jin23	18
Home	18
Välkommen	18
Java för Jin23	19
Java	19
Java för Jin23	20
Skolan	20
Här är skolans webbplatser	20
Java för Jin23	21
Skolportalen	21
Java för Jin23	22
Google Classroom	22
Java för Jin23	23
Google Calendar	23
Java för Jin23	24
Google Documents	24
Java för Jin23	25
Discord	25
Java för Jin23	26
Info	26
Versionshistorik	26
Java för Jin23	27
Om sidan	27
Java för Jin23	28
Licens för materialet på denna webbsida	28
Sammanfattning	28
Enkelt förklarat	28
Java för Jin23	29
MIT-licens för Just the docs	29
Java för Jin23	30
Verktyg	30
1. IDE (Integrated Development Environment)	30
2. Byggverktyg	30
3. Versionshanteringssystem	30
4. Enhetstestningsramverk	31
5. Felsökning och profileringsverktyg	31
Summering	31
Java för Jin23	32
Installation	32
Java för Jin23	33
Git installation	33
Java för Jin23	34
Visual Studio installation	34
Beskrivning	34
Java för Jin23	35
VS Code installation	35
Beskrivning	35
Java för Jin23	36
IntelliJinstallation	36
Addons till IntelliJ som jag rekommenderar	36

Beskrivning	36
Java för Jin23	37
NetBeans installation	37
Beskrivning	37
Java för Jin23	38
Rider installation	38
Addons till Rider som jag rekommenderar	38
Beskrivning	38
Java för Jin23	39
Förslag	39
Java för Jin23	40
Evernote	40
Java för Jin23	41
Githubstudent	41
Fördelar med Github Student Developer Pack	41
Hur man registrerar sig för Github Student Developer Pack	41
Java för Jin23	43
Google Keep	43
Java för Jin23	44
Miro Board	44
Beskrivning	44
Java för Jin23	45
Notion	45
Java för Jin23	46
ai	46
Användning	46
Begränsningar	46
Att tänka på som studerande	46
Java för Jin23	48
Jetbrains-tools	48
Nyttan av JetBrains-verktyg	48
Begränsningar av JetBrains-verktyg	48
Att tänka på som studerande	49
Tips	49
Java för Jin23	50
Microsoft-tools	50
Fördelar med Visual Studio Community och Visual Studio Code	50
Begränsningar av Visual Studio Community och Visual Studio Code	50
Att tänka på som studerande	50
Tips	51
Java för Jin23	52
Github Copilot	52
Användning	52
Exempel på prompts	52
Exempel	52
Nyttan av det	52
Begränsningar	53
Att tänka på som studerande	53
Plagiering / Fusk	53
Java för Jin23	54
Tabnine	54
Användning	54
Begränsningar	54
Att tänka på som studerande	54
Tips	54
Java för Jin23	55
Chatgpt	55
Vad är ChatGPT?	55

Exempel på prompts	55
Användning	56
1. Få inspiration till nya projekt:	56
2. Använd ChatGPT som en extra lärare:	56
3. Hantera kodgenerering:	56
Begränsningar	56
Att tänka på som studerande	56
Extra prompts	57
Länkar	57
Java för Jin23	58
Bard	58
Vad är Bard?	58
Användning	58
Exempel på prompts	58
Begränsningar	59
Att tänka på som studerande	59
Plagiering / Fusk	59
Att använda Bard för att få inspiration till nya projekt	59
Att använda Bard som en extra lärare	59
Tips	60
Java för Jin23	62
If	62
Introduktion	62
Exempel	62
Fördelar	62
Begränsningar	62
Användningsområden	62
TL;DR	63
Obligatorisk dad joke	63
Java för Jin23	64
Else	64
Beskrivning	64
Exempel	64
Sammanfattning	64
Termer och förklaringar	64
Obligatorisk dad-joke	65
Java för Jin23	66
Else if	66
Användningsområden	66
Kodexempel	66
Betygsberäkning	66
Validering	66
Menyval	67
Datum- och tidshantering	67
Sammanfattning	67
Java för Jin23	69
Ternary if	69
Exempel	69
Slutsats	69
Mer läsning	70
Termer	70
Obligatorisk pappaskämt	70
Java för Jin23	71
Variabler	71
Exempel	71
Java för Jin23	72
Typer	72
Primitiva typer	72
Referenstyper	72
Termer	73
TL;DR-sammanfattning	73
Java för Jin23	74
Datastrukturer	74

Beskrivning	74
Array	74
List	74
Stack	75
Queue	75
Dictionary	75
HashSet	75
Ännu fler datastrukturer	75
Referenser	76
Java för Jin23	77
Array övningar	77
1. Skapa en array	77
2. Tilldela värden till en array	77
3. Hämta värden från en array	77
4. Uppdatera värden i en array	77
5. Loopa igenom en array	77
6. Array av objekt	78
7. Tvådimensionell array	78
8. Sök efter ett värde i en array	78
9. Sortera en array	78
10. Summera värden i en array	78
Java för Jin23	80
Plocka ut en del av en array och skapa en ny array av det.	80
Plocka ut en del av en array och skapa en ny array av det	80
Kodexempel	80
Resultat	80
Java för Jin23	82
Summera element i en array	82
Beskrivning av övningen	82
Kodmall	82
Förväntad output	82
Facit	82
Java för Jin23	83
Hitta det största elementet i en array	83
Kodmall	83
Förväntad output	83
Facit	83
Java för Jin23	84
Invertera en array	84
Beskrivning av övningen	84
Kodmall	84
Förväntad output	84
Facit	84
Java för Jin23	86
Arrayer	86
Exempel	86
Slutsats	87
Obligatorisk dad-joke	87
Java för Jin23	88
Dictionary (HashMap)	88
Beskrivning	88
Exempel	88
Slutsats	89
TL;DR	89
Java för Jin23	91
Loopar	91
Typer av loopar	91
Mer delar av loopar	91
Java för Jin23	92
Loopar i Java	92
Introduktion	92

Fördelar	92
Begränsningar	92
Användningsområden	92
Typer av loopar i Java	92
Exempel	93
Slutsats	93
TL;DR	93
Termer	93
Obligatorisk dad joke	93
Java för Jin23	95
For loopar	95
For-loop	95
Fördelar	95
Nackdelar	95
Java för Jin23	97
Foreach-loop	97
Fördelar	97
Nackdelar	97
Java för Jin23	98
Nästlade loopar	98
Java för Jin23	99
While	99
Fördelar	99
Begränsningar	99
Användningsområden	100
Exempel	100
Sammanfattning	100
Java för Jin23	101
Do While	101
Beskrivning	101
Exempel	101
Sammanfattning	101
Termer	102
Java för Jin23	103
Rekursiva loopar	103
Rekursion i Java	104
Obligatorisk dad-joke	104
Java för Jin23	105
Evig loop	105
Fördelar	105
Nackdelar	105
Exempel	105
Exempel på oändlig loop	106
Java för Jin23	107
Loop kontroller	107
Break	107
Continue	107
Return	108
yield	108
Java för Jin23	110
Objektorienterad programmering (OOP)	110
När du har läst detta kommer du att kunna	110
Innehållsförteckning	110
Introduktion	110
Fördelar	110
Begränsningar	111
Tillämpningsområden	111
Exempel	111
Slutsats	113
TL;DR	113

Java för Jin23	114
Klasser och Objekt inom programmering med Java	114
TL;DR	114
När du läst detta ska du kunna	114
Introduktion	114
Vad är Klasser och Objekt?	114
Klassers namn	115
Fördelar	116
Begränsningar	117
Användningsområden	117
Exempelkod	117
Output	118
Slutsats	119
Obligatorisk dad joke	119
Java för Jin23	120
Klasskomposition	120
Fördelar	120
Begränsningar	120
Exempel på användning	120
Kodexempel	121
Output	122
Slutsats	122
Java för Jin23	123
Inkapsling	123
Vad är inkapsling?	123
Begränsningar med inkapsling	123
Användningsområden för inkapsling i Java	124
Exempel på inkapsling i Java	124
Ett annat exempel	125
Sammanfattning	127
Obligatorisk dad-joke	127
Java för Jin23	128
Java för Jin23	132
Komposition över arv	132
Vad är det för något?	132
Varför är det viktigt?	132
Fördelar	132
Begränsningar	132
Enkel förklaring	133
Exempel 1	133
Exempel 2	135
Sammanfattning	136
Java för Jin23	137
Åtkomstmoderator	137
Java för Jin23	138
Private	138
Beskrivning	138
Exempel	138
Java för Jin23	139
Protected	139
Beskrivning	139
Exempel	139
Förklaring	139
Obligatorisk Dad-Joke	139
Java för Jin23	140
Public	140
Beskrivning	140
Exempel	140
Förklaring	140
Getters och setters?	140
Exempel	141

Sammanfattning	141
Java för Jin23	142
Static	142
Beskrivning	142
Exempel	142
Obligatorisk Dad-joke	143
Java för Jin23	144
Polymorfism	144
Beskrivning	144
Java för Jin23	145
Java för Jin23	147
Plugins	147
Fördelar med att använda Plugins	147
Hur man implementerar ett pluginsystem med gränssnitt (interfaces)	147
Steg 1: Definiera ett gränssnitt för Plugins	147
Steg 2: Implementera Plugins	148
Steg 3: Ladda och köra Plugins	148
Slutsats	149
Användningsområden för Pluginsystem	149
Java för Jin23	150
Abstrakta klasser	150
Beskrivning	150
Exempel	150
Referenser	152
Java för Jin23	153
Exempel	153
Beskrivning	153
Fördelar	153
Begränsningar	153
Användningsområden	153
Kodexempel	154
Termer	157
Slutsats	157
Referenser	158
Java för Jin23	159
delegater	159
Java för Jin23	160
Exempel	160
Java för Jin23	162
Events	162
Java för Jin23	164
Exempel	164
Java för Jin23	168
Filhantering	168
Skapa en klass	168
Läsa en textfil	169
Skriva till en textfil	169
Läsa och skriva till en Json-fil	169
Läsa och skriva binära filer	170
Läsa och skriva binära filer med Base64	170
Läsa och skriva binära filer med Base64 och GZIP	170
Övrig filhantering	171
Kolla om en fil finns	171
Skapa en tom fil om den inte finns	171
Skapa en mapp om den inte finns	171
Radera en fil	171
Radera en mapp om den är tom	171
Flytta en fil	171
Lista alla filer i en mapp	172
Lista alla filer i en mapp och dess undermappar	172
Hitta en fil som med namn som innehåller "Katt"	172

Hitta en fil som med namn som innehåller "Hemlighet" och radera den	172
Hitta en fil med texten "Lösenord" i sig	172
Få information om filen	173
Termer och förklaringar	173
Sammanfattning	175
Java för Jin23	176
CSV	176
Fördelar	176
Begränsningar	176
Användningsområden	177
Kodexempel	177
Läsning av CSV-fil	177
Skrivning till CSV-fil	177
Kodförklaringar	178
Files.readAllLines	178
String.split	179
StringBuilder	179
StringBuilder.append	179
Files.write	179
Slutsats	179
Termer	179
Länkar	179
Java för Jin23	180
File-klassen	180
TL;DR	180
Beskrivning	180
Exempel	181
Slutsats	181
Termer	182
Java för Jin23	183
Json	183
Gson-biblioteket	183
Skapa JSON-manuellt med StringBuilder	184
Läsa JSON-data	184
Fördelar och nackdelar med JSON	185
Fördelar:	185
Nackdelar:	185
Varför använda Gson istället för org.json?	185
Termer och förklaringar	185
Obligatorisk Dad Joke	186
Java för Jin23	187
Mappar	187
Sammanfattning	188
Obligatorisk Dad-joke	188
Java för Jin23	189
Text	189
Skapa en textfil	189
Läs in en textfil	189
Spara en lista i en textfil	189
Läs in en lista från en textfil	190
Lägg till text i en textfil	190
Skriv över en textfil	191
Radera en textfil	191
Läs in en textfil från en URL	191
Spara en textfil från en URL	192
Lista på termer i artikeln	192
Obligatorisk Dad-joke	192
Java för Jin23	193
Path-klassen	193
Java för Jin23	195
Asynkron	195
TL;DR	195
När du läst detta ska du kunna	195

Introduktion	195
Beskrivning	195
När ska man använda asynkrona metoder?	196
Sammanfattning	196
Java	196
Fördelar	196
Begränsningar	196
Användningsområden	197
Exempelkod - Asynkrona anrop (Java)	197
Slutsats	198
Termtabell	198
Obligatorisk Dad-joke	198
Java för Jin23	199
Exempel	199
Kod	199
Slutsats	201
Termtabell	201
Obligatorisk Dad-joke	201
Java för Jin23	202
API	202
TL; DR	202
Fördjupning	202
Exempel	202
Referenser	203
Obligatorisk Dad-joke	204
Java för Jin23	205
Övrigt	205
Java för Jin23	206
SQL	206
Java för Jin23	207
Tabeller	207
Lägg till en kolumn	207
Ta bort en kolumn	207
Referenser	207
Lägg till en kolumn och gör den till en ForeignKey	207
Ta bort en ForeignKey	207
Skapa en tabell med en ForeignKey	208
Skapa en tabell med en ForeignKey och en primärnyckel	208
Referenser	208
Java för Jin23	209
Databaser	209
Skapa en databas om den inte finns	209
Ta bort en databas	209
Ändra namn på databasen	209
Java för Jin23	210
Constraints	210
Foreign Key	210
Exempel	210
Exempel på hur man kopplar ihop tabellerna	210
Referenser	211
Java för Jin23	212
Ordlista	212
Java för Jin23	213
Git	213
Java för Jin23	214
Programmering	214
Java för Jin23	215
Programmeringsspråk	215
Java för Jin23	216

Webb	216
Java för Jin23	217
Yrken	217
References	218

Java för Jin23

Marcus Medina, Campus Mölndal

2023-07-22 12:18:14

Innehållsförteckning

Home

1. Välkommen

Java

Skolan

1. Här är skolans webbplatser

Skolportalen

Google Classroom

Google Calendar

Google Documents

Discord

Info

Versionshistorik

Om sidan

MIT-licens för Just the docs

Verktyg

1. 2. Byggverktyg
2. 3. Versionshanteringssystem
3. 4. Enhetstestningsramverk
4. Summering

Installation

Git installation

Visual Studio installation

1. Beskrivning

VS Code installation

1. Beskrivning

IntelliJ installation

1. Beskrivning

NetBeans installation

1. Beskrivning

Rider installation

1. Beskrivning

Förslag

Evernote

Githubstudent

Google Keep

Miro Board

1. Beskrivning

Notion

ai

1. Användning
2. Begränsningar
3. Att tänka på som studerande

Jetbrains-tools

1. Nyttan av JetBrains-verktyg
2. Att tänka på som studerande
3. Tips

Microsoft-tools

1. Att tänka på som studerande
2. Tips

Github Copilot

1. Användning
2. Exempel på prompts
 1. Exempel
3. Nyttan av det
4. Begränsningar
5. Att tänka på som studerande
 1. Plagiering / Fusk

Tabnine

1. Användning
2. Begränsningar
3. Att tänka på som studerande
4. Tips

Chatgpt

1. Vad är ChatGPT?
2. Exempel på prompts
3. Användning
 1. 3. Hantera kodgenerering:
4. Begränsningar
5. Att tänka på som studerande
6. Extra prompts
7. Länkar

Bard

1. Vad är Bard?
2. Användning
3. Exempel på prompts
4. Begränsningar
5. Att tänka på som studerande
 1. Plagiering / Fusk
6. Tips

If

1. Introduktion
2. Exempel
3. Fördelar
4. Begränsningar
5. Användningsområden
6. TL;DR
7. Obligatorisk dad joke

Else

1. Beskrivning
2. Exempel
3. Sammanfattning
4. Termer och förklaringar
5. Obligatorisk dad-joke

Else if

1. Användningsområden
2. Kodexempel
 1. Betygsberäkning
 2. Validering
 3. Menyval
 4. Datum- och tidshantering
3. Sammanfattning

Ternary if

1. Exempel
2. Slutsats
3. Mer läsning
4. Termer
5. Obligatorisk pappaskämt

Variabler

1. Exempel

Typer

1. Primitiva typer
2. Referenstyper
3. Termer
4. TL;DR-sammanfattning

Datastrukturer

1. Beskrivning
2. Array
3. List
4. Stack
5. Queue
6. Dictionary
7. HashSet
8. Ännu fler datastrukturer
9. Referenser

Array övningar

1. Skapa en array
2. Hämta värden från en array
3. Uppdatera värden i en array
4. Loopa igenom en array
5. Array av objekt
6. Tvådimensionell array
7. Sortera en array
8. 10. Summera värden i en array

Summera element i en array

1. Beskrivning av övningen
2. Kodmall

Invertera en array

1. Beskrivning av övningen
2. Kodmall
3. Förväntad output
4. Facit

Arrayer

1. Exempel
2. Slutsats
3. Obligatorisk dad-joke

Dictionary (HashMap)

1. Beskrivning
2. Exempel
3. Slutsats
4. TL;DR

Loopar

1. Typer av loopar
2. Mer delar av loopar

Loopar i Java

1. Introduktion
2. Fördelar
3. Begränsningar
4. Användningsområden
5. Typer av loopar i Java
6. Exempel
7. Slutsats
8. TL;DR
9. Termer
10. Obligatorisk dad joke

For loopar

1. Fördelar
2. Nackdelar

Foreach-loop

1. Fördelar
2. Nackdelar

Nästlade loopar

While

1. Fördelar
2. Begränsningar
3. Användningsområden
4. Exempel
5. Sammanfattning

Do While

1. Beskrivning
2. Exempel
3. Sammanfattning
4. Termer

Rekursiva loopar

1. Rekursion i Java
2. Obligatorisk dad-joke

Evig loop

1. Fördelar
2. Nackdelar
3. Exempel
4. Exempel på oändlig loop

Loop kontroller

1. Break
2. Continue
3. Return
4. yield

Klasskomposition

1. Fördelar
2. Begränsningar
3. Exempel på användning
4. Kodexempel
 1. Output
5. Slutsats

Inkapsling

1. Vad är inkapsling?
2. Begränsningar med inkapsling
3. Exempel på inkapsling i Java
4. Ett annat exempel
5. Sammanfattning
6. Obligatorisk dad-joke

Komposition över arv

1. Vad är det för något?
2. Varför är det viktigt?
3. Fördelar
4. Begränsningar
5. Enkel förklaring
6. Exempel 1
7. Exempel 2
8. Sammanfattning

Åtkomstmoderator

Private

1. Beskrivning
2. Exempel

Protected

1. Beskrivning
2. Exempel
3. Förklaring
4. Obligatorisk Dad-Joke

Public

1. Beskrivning
2. Exempel
3. Förklaring
4. Getters och setters?
5. Exempel
6. Sammanfattning

Static

1. Beskrivning
2. Exempel
3. Obligatorisk Dad-joke

Polymorfism

1. Beskrivning

Plugins

1. Slutsats

Abstrakta klasser

1. Beskrivning
2. Exempel
3. Referenser

Exempel

1. Beskrivning
2. Fördelar
3. Begränsningar
4. Användningsområden
5. Kodexempel
6. Termer
7. Slutsats
8. Referenser

delegater

Exempel

Events

Exempel

Filhantering

1. Skapa en klass
2. Läsa en textfil
3. Skriva till en textfil
4. Läsa och skriva binära filer
5. Övrig filhantering
 1. Kolla om en fil finns
6. Radera en fil
7. Radera en mapp om den är tom
8. Flytta en fil
9. Lista alla filer i en mapp
10. Få information om filen
11. Termer och förklaringar
12. Sammanfattning

CSV

1. Fördelar
2. Begränsningar
3. Användningsområden
4. Kodexempel
 1. Läsning av CSV-fil
 2. Skrivning till CSV-fil
5. Kodförklaringar
 1. Files.readAllLines
 2. String.split
 3. StringBuilder
 4. StringBuilder.append
 5. Files.write
6. Slutsats
7. Termer
8. Länkar

File-klassen

1. TL:DR
2. Beskrivning
3. Exempel
4. Slutsats
5. Termer

Json

1. Gson-biblioteket
2. Läsa JSON-data
 1. Fördelar
 2. Nackdelar
3. Termer och förklaringar
 1. Obligatorisk Dad Joke

Mappar

1. Obligatorisk Dad-joke

Text

1. Skapa en textfil
2. Läs in en textfil
3. Spara en lista i en textfil
4. Lägg till text i en textfil
5. Skriv över en textfil
6. Radera en textfil
7. Spara en textfil från en URL
8. Lista på termer i artikeln
9. Obligatorisk Dad-joke

Path-klassen

Asynkron

1. TL;DR
2. Introduktion
3. Beskrivning
4. Sammanfattning
5. Java
6. Fördelar
7. Begränsningar
8. Användningsområden
9. Slutsats
10. Termtabell
11. Obligatorisk Dad-joke

Exempel

1. Kod
2. Slutsats
3. Termtabell
4. Obligatorisk Dad-joke

API

1. TL;DR
2. Fördjupning
3. Exempel
4. Referenser
5. Obligatorisk Dad-joke

Övrigt

SQL

Tabeller

1. Referenser
2. Ta bort en ForeignKey
3. Referenser

Databaser

Constraints

1. Foreign Key
2. Exempel
3. Referenser

Ordlista

Git

Programmering

Programmeringsspråk

Webb

Yrken

Java för Jin23

Marcus Medina

Home

Välkommen

Home

Välkommen

Välkommen till Campus Mölndals fantastiska elektroniska bok för JIN23. Här hittar du allt du behöver för att komma igång med programmering i Java.

Java är ett objektorienterat programmeringsspråk som utvecklades av Sun Microsystems (numera ägt av Oracle). Det används ofta för att skapa programvaror och plattformar som sträcker sig från små mobila enheter till stora datorsystem. Java har blivit populärt på grund av sin plattformsoberoende natur och förmåga att köra på olika operativsystem.

Java har en enkel och lättförståelig syntax som gör det till ett idealiskt språk för nybörjare. Samtidigt erbjuder det avancerade funktioner och bibliotek för att hantera komplexa program. Generiska typer, anonyma inre klasser, lambda-uttryck och trådhantering är några av de kraftfulla funktionerna som Java erbjuder.

En av de stora fördelarna med Java är dess stora ekosystem av bibliotek och ramverk. Det finns ett brett utbud av bibliotek för att hjälpa till med allt från grafisk användargränssnittsutveckling till databashantering och nätverksprogrammering. Ramverk som Spring och Hibernate förenklar utvecklingsprocessen och främjar goda designmönster.

Java används inom en mängd olika områden, inklusive webbutveckling, mobilapputveckling, spelutveckling och företagsapplikationer. Det är ett mångsidigt språk som ger utvecklare möjlighet att skapa kraftfulla och skalbara program.

Java för Jin23

Marcus Medina

Java

Java

Java är ett objektorienterat programmeringsspråk som utvecklades av Sun Microsystems. Det används ofta för att skapa applikationer för olika plattformar. Java är ett modern, flexibel och mångsidig programmeringsspråk som ger utvecklare möjlighet att skapa allt från enkla webbapplikationer till komplexa distribuerade system.

Java har en stark och tydlig syntax som gör det lätt att lära sig och använda. Språket har ett brett utbud av funktioner som hjälper utvecklare att skapa avancerade system. Funktioner som generiska typer, anonyma metoder, lambda-uttryck och parallell programmering gör Java till ett mycket kraftfullt programmeringsspråk. Java stöder också många olika typer av programutveckling, inklusive webbappar, mobilappar, spel och systemprogramvara.

Och nu är tanken att du ska lära dig det, bli en riktig grym kodare och få ett jobb som utvecklare. Har du kommit så långt som till att du läser detta så är du reda på god väg!

Som Mary Poppins sa "*Well Begun Is Half Done!*".

Ett annat citat från Mary Poppins är "*In every job that must be done there is an element of fun. You find the fun and snap, the job's a game!*" och det är precis vad vi ska göra här. Vi ska ha kul och lära oss Java samtidigt.

Så låt oss börja!

Java för Jin23

Marcus Medina

Skolan

Här är skolans webbplatser

Skolan

Här är en lista på webbtjänster vi använder i skolan.

Här är skolans webbplatser

- Campus Mölndals hemsida
- LinkedIn
- Instagram
- Facebook
- Marcus Medina
- Annika Lund
- #CampusMölndal

LinkedIn är ett måste när man vill skapa sig en karriär. Så tveka inte och skapa ett konto. Det är gratis och du kan skapa ett konto med din skolmail.

Java för Jin23

Marcus Medina

Skolportalen

Skolportalen

Öppna skolportalen

Skolportalen är din ingång till skolan och dina kurser. Här kan du se dina betyg, läsa om dina kurser och se dina scheman. Här finns även länkar till Google Classroom och Google Drive. Plus en massa annat.

Java för Jin23

Marcus Medina

Google Classroom

Google Classroom

Öppna Google Classroom{: .btn .btn-blue }

Vi använder Google Classroom på Campus Mölndal.

I ditt klassrum hittar du information, länkar och planering för kursen. Du kan också lämna in uppgifter och inlämningar där.

Varje kurs har sin egen Meet kanal där.

Java för Jin23

Marcus Medina

Google Calendar

Google Calendar

Öppna Google Calendar{: .btn .btn-blue }

Lektioner, tentor, inlämningar och annat skoj hamnar i klassens google calendar. Du kan prenumerera på kalendern och få alla händelser direkt i din telefon.

Java för Jin23

Marcus Medina

Google Documents

Google Documents

Öppna Google Documents{: .btn .btn-blue }

Vi använder Google Documents för att skriva uppsatser, föra anteckningar och annat. Då du får ett Google konto av skolan kan du samla all din material där. Det är också enkelt att dela med sig av dokument till andra.

Java för Jin23

Marcus Medina

Discord

Discord

Installera Discord{: .btn .btn-blue }

När du blir intagen till kursen får du en länk till en Discordgrupp som är skapad för klassen.

I den postas det information som behöver komma ut snabbt, exempelvis påminnelser om möten och annat. Där sker även det mesta av online kommunikationen mellan studenter och utbildare.

(Email är så 90tal...)

Java för Jin23

Marcus Medina

Info

Versionshistorik

Info

Här kommer jag att posta information om sidan, som t.ex. versionshistorik, användarvillkor, hur man använder den mm. Denna sidan är baserad på CLO22 som är en sida som jag skapat för att samla information om C# och .NET. All information "översattes" till Java.

Versionshistorik

Version	Datum	Beskrivning
1.0.1	2022-12-09	Uppdatering av sidan. Mermaid funkar inte, diagrammen syns inte :’(
1.0.0	2022-12-09	Första publicering. Första publicering,

Java för Jin23

Marcus Medina

Om sidan

Om sidan

Denna sidan skapades för att kunna samla alla dokument på en plats, för att göra det enklare för studenter att hitta material från lektioner och annat.

Skicka gärna in förslag på förbättringar eller nya dokument till mig.

Ni hittar mig på Discord eller skolans mail.

/Marcus

Java för Jin23

Marcus Medina

Licens för materialet på denna webbsida

Sammanfattning
Enkelt förklarat

Licens för materialet på denna webbsida

Materialet på denna webbsida är licensierat under en Creative Commons BY-NC-SA (Attribution-NonCommercial-ShareAlike) licens. Det innebär att du fritt får använda materialet för **icke-kommersiella ändamål**, under förutsättning att du **anger källan** och delar **ditt material under samma licens**.

Denna licens har valts för att ge studenterna friheten att använda koden på det sätt de önskar, samtidigt som materialet inte tillåts kopieras eller användas i kommersiella syften av andra utbildningscenter än Campus Mölndal.

För mer information om Creative Commons BY-NC-SA-licensen och dess villkor kan du besöka Creative Commons webbplats.

Sammanfattning

- **Licens:** Creative Commons BY-NC-SA (Attribution-NonCommercial-ShareAlike)
- **Användning:** Materialet får användas för icke-kommersiella ändamål.
- **Krav:** Du måste ange källan och dela ditt material under samma licens.
- **Undantag:** Materialet får inte kopieras eller användas i kommersiella syften av andra utbildningscenter än Campus Mölndal.

Vänligen observera att licensvalet är noggrant genomtänkt för att uppnå de önskade användningsvillkoren för materialet. Vid eventuella tvivel eller frågor rekommenderas det att du konsulterar en juridisk expert för att säkerställa att licensen är korrekt och uppfyller dina behov och jurisdiktionens krav.

Enkelt förklarat

Du får: + Använda koden för icke-kommersiella ändamål, under förutsättning att du anger källan. + Använda koden i ett projekt och dela det under samma licens.

Du får inte: + Kopiera koden eller artiklarna för spridning. + Använda koden eller artiklar i kommersiella syften av andra utbildningscenter/företag än Campus Mölndal.

Java för Jin23

Marcus Medina

MIT-licens för Just the docs

MIT-licens för Just the docs

Upphovsrätt (c) 2023 Marcus Medina, Campus Mölndal

Härmed beviljas tillstånd, kostnadsfritt, till varje person som erhåller en kopia av denna programvara och tillhörande dokumentationsfiler ("Programvaran"), att använda Programvaran utan begränsningar, inklusive, utan begränsning, rättigheter att använda, kopiera, ändra, sammanfoga, publicera, distribuera, underlicensiera och/eller sälja kopior av Programvaran samt att tillåta personer som erhåller Programvaran att göra detta, under förutsättning att följande villkor uppfylls:

Ovanstående upphovsrättsnotis och denna licensnotis ska inkluderas i alla kopior eller betydande delar av Programvaran.

PROGRAMVARAN TILLHANDAHÅLLS "SOM DEN ÄR", UTAN GARANTI AV NÅGOT SLAG, UTTRYCKT ELLER UNDERFÖRSTÅDD, INKLUSIVE MEN INTE BEGRÄNSAT TILL GARANTIER OM SÄLJBARHET, LÄMPLIGHET FÖR ETT SPECIFIKT ÄNDAMÅL OCH OÖVERTRÄDELSE AV IMMATERIELLA RÄTTIGHETER. I INGA HÄNDELSER SKA UPPHOVSRÄTTSHAVERNA ELLER UPPHOVSRÄTTENS INNEHAVARE ANSVARA FÖR SKADOR ELLER ANDRA FORDRINGAR, OAVSETT OM DET GÄLLER AVTALSBROTT, SKADESTÅND ELLER ANNAT, SOM UPPKOMMER AV, UTAN BEGRÄNSNING, I SAMBAND MED PROGRAMVARAN ELLER ANVÄNDNINGEN AV ELLER ANDRA VERKSAMHETER MED PROGRAMVARAN.

- MIT-licens (engelska)

Java för Jin23

Marcus Medina

Verktyg

1. IDE (Integrated Development Environment)
 2. Byggverktyg
 3. Versionshanteringssystem
 4. Enhetstestningsramverk
 5. Felsökning och profileringsverktyg
- Summering

Verktyg

I denna artikel kommer vi att titta på några användbara verktyg för att underlätta utvecklingen av Java-program. Genom att använda dessa verktyg kan vi öka vår produktivitet och förbättra kvaliteten på vår kod. Det är viktigt att välja verktyg som bäst passar våra behov och att utforska och använda dem på rätt sätt.

1. IDE (Integrated Development Environment)

En IDE är en programvara som ger oss en miljö för att skriva, kompilera och testa våra Java-program. Här är några populära IDE:er för Java:

- **Eclipse:** En kraftfull och flexibel IDE som är mycket populär bland Java-utvecklare. Den har omfattande funktioner och stöd för att underlätta utvecklingsprocessen.
- **IntelliJ IDEA:** En annan populär IDE som erbjuder avancerade funktioner och verktyg för att öka produktiviteten. Den har också stöd för andra programmeringsspråk.
- **NetBeans:** En lättanvänd IDE som är utvecklad specifikt för Java-programmering. Den har en intuitiv användargränssnitt och erbjuder en rad användbara funktioner.

2. Byggverktyg

Byggverktyg hjälper oss att automatisera byggprocessen för våra Java-projekt. De hanterar kompilering, beroenden, testning och paketering av vår kod. Här är några populära byggverktyg för Java:

- **Maven:** Ett kraftfullt verktyg för att hantera projektberoenden och automatisera byggprocessen. Det använder en XML-baserad konfigurationsfil för att definiera projektets struktur och beroenden.
- **Gradle:** Ett flexibelt byggverktyg som erbjuder en DSL (Domain-Specific Language) för att definiera byggsript. Det stöder också Maven-repositorier och har enkla och läsbara konfigurationsfiler.

3. Versionshanteringssystem

Ett versionshanteringssystem hjälper oss att hantera ändringar i vår kod och samarbeta med andra utvecklare. Det spårar ändringar, hanterar konflikter och gör det möjligt för oss att återställa tidigare versioner av vår kod. Här är några populära versionshanteringssystem för Java:

- **Git:** Ett distribuerat versionshanteringssystem som är mycket populärt och används av många utvecklare runt om i världen. Det erbjuder enkla och kraftfulla kommandon för att hantera ändringar och samarbeta med andra.

- **Subversion:** Ett centralt versionshanteringssystem som används av många företag och organisationer. Det erbjuder enkla kommandon och en webbgränssnitt för att hantera ändringar och samarbeta med andra.

4. Enhetstestningsramverk

Enhetstestning är en viktig del av utvecklingsprocessen för att säkerställa att vår kod fungerar som förväntat. Ett enhetstestningsramverk hjälper oss att skriva och köra tester för vår Java-kod. Här är några populära enhetstestningsramverk för Java:

- **JUnit:** Ett robust och populärt enhetstestningsramverk för Java. Det erbjuder enkla och kraftfulla funktioner för att skriva och köra tester.
- **TestNG:** Ett flexibelt och utökat enhetstestningsramverk för Java. Det erbjuder avancerade funktioner som parallell körning av tester och konfiguration av testfall.

5. Felsökning och profileringsverktyg

Felsökning och profilering är viktiga för att hitta och åtgärda problem i vår Java-kod. Här är några användbara verktyg för felsökning och profilering i Java:

- **Eclipse Memory Analyzer:** Ett verktyg för att analysera minnesdumpar och hitta minnesläckor och andra minnesrelaterade problem.
- **VisualVM:** Ett verktyg för att övervaka och profilera Java-program i realtid. Det ger detaljerad information om minnesanvändning, CPU-användning och andra prestandarelaterade mätvärden.

Summering

Att använda rätt verktyg kan göra en stor skillnad i vår utvecklingsprocess. I denna artikel har vi tittat på några användbara verktyg för att underlätta utvecklingen av Java-program. Genom att välja och använda dessa verktyg på rätt sätt kan vi öka vår produktivitet och förbättra kvaliteten på vår kod.

Java för Jin23

Marcus Medina

Installation

Installation

Här finns en lista på program du kan komma att behöva under utbildningens gång.

Filen editerades senast 2022-12-09

Java för Jin23

Marcus Medina

Git installation

Git installation

Installera Git

Filen editerades senast 2022-12-09 ## Beskrivning

Git är ett versionhanteringssystem som används för att spara och hantera källkod. Det är ett av de mest använda versionhanteringssystemen i dagens samhälle. Det är ett gratis program som är öppen källkod. Den här installationen är för Windows. För att installera Git på Mac eller Linux, gå till Git installation. Du behöver inte den om du bara arbetar med Visual Studio, men ska du in på Console / Terminalen så behöver du det.

Java för Jin23

Marcus Medina

Visual Studio installation
Beskrivning

Visual Studio installation

Installera Visual Studio

Filen editerades senast 2022-12-09 ## Addons till Visual Studio som jag rekommenderar

- Codemaid
- Roslynator
- Github Copilot
- Smart paste
- Add new page

Beskrivning

Visual Studio är ett IDE för Java och .NET. Det är främst den som används när man arbetar med Java.

Java för Jin23

Marcus Medina

VS Code installation

Beskrivning

VS Code installation

Installera Visual Studio Code

Filen editerades senast 2022-12-09 ## Addons till VS Code som jag rekommenderar

- Roslynator
- [Github Copilot]

Beskrivning

VS Code är ett IDE för Java och .NET. Det är ett gratis program som är öppen källkod. Den är mer anpassningsbar än Visual Studio och den har inte lika många funktioner som Visual Studio. Den är mer anpassad för webbutveckling än Visual Studio.

Java för Jin23

Marcus Medina

IntelliJInstallation

Addons till IntelliJ som jag rekommenderar

Beskrivning

IntelliJInstallation

Installera JetBrains IntelliJ

Addons till IntelliJ som jag rekommenderar

- Github Copilot

Beskrivning

JetBrains IntelliJ IDEA är ett IDE för Java och andra programmeringsspråk. Den erbjuder både en gratisversion (Community Edition) och en betald version (Ultimate Edition) med fler funktioner. Det är ett mycket populärt val bland Java-utvecklare tack vare dess omfattande funktioner och användarvänliga gränssnitt.

Java för Jin23

Marcus Medina

NetBeans installation

Beskrivning

NetBeans installation

Installera NetBeans

Beskrivning

Apache NetBeans är ett gratis och öppen källkods IDE för Java och andra programmeringsspråk som PHP, JavaScript, och C/C++. NetBeans erbjuder en komplett uppsättning funktioner för professionell utveckling, inklusive debugger, profiler, refactoringverktyg, statisk analys, versionhantering och mycket mer. Dess användarvänliga gränssnitt och omfattande hjälpmaterial gör det till ett bra val för både nybörjare och erfarna utvecklare.

Java för Jin23

Marcus Medina

Rider installation

Addons till Rider som jag rekommenderar

Beskrivning

Rider installation

Installera JetBrains Rider

Addons till Rider som jag rekommenderar

- Github Copilot

Beskrivning

JetBrains Rider är ett IDE för Java och .NET. Det är ett gratis program som är öppen källkod. Den är som Visual Studio Code och den har JetBrains Resharper inbyggt.

Java för Jin23

Marcus Medina

Förslag

Förslag

Här finns förslag på andra program som kan vara bra att använda under studietiden.

Filen editerades senast 2022-12-09

Java för Jin23

Marcus Medina

Evernote

Evernote

Installera Evernote

Evernote är ett program som kan användas för att skriva anteckningar, skapa listor och göra anteckningar om bilder. Det är enkelt att dela med sig av anteckningar till andra. Det finns även en mobilapp som gör det enkelt att skriva anteckningar på språng. Det är gratis att skapa ett konto och använda programmet. Men vill man ha mer utrymme eller fler funktioner så får man skaffa ett betalkonto. Studenter kan få 40 % rabatt på ett årsabonnemang på Evernote Personal.. Det finns även en webbversion som du kan använda om du inte vill installera programmet på din dator.

Filen editerades senast 2022-12-09

Java för Jin23

Marcus Medina

Githubstudent

Fördelar med Github Student Developer Pack

Hur man registrerar sig för Github Student Developer Pack

Visst, här är en artikel om Github Student Developer Pack:

Githubstudent

Github Student Developer Pack är ett gratispaket som ger studenter tillgång till ett brett utbud av verktyg och tjänster för att hjälpa dem att lära sig och utveckla sina kodningskunskaper. Paketet inkluderar:

- Github Pro-konto: Detta ger studenter tillgång till premiumfunktioner som privat lagring, 2FA och 100 GB lagring.
- GitHub Codespaces: Detta ger studenter möjlighet att utveckla kod i molnet utan att behöva installera något programvara på sin dator.
- GitHub Sponsors: Detta ger studenter möjlighet att få sponsring från företag och individer för deras kodningsprojekt.
- GitHub Education Cloud: Detta ger studenter tillgång till en mängd olika utbildningsresurser, inklusive kurser, handledningar och exempelkod.

Github Student Developer Pack är ett utmärkt sätt för studenter att lära sig och utveckla sina kodningskunskaper. Paketet är gratis och innehåller en mängd olika verktyg och tjänster som kan hjälpa studenter att bli bättre kodare.

Fördelar med Github Student Developer Pack

Det finns många fördelar med att använda Github Student Developer Pack. Några av fördelarna inkluderar:

- Gratis: Github Student Developer Pack är helt gratis, vilket gör det till ett mycket prisvärt erbjudande för studenter.
- Bredt utbud av verktyg och tjänster: Github Student Developer Pack innehåller ett brett utbud av verktyg och tjänster som kan hjälpa studenter att lära sig och utveckla sina kodningskunskaper.
- Enkelt att använda: Github Student Developer Pack är mycket enkelt att använda, vilket gör det till ett bra val för studenter som är nya på kodning.
- Stor community: Github har en stor community av användare och utvecklare, vilket gör det till ett bra ställe att lära sig och få hjälp med kodning.
- Gratis Github Copilot

Hur man registrerar sig för Github Student Developer Pack

För att registrera dig för Github Student Developer Pack måste du:

Du får mailadress från skolan, använd den för registrering och koppla sedan ditt privata konto till den du registrerat för Github med skolans konto.

1. Ha en giltig studentmejladress.
2. Besök Githubs webbplats och klicka på "Registrera dig för studentkonto".
3. Ange din studentmejladress och lösenord.
4. Bekräfta din registrering genom att klicka på länken i det e-postmeddelande som du får från Github.
5. En gång du har registrerat dig kan du börja använda Github Student Developer Pack.

Om det inte fungerar med mailadressen behöver du använda Studieintyg från skolan, det är bara att fråga om det så mailar vi över den.

Alla Campus Mölndal YH studerande har tillgång till Github Student Developer Pack.

Java för Jin23

Marcus Medina

Google Keep

Google Keep

Installera Google Keep{: .btn }

Filen editerades senast 2022-12-09 ## Beskrivning

Google Keep är ett program som är gjort av Google. Man använder det till att spara anteckningar. Det är och öppen källkod. Det är ungefär som att ha en tavla med post-it lappar.

Java för Jin23

Marcus Medina

Miro Board
Beskrivning

Miro Board

Öppna Miro Board

Beskrivning

Miro är en verktyg för att skapa och dela idéer. Det är ett enkelt sätt att samarbeta med andra. Det är enkelt att skapa olika typer av innehåll, som till exempel flödesscheman, wireframes, moodboards, mindmaps, backlog och mycket mer.

Det finns också en mobilapp som gör det enkelt att skriva anteckningar när man är på språng.

Miro har en presentationsläge som kan användas för att presentera skoluppgifter eller projekt.

Java för Jin23

Marcus Medina

Notion

Notion

Öppna Notion{ : .btn .btn-blue }

Notion är en digital anteckningsbok. Du kan skriva anteckningar, skapa listor och göra anteckningar om bilder. Det är enkelt att dela med sig av anteckningar till andra. Det finns även en mobilapp som gör det enkelt att skriva anteckningar på språng. Vill man ha mer funktionalitet kan man prenumerera på deras tjänst, men den är gratis som pro för studenter.

Filen editerades senast 2022-12-09

Java för Jin23

Marcus Medina

ai

Användning
Begränsningar
Att tänka på som studerande

Visst, här är en artikel om AI-hjälpmedel som är tillgängliga för studerande:

ai

Artificiell intelligens (AI) är ett brett fält inom datavetenskap som handlar om att skapa intelligenta agenter. Intelligenta agenter är system som kan tänka, lära sig och agera självständigt. AI används inom en mängd olika områden, bland annat medicin, finans, transport och utbildning.

Det vi ser mest av nu är Generativ AI, alltså AI som kan skapa saker. Det kan vara allt från att skapa musik, bilder, texter och mycket mer.

Den kan skapa ny information från befintlig information. Den kan göra detta genom att lära sig mönster och strukturer i data. När AI har lärt sig dessa mönster kan den använda dem för att skapa nya exempel som liknar de ursprungliga exemplen.

Ett populärt exempel på AI som skapar ny information är GANs. GANs består av två neurala nätverk som arbetar tillsammans. Ett nätverk (generatorn) skapar nya data, medan det andra nätverket (diskriminatorn) försöker avgöra om de nya datan är äkta eller inte. Genom denna process lär sig generatoren att skapa allt mer realistiska data.

AI som skapar ny information är ett kraftfullt verktyg som har potential att revolutionera många olika branscher. Det är fortfarande under utveckling, men det har redan börjat användas i en mängd olika tillämpningar.

Användning

Det finns många olika sätt att använda AI i utbildning. AI kan användas för att:

- Ge personligt stöd till studerande
- Automatisera uppgifter
- Skapa nya läromedel
- Öka studerandenas engagemang
- Förbättra studerandenas lärande

Begränsningar

AI är fortfarande under utveckling, och det finns vissa begränsningar med AI-hjälpmedel. Till exempel kan AI-hjälpmedel vara dyra, och de kan inte alltid förstå alla studerandes behov. Det är viktigt att vara medveten om dessa begränsningar när man använder AI-hjälpmedel i utbildning.

Att tänka på som studerande

Om du är studerande är det viktigt att tänka på följande när du använder AI-hjälpmedel:

- Använd AI-hjälpmedel för att stödja ditt lärande, inte för att ersätta det.
- Var medveten om begränsningarna med AI-hjälpmedel.

- Använd AI-hjälpmedel för att lära dig på ett sätt som är engagerande och meningsfullt för dig.

AI är ett kraftfullt verktyg som kan hjälpa studerande att lära sig mer effektivt. Genom att vara medveten om AI:s begränsningar och fördelar kan studerande använda AI-hjälpmedel för att förbättra sitt lärande.

Java för Jin23

Marcus Medina

Jetbrains-tools

- Nyttan av JetBrains-verktyg
- Begränsningar av JetBrains-verktyg
- Att tänka på som studerande
- Tips

Jetbrains-tools

JetBrains är en utvecklingsplattform som erbjuder ett brett utbud av verktyg för utvecklare. Dessa verktyg är utformade för att hjälpa utvecklare att vara mer produktiva och effektiva, och de kan användas för att utveckla programvara i en mängd olika språk och plattformar.

JetBrains-verktyg är tillgängliga för både studenter och företag. Studenter kan få en gratis licens för JetBrains-verktyg genom att registrera sig för en JetBrains Education-konto.

Här är några av de mest populära JetBrains-verktygen:

- IntelliJ IDEA: En fullfjädrad integrerad utvecklingsmiljö (IDE) för Java, Kotlin, Scala och andra språk.
- PyCharm: En fullfjädrad IDE för Python.
- WebStorm: En fullfjädrad IDE för HTML, CSS och JavaScript.
- CLion: En fullfjädrad IDE för C och C++.
- Rider: En fullfjädrad IDE för .NET.
- DataGrip: En fullfjädrad IDE för SQL.
- GoLand: En fullfjädrad IDE för Go.
- RubyMine: En fullfjädrad IDE för Ruby.
- AppCode: En fullfjädrad IDE för Swift.
- PhpStorm: En fullfjädrad IDE för PHP.

JetBrains-verktyg erbjuder en mängd olika funktioner som kan hjälpa utvecklare att vara mer produktiva och effektiva. Dessa funktioner inkluderar:

- Syntaxfärgning och autoinmatning
- Felkontroll och intellisense
- Kodförbättring och omstrukturering
- Modulhantering och projekthantering
- Testning och debuggning
- Dokumentation och dokumentation

JetBrains-verktyg är ett kraftfullt verktyg som kan hjälpa utvecklare att vara mer produktiva och effektiva. De är tillgängliga för både studenter och företag, och de kan användas för att utveckla programvara i en mängd olika språk och plattformar.

Nyttan av JetBrains-verktyg

Det finns många fördelar med att använda JetBrains-verktyg. Några av de mest uppenbara fördelarna är:

- De kan hjälpa dig att skriva bättre kod snabbare.
- De kan hjälpa dig att hitta och fixa fel i din kod.
- De kan hjälpa dig att organisera och strukturera din kod.
- De kan hjälpa dig att testa din kod.
- De kan hjälpa dig att dokumentera din kod.

Begränsningar av JetBrains-verktyg

Det finns några få begränsningar med JetBrains-verktyg. Några av de mest uppenbara begränsningarna är:

- De kan vara dyra.
- De kan vara komplexa att använda.
- De kan inte användas för alla språk och plattformar.

Att tänka på som studerande

Om du är en studerande som funderar på att använda JetBrains-verktyg är det några saker du bör tänka på. För det första är JetBrains-verktyg dyra. Du kan dock få en gratis licens genom att registrera dig för en JetBrains Education-konto. För det andra kan JetBrains-verktyg vara komplexa att använda. Om du är nybörjare kan du behöva ta en kurs eller läsa en manual för att lära dig hur du använder dem. För det tredje kan JetBrains-verktyg inte användas för alla språk och plattformar. Om du utvecklar programvara i ett språk eller på en plattform som inte stöds av JetBrains-verktyg kan du behöva använda ett annat verktyg.

Tips

Om du bestämmer dig för att använda JetBrains-verktyg är det några tips som kan hjälpa dig att få ut det mesta av dem. För det första, ta dig tid att lära dig hur du använder verktygen. Det finns många resurser tillgängliga online och i böcker som kan hjälpa dig att lära dig hur du använder JetBrains-verktyg. För det andra, använd verktygen för att automatisera dina uppgifter. JetBrains-verktyg kan hjälpa dig att automatisera många av de uppgifter som du gör när du utvecklar programvara. För det tredje, använd verktygen för att förbättra din kod. JetBrains-verktyg kan hjälpa dig att hitta och fixa fel i din kod, och de kan också hjälpa dig att förbättra kvaliteten på din kod.

Java för Jin23

Marcus Medina

Microsoft-tools

Fördelar med Visual Studio Community och Visual Studio Code
Begränsningar av Visual Studio Community och Visual Studio Code
Att tänka på som studerande
Tips

Microsoft-tools

Microsoft erbjuder två olika IDE:er (integrerade utvecklingsmiljöer) för programutveckling: Visual Studio Community och Visual Studio Code.

- **Visual Studio Community** är ett fullfjädrad IDE som främst används för .NET-utveckling, men det kan också användas för att utveckla i andra språk, till exempel Python och C++.
- **Visual Studio Code** är en lättviktig texteditor som kan anpassas för att användas med en mängd olika språk, inklusive .NET, Python, JavaScript, TypeScript, C++, C#, Java, PHP, Go och Rust.
- SQL Server Management Studio (SSMS) är en IDE för SQL Server. Den kan användas för att utveckla och administrera SQL Server-databaser.

Båda IDE:erna är gratis att använda och är tillgängliga för Windows, macOS och Linux.

Fördelar med Visual Studio Community och Visual Studio Code

- Båda IDE:erna erbjuder en mängd olika funktioner som kan hjälpa utvecklare att vara mer produktiva och effektiva, till exempel syntaxfärgning, autoindentering, felkontroll, kodförbättring och omstrukturering.
- Båda IDE:erna har en stor community av användare och utvecklare som kan ge stöd och hjälp.
- Båda IDE:erna är uppdaterade med de senaste funktionerna och buggfixarna.
- SQL Server Management Studio (SSMS) hjälper till hanteringen av scheman, tabeller, index, vyer, lagrade procedurer, funktioner, triggers, användare, roller, säkerhet, säkerhetskopior, återställningar, replikering, loggar, jobb, schemaläggning, övervakning, underhållsplaner, databasdiagram mm.

Begränsningar av Visual Studio Community och Visual Studio Code

- Visual Studio Community är inte lika kraftfullt som Visual Studio Professional eller Enterprise, och det saknas vissa funktioner som är tillgängliga i de dyrare versionerna.
- Visual Studio Code är en lättviktig texteditor, och det saknar vissa funktioner som är tillgängliga i fullfjädrade IDE:er, till exempel intellisense och kodförbättring.
- SQL Server Management Studio (SSMS) är inte tillgängligt för macOS eller Linux.
- SQL Server äter upp mycket minne och kan vara långsamt. Så det är viktigt att ha en bra dator för att kunna köra det. SSMS är dock snäll.

Att tänka på som studerande

Om du är en studerande som funderar på att använda Visual Studio Community eller Visual Studio Code är det några saker du bör tänka på.

- Visual Studio Community är ett bra val för studenter som utvecklar i .NET.
- Visual Studio Code är ett bra val för studenter som utvecklar i andra språk, till exempel Python, JavaScript eller TypeScript.
- Det är bra att lära sig att använda båda IDE:erna, så att du kan välja det bästa verktyget för jobbet.
- Det är bra att lära sig att skriva kod utan hjälpmedel, så att du inte blir beroende av IDE:erna.

Tips

Om du bestämmer dig för att använda Visual Studio Community eller Visual Studio Code är det några tips som kan hjälpa dig att få ut det mesta av dem.

- Ta dig tid att lära dig hur du använder verktygen. Det finns många resurser tillgängliga online och i böcker.
- Använd verktygen för att automatisera dina uppgifter. Visual Studio Community och Visual Studio Code kan hjälpa dig att automatisera många av de uppgifter som du gör när du utvecklar programvara.
- Använd verktygen för att förbättra din kod. Visual Studio Community och Visual Studio Code kan hjälpa dig att hitta och fixa fel i din kod, och de kan också hjälpa dig att förbättra kvaliteten på din kod.

Java för Jin23

Marcus Medina

Github Copilot

Användning

Exempel på prompts

Exempel

Nyttan av det

Begränsningar

Att tänka på som studerande

Plagiering / Fusk

Github Copilot

Github Copilot är en AI-assistent för programmerare som är utvecklad av OpenAI och GitHub. Den är baserad på GPT-3, OpenAIs stora språkmodell, och kan hjälpa programmerare att skriva kod snabbare och mer effektivt.

Användning

Github Copilot kan användas för att skriva alla typer av kod, inklusive Python, Java, C++, och JavaScript. Den kan användas för att skriva ny kod, eller för att förbättra befintlig kod.

Github Copilot fungerar genom att generera förslag på kod baserat på den kod som du skriver. Du kan sedan acceptera eller avslå förslagen, eller redigera dem efter behov.

Exempel på prompts

Här är några exempel på prompts som du kan använda med Github Copilot:

- // Skriv en funktion som returnerar summan av två tal.
- // Skriv en klass som representerar en lista.
- // Skriv en funktion som skriver ut alla tal från 1 till 10.

Efter att ha skrivit in prompten trycker du ENTER och väntar på att den ska bli klar

Exempel

// Skapa en metod som rekursivt räknar ut Fibonacci-talen för ett givet tal.

Tryck enter

```
public static int fibonacci(int n) {  
    if (n <= 1) {  
        return n;  
    }  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

Glöm inte att kommentera vart du har fått koden ifrån. I detta fall är det från Github Copilot.

// Genererad av Github Copilot

Nyttan av det

Github Copilot kan hjälpa dig att skriva kod snabbare och mer effektivt. Den kan också hjälpa dig att lära dig nya språk och ramverk, eftersom den kan generera kod för dig att läsa och lära dig av. Det är viktigt att du förstår den kod som den genererar, så att du kan lära dig av den.

Begränsningar

Github Copilot är fortfarande under utveckling, så den har vissa begränsningar. Till exempel kan den inte förstå all kod, och den kan ibland generera felaktig kod.

Github Copilot kostar per månad, så du bör inte använda den om du inte har råd med den.

Att tänka på som studerande

Om du är studerande är det viktigt att tänka på följande när du använder Github Copilot:

- Ange alltid att du har använt Github Copilot när du lämnar in kod.
- Kontrollera alltid koden som Github Copilot genererar noggrant för att se till att den är korrekt.
- Använd inte Github Copilot för att fuska.
- Github Copilot är gratis för studerande.

Plagiering / Fusk

Plagiering är att kopiera någon annans arbete utan att ange källan. Fusk är att försöka få ett högre betyg än du har förtjänat genom att fuska.

Om du kopierar en klasskamrats kod utan att ange källan, så har du plagierat. Om du kopierar en klasskamrats kod och anger källan, så har du inte plagierat.

Om du fuska, så kommer du att få en lägre betyg eller bli utesluten från kursen.

Det är viktigt att vara ärlig och att inte fuska. Om du har frågor om plagiering eller fusk, så prata med din lärare.

Java för Jin23

Marcus Medina

Tabnine

Användning

Begränsningar

Att tänka på som studerande

Tips

Tabnine

Tabnine är en AI-baserad kodförslagstjänst som hjälper utvecklare att skriva bättre kod snabbare. Den fungerar genom att analysera din kod och sedan föreslå förbättringar, till exempel nya metoder, funktioner och variabler. Tabnine är tillgänglig för en mängd olika språk, inklusive Python, Java, C++ och JavaScript.

Användning

Tabnine kan användas på en mängd olika sätt. Du kan använda den för att:

- Skriva ny kod
- Förbättra befintlig kod
- Hitta och fixa fel
- Lära dig nya kodningstekniker

Begränsningar

Tabnine är fortfarande under utveckling, så det finns några begränsningar. Till exempel kan den inte förstå alla typer av kod, och den kan ibland generera felaktig kod.

Att tänka på som studerande

Om du är en studerande som funderar på att använda Tabnine är det några saker du bör tänka på.

- Tabnine är ett verktyg som kan hjälpa dig att skriva bättre kod, men det är inte ett substitut för att lära dig att kod. Du bör fortfarande lära dig de grundläggande begreppen i kodning och hur du skriver korrekt kod.
- Tabnine kan generera felaktig kod, så du bör alltid kontrollera koden som den genererar innan du använder den.
- Du bör inte använda Tabnine för att fuska. Om du använder Tabnine för att generera kod som du inte förstår, kommer du inte att lära dig något och du kommer inte att kunna lösa problem som kräver att du förstår koden.
- Tabnine är gratis men begränsad, den bättre versionen kostar, så du bör inte använda den om du inte har råd med den.

Tips

Om du bestämmer dig för att använda Tabnine är det några tips som kan hjälpa dig att få ut det mesta av det.

- Lär dig hur man använder Tabnines inställningar för att anpassa det till dina behov.
- Använd Tabnine för att få feedback på din kod, men använd inte den som en ersättning för att lära dig att kod.
- Kontrollera alltid koden som Tabnine genererar innan du använder den.

Java för Jin23

Marcus Medina

Chatgpt

Vad är ChatGPT?

Exempel på prompts

Användning

1. Få inspiration till nya projekt:
2. Använd ChatGPT som en extra lärare:
3. Hantera kodgenerering:

Begränsningar

Att tänka på som studerande

Extra prompts

Länkar

Chatgpt

Vad är ChatGPT?

ChatGPT är en avancerad språkmodell utvecklad av OpenAI. Den bygger på den banbrytande GPT-3.5-arkitekturen och är utformad för att kunna generera text som svar på användargivna prompts. Modellen är tränad på en enorm mängd textdata från olika källor och kan svara på frågor, ge förklaringar och till och med skapa dialoger. ChatGPT har blivit en användbar resurs för elever, forskare, författare och många andra som behöver hjälp med att generera text av hög kvalitet.

Exempel på prompts

Prompts är de instruktioner eller frågor du ger till ChatGPT för att få ett svar eller en textgenerering. Genom att formulera väldefinierade och tydliga prompts kan du få bättre resultat från modellen. Här är några exempel på prompts som du kan använda:

1. "Kan du förklara konceptet [lägg till begrepp eller ämne här] på ett enkelt sätt?"
2. "Jag behöver hjälp med att förstå [specificera ämnet]. Kan du ge mig en grundläggande förklaring?"
3. "Kan du ge mig några exempel på [specificera ämnet]?"
4. "Vad är skillnaden mellan [specificera två begrepp]?"
5. "Kan du ge mig några tips för att förbättra mina skrivfärdigheter?"
6. "Jag har ett fel i min kod som jag inte kan hitta, kan du förklara vad som är fel?" (klistra in koden i raden under)
7. "Jag ska skapa ett projekt om [specificera ämnet], kan du ge mig några idéer?"
8. "Jag ska skapa ett projekt om [specificera ämnet], kan du hjälpa mig att planera, steg för steg?"
9. "Jag ska skapa ett projekt om [specificera ämnet], kan du hjälpa mig att göra en utför backlog?"
10. "Jag fick detta felmeddelande när jag körde min kod: [klistra in felmeddelandet här]. Vad betyder det?"

Självklart kan du lägga till följdfrågor eller förtydliganden för att få mer specifika svar.

1. "Kan du göra det mer utförligt?"
2. "Kan du ge mig ett exempel?"
3. "Kan du ge mig ett annat exempel?"
4. "Kan du ge mig ett exempel som är mer relevant för [specificera ämnet]?"

Dessa prompts kan användas som en startpunkt för att interagera med ChatGPT och få information eller stöd för dina studier. Kom ihåg att vara så specifik som möjligt i dina prompts för att få relevanta och användbara svar.

Jag hoppas att dessa exempel på prompts hjälper dig att använda ChatGPT mer effektivt som en studerande. Låt mig veta om det finns något annat du behöver hjälp med!

Användning

ChatGPT kan användas på olika sätt för att hjälpa och underlätta skrivprocessen. Här är några exempel på hur du kan dra nytta av ChatGPT:

1. Få inspiration till nya projekt:

Du kan använda ChatGPT för att generera idéer och få inspiration när du står inför skrivblockering eller behöver utforska nya ämnen. Genom att ge en prompt eller en beskrivning kan du få förslag och riktlinjer för att komma igång med ditt skrivande.

2. Använd ChatGPT som en extra lärare:

Om du stöter på svåra begrepp eller inte förstår ett ämne kan ChatGPT vara till hjälp. Du kan ställa frågor eller ge en kort förklaring till modellen, och den kan generera en mer utförlig förklaring eller exempel för att hjälpa till med din förståelse. Tänk på att även om ChatGPT kan ge användbara svar, är det alltid bra att bekräfta informationen genom andra källor.

3. Hantera kodgenerering:

ChatGPT kan också hjälpa till med kodgenerering genom att ge exempel eller lösningsförslag för specifika programmeringsproblem. Det kan vara användbart när du fastnar eller behöver hjälp med att implementera vissa funktioner. Om du väljer att använda kod som genererats av ChatGPT, kom ihåg att lägga till en kommentar som tydligt anger att koden är genererad av modellen.

Begränsningar

Det är viktigt att vara medveten om begränsningarna med ChatGPT och att använda det som ett verktyg med viss försiktighet:

- **Brist på kontextförståelse:** ChatGPT saknar kontextförståelse över olika frågor. Det innebär att det kan generera svar som kan verka rimliga men faktiskt vara felaktiga eller missvisande. Var alltid medveten om detta och verifiera information från andra tillförlitliga källor.
- **Risk för generering av opålitlig information:** Eftersom modellen tränas på en mängd olika källor kan den även generera svar som är felaktiga, opålitliga eller kontroversiella. Var kritisk när du bedömer svaren och använd din egen bedömning för att avgöra om informationen är korrekt och pålitlig.
- **Brist på mänskligt omdöme:** ChatGPT saknar förmågan att utöva mänskligt omdöme och kan därför inte bedöma eller anpassa sig till etiska eller moraliska aspekter av skrivandet. Var medveten om detta och använd din egen bedömning för att avgöra vad som är lämpligt och passande att använda från modellen.

Att tänka på som studerande

När du använder ChatGPT som elev är det viktigt att vara medveten om akademiska integritetsprinciper och undvika fusk eller plagiering. Här är några riktlinjer att följa:

- **Använd ChatGPT för att få inspiration till nya projekt:** Modellen kan hjälpa dig att generera idéer och ge riktlinjer för ditt eget arbete. Se till att använda informationen som en utgångspunkt och utveckla den vidare på ett originellt sätt.

- **Använd ChatGPT som en extra lärare:** Du kan använda modellen för att få förklaringar på saker du inte förstår, men det är viktigt att alltid komplettera med egen forskning och kontrollera informationen från andra tillförlitliga källor.
- **Hantera kodgenerering med försiktighet:** Om du använder ChatGPT för att generera kod, kom ihåg att alltid ange att koden är genererad av modellen. Att kopiera en hel uppgift eller en klasskamrats kod utan att ange källan är inte tillåtet och anses vara plagiering eller fusk. Det är viktigt att visa att du har förstått uppgiften genom att bearbeta och anpassa den genererade koden på ett meningsfullt sätt.
- **Var ärlig och respektera akademiska riktlinjer:** Att vara ärlig och respektera akademiska integritetsprinciper är avgörande för din personliga och professionella utveckling. Var noga med att följa dina lärares riktlinjer och ta ansvar för ditt eget arbete.

Kom ihåg att ChatGPT är ett verktyg som kan vara till hjälp, men det ersätter inte din egen kreativitet, ansträngning och kritiska tänkande. Genom att använda det på rätt sätt kan du dra nytta av dess potential och förbättra din skrivförmåga.

Extra prompts

Personlighet:

“System prompt: Du ska nu agera som {valfri personlighet}” + En lärare i programmering som ska förklara för nybörjare + Darth Vader + Elon Musk + Hund + Katt + Taylor Swift + Yoda + En svartsjuk flickvän / pojkvän + En romantisk flickvän / pojkvän + Skåning / Göteborgare / Stockholmare / Smålänning

Förklaringar: + “Förklara {valfritt ämne} på ett enkelt sätt” + “Förklara {valfritt ämne} ELI5” + “Förklara {valfritt ämne} för en nybörjare”

Prompt engineering:

- “Kan du hjälpa mig att skapa en Persona som passar mitt sätt att lära mig programmering, så att jag kan lära mig mer effektivt? Ställ frågor om mig och mina intressen, och skapa en personlighet som passar mig.”.

Efter att ha skrivit det kommer ChatGPT att ställa en massa frågor som du ska svara på, ju mer detaljerat desto bättre. När du är klar med det, så kommer den att skapa en personlighet som passar dig, och du kan använda den för att ställa frågor till den. Skriv sedan >>> System prompt: Du ska nu agera som {den personligheten}. och klistra in hela personligheten den skapade <<<

“>>> <<<” är för att den ska fatta att allt hänger samman

Om du gillar personligheten, kopiera allt från >>> till <<< och spara det i notepadden så kan du återanvända det i framtiden.

Du kan skapa personligheter till andra saker, det är upp till din fantasi.

Länkar

215+ Chat prompts for ChatGPT 100 best ChatGPT prompts Getting the most out of ChatGPT 120 best prompts for ChatGPT

Java för Jin23

Marcus Medina

Bard

Vad är Bard?

Användning

Exempel på prompts

Begränsningar

Att tänka på som studerande

Plagiering / Fusk

Att använda Bard för att få inspiration till nya projekt

Att använda Bard som en extra lärare

Tips

Bard

Bard är en stor språkmodell, även känd som en konversations-AI eller chatbot, tränad för att vara informativ och omfattande. Den är tränad på en enorm mängd textdata och kan kommunicera och generera människoliknande text som svar på ett brett spektrum av uppmaningar och frågor. Till exempel kan den ge sammanfattningar av faktaämnen eller skapa berättelser.

Vad är Bard?

Bard är en stor språkmodell från Google AI, tränad på en enorm mängd textdata. Den kan generera text, översätta språk, skriva olika typer av kreativt innehåll och svara på dina frågor på ett informativt sätt. Bard är fortfarande under utveckling, men den har lärt sig att utföra många typer av uppgifter, inklusive

- Bard kommer att göra sitt bästa för att följa dina instruktioner och slutföra dina förfrågningar eftertänksamt.
- Bard kommer att använda sin kunskap för att svara på dina frågor på ett omfattande och informativt sätt, även om de är öppna, utmanande eller konstiga.
- Bard kommer att generera olika kreativa textformat, som dikter, kod, skript, musikstycken, e-post, brev, etc. Den kommer att göra sitt bästa för att uppfylla alla dina krav.

Användning

Bard kan användas för en mängd olika uppgifter, inklusive

- Att få hjälp med dina studier. Bard kan hjälpa dig att lära dig nya saker genom att ge dig sammanfattningar av faktaämnen eller genom att skapa berättelser som är relaterade till det du studerar.
- Att skriva kreativt innehåll. Bard kan hjälpa dig att skriva dikter, kod, skript, musikstycken, e-post, brev, etc.
- Att översätta språk. Bard kan översätta text från ett språk till ett annat.
- Att svara på dina frågor. Bard kan svara på dina frågor på ett informativt sätt, även om de är öppna, utmanande eller konstiga.

Exempel på prompts

Bard kan svara på ett brett spektrum av uppmaningar och frågor. Här är några exempel:

- Ge mig en sammanfattning av den franska revolutionen.
- Skriv en dikt om kärlek.
- Översätt den här meningen från engelska till spanska.

- Varför är himlen blå?
- Vad är meningen med livet?

Begränsningar

Bard är fortfarande under utveckling, så det finns vissa begränsningar för vad den kan göra. Till exempel kan den inte alltid ge exakta eller fullständiga svar på öppna, utmanande eller konstiga frågor. Dessutom är Bard inte alltid kapabel att generera kreativt innehåll som är av hög kvalitet.

Att tänka på som studerande

Om du är studerande är det några saker du bör tänka på när du använder Bard. För det första är det viktigt att komma ihåg att Bard inte är en mänsklig lärare. Den kan inte ge dig samma nivå av personlig uppmärksamhet och stöd som en mänsklig lärare kan. För det andra är det viktigt att inte förlita dig för mycket på Bard. Det är viktigt att du fortfarande lär dig och förstår det material du studerar. Slutligen är det viktigt att använda Bard på ett etiskt sätt. Du bör inte använda Bard för att fuska eller för att kopiera annans arbete.

Plagiering / Fusk

Plagiering är att kopiera annans arbete utan att ge kredit. Fusk är att använda otillåtna hjälpmedel för att lösa en uppgift. Det är inte tillåtet att fuska eller plagiera. Om du kopierar annans arbete utan att ge kredit kommer du att bli disciplinerad. Om du använder otillåtna hjälpmedel för att lösa en uppgift kommer du också att bli disciplinerad.

Att använda Bard för att få inspiration till nya projekt

Bard kan användas för att få inspiration till nya projekt. Till exempel

- **Fråga Bard om idéer.** Om du står inför en skrivstopp, eller om du letar efter nya idéer för ett projekt, kan du fråga Bard om hjälp. Bard kan komma med en mängd olika idéer, allt från berättelser till kod till presentationer.
- **Be Bard om feedback.** När du har en idé kan du be Bard om feedback. Bard kan hjälpa dig att identifiera styrkor och svagheter i din idé, och den kan ge förslag på hur du kan förbättra den.
- **Använd Bard som en brainstorming-partner.** Om du arbetar med ett team kan du använda Bard som en brainstorming-partner. Bard kan komma med nya idéer, och den kan hjälpa dig att lösa problem.

Bard är ett kraftfullt verktyg som kan hjälpa dig att få inspiration till nya projekt. Genom att använda Bard kan du utöka dina idéer och skapa något unikt och intressant.

Här är några ytterligare tips för att använda Bard för att få inspiration till nya projekt:

- Var öppen för nya idéer. Bard kan komma med idéer som du aldrig skulle ha tänkt på själv.
- Var inte rädd att experimentera. Bard kan hjälpa dig att testa nya idéer och se vad som fungerar.
- Ha kul! Att få inspiration ska vara roligt. Om du inte har roligt kommer du inte att vara lika kreativ.

Att använda Bard som en extra lärare

Bard kan användas som en extra lärare. Till exempel

- **Be Bard om hjälp med dina studier.** Om du stöter på svåra begrepp eller inte förstår ett ämne kan du be Bard om hjälp. Bard kan ge dig en sammanfattning av faktaämnena, och den kan ge dig exempel på hur du kan använda dina kunskaper.
- "Kan du förklara konceptet [lägg till begrepp eller ämne här] på ett enkelt sätt?"

- “Jag behöver hjälp med att förstå [specificera ämnet]. Kan du ge mig en grundläggande förklaring?”
- “Kan du ge mig några exempel på [specificera ämnet]?”
- “Vad är skillnaden mellan [specificera två begrepp]?”
- “Kan du ge mig några tips för att förbättra mina skrivfärdigheter?”
- “Jag har ett fel i min kod som jag inte kan hitta, kan du förklara vad som är fel?” (klistra in koden i raden under)
- “Jag ska skapa ett projekt om [specificera ämnet], kan du ge mig några idéer?”
- “Jag ska skapa ett projekt om [specificera ämnet], kan du hjälpa mig att planera, steg för steg?”
- “Jag ska skapa ett projekt om [specificera ämnet], kan du hjälpa mig att göra en utför backlog?”
- “Jag fick detta felmeddelande när jag körde min kod: [klistra in felmeddelandet här]. Vad betyder det?”

Självklart kan du lägga till följdfrågor eller förtydliganden för att få mer specifika svar.

- “Kan du göra det mer utförligt?”
- “Kan du ge mig ett exempel?”
- “Kan du ge mig ett annat exempel?”
- “Kan du ge mig ett exempel som är mer relevant för [specificera ämnet]?”

Dessa prompts kan användas som en startpunkt för att interagera med Bard och få information eller stöd för dina studier. Kom ihåg att vara så specifik som möjligt i dina prompts för att få relevanta och användbara svar.

Tips

Här är några tips för att använda Bard mer effektivt som en studerande:

- Var specifik. Ju mer specifik du är, desto bättre svar kommer du att få.
- Var tydlig. Ju tydligare du är, desto bättre svar kommer du att få.
- Var tålmodig. Ibland kan det ta ett tag för Bard att generera ett svar.
- Var kreativ. Bard kan hjälpa dig att tänka utanför boxen.
- Var inte rädd för att experimentera. Bard kan hjälpa dig att testa nya idéer och se vad som fungerar.
- Var inte rädd för att misslyckas. Bard kan hjälpa dig att lära dig av dina misstag.
- Var inte rädd för att fråga. Bard kan hjälpa dig att lära dig mer om ett ämne.
- Var inte rädd för att be om hjälp. Bard kan hjälpa dig att lära dig mer om ett ämne.
- Var inte rädd för att be om feedback. Bard kan hjälpa dig att lära dig mer om ett ämne.
- Var inte rädd för att be om förslag. Bard kan hjälpa dig att lära dig mer om ett ämne.

Personligheter

Bard kan anta olika personligheter. Här är några exempel:

- “Du ska nu agera som en lärare för en studerande som behöver hjälp med att förstå ett ämne. Tänk på att förklara ELI5 och använda enkla ord.”
- “Du ska nu agera som en politiker, var så diffus som möjligt och använd många ord för att säga ingenting.”
- “Du ska nu agera som en poet, var så poetisk som möjligt och använd många metaforer.”
- “Du ska nu agera som en programmerare, var så teknisk som möjligt och använd många tekniska termer.”
- “Du ska nu agera som en diplomat, var så diplomatisk som möjligt och använd många ord för att säga ingenting.”
- “Du ska nu agera som en författare, var så kreativ som möjligt och använd många metaforer.”

- “Du ska nu agera som en bilförsäljare av begagnade bilar, var så övertygande som möjligt och använd många adjektiv.”
- “Du ska nu agera som en förälskad tonåring, var så romantisk som möjligt och använd många metaforer.”
- “Du ska nu agera som en kock, var så kreativ som möjligt och använd många metaforer.”
- “Du ska nu agera som en advokat, var så övertygande som möjligt och använd många adjektiv.”

Känslor

Bard kan anta olika känslor. Här är några exempel:

- “Du ska nu agera som en glad person, var så glad som möjligt och använd många utropstecken.”
- “Du ska nu agera som en ledsen person, var så ledsen som möjligt och använd många utropstecken.”
- “Du ska nu agera som en arg person, var så arg som möjligt och använd många utropstecken.”

Tidsperioder

Bard kan anta olika tidsperioder. Här är några exempel:

- “Du ska nu agera som en person från medeltiden, var så medeltida som möjligt och använd många medeltida termer.”
- “Du ska nu agera som en person från framtiden, var så futuristisk som möjligt och använd många futuristiska termer.”
- “Du ska nu agera som en person från 1800-talet, var så 1800-tal som möjligt och använd många 1800-talstermer.”
- “Du ska nu agera som en person från 1900-talet, var så 1900-tal som möjligt och använd många 1900-talstermer.”
- “Du ska nu agera som en person från 1980-talet, var så 2000-tal som möjligt och använd många 1980-talstermer.”
- “Du ska nu agera som en person från 1960-talet, var så 2000-tal som möjligt och använd många 1960-talstermer.”

Det är bra att tänka på att Bard är ganska ny så man får påminna den om sin roll några gånger innan den går in i det helt

Java för Jin23

Marcus Medina

If

- Introduktion
- Exempel
- Fördelar
- Begränsningar
- Användningsområden
- TL;DR
- Obligatorisk dad joke

If

En introduktion till ämnet If på språket 'Svenska' och kodspråk 'Java'.

Introduktion

If är en kontrollstruktur som används inom Java för att ställa logiska frågor. Svaren på dessa frågor kan vara antingen sant eller falskt. Baserat på svaret kommer Java att utföra olika handlingar. Om svaret är sant kommer koden efter if att exekveras. Om svaret är falskt kommer Java att hoppa över koden efter if och fortsätta med resten av programmet.

Exempel

Här är ett exempel på hur man använder if i Java:

```
int ålder = 18;
if (ålder >= 18) {
    System.out.println("Du är myndig");
}
```

I exemplet ovan kontrollerar vi om åldern är större eller lika med 18. Om det är sant skrivs meddelandet "Du är myndig" ut i konsolfönstret. Om åldern är mindre än 18 kommer koden efter if att hoppas över.

Fördelar

- If-satsen ger oss möjlighet att kontrollera olika villkor och anpassa koden baserat på dessa villkor.
- Genom att använda if-satsen kan vi skapa mer flexibla och dynamiska program som kan hantera olika scenarier.
- If-satsen är grundläggande för att kunna implementera logik och beslutsfattande i våra program.

Begränsningar

- If-satser kan ibland leda till komplex och svårhanterlig kod om de används för mycket eller på ett ogenomtänkt sätt.
- Det är viktigt att vara noggrann med att utvärdera och testa olika villkor för att undvika logiska fel eller buggar i koden.
- En felaktigt formulerad eller felaktigt placerad if-sats kan leda till att koden inte fungerar som förväntat.

Användningsområden

If-satser används i många olika situationer och scenarier inom programmering. Här är några exempel på användningsområden för if-satser i Java:

1. Användarens validering: Om vi vill kontrollera om en användare har angett giltig indata kan vi använda en if-sats för att validera input och vidta lämpliga åtgärder baserat på resultatet.
2. Styra programflödet: If-satser kan användas för att styra hur programmet ska bete sig baserat på olika villkor. Till exempel kan vi använda en if-sats för att avgöra om ett visst block av kod ska utföras eller inte.—
3. Loopar: If-satser kan användas inuti loopar för att kontrollera när loopen ska avslutas eller fortsätta. Detta ger oss möjlighet att skapa mer flexibla och dynamiska loopar.
4. Felsökning och hantering av undantag: If-satser kan användas för att hantera olika undantagssituationer och felsöka problem i koden. Genom att kontrollera olika villkor kan vi vidta lämpliga åtgärder för att hantera fel och undantag.

TL;DR

If är ett kommando inom Java som används för att ställa logiska frågor och utföra olika handlingar baserat på svaren. Det ger oss möjlighet att kontrollera villkor och anpassa programbeteendet. If-satser används för att validera användarinput, styra programflödet, hantera loopar och hantera undantagssituationer.

Obligatorisk dad joke

Varför var "if" så bra på att lösa problem? För att den alltid hade en "else" i ärmen!

Java för Jin23

Marcus Medina

Else

- Beskrivning
- Exempel
- Sammanfattning
- Termer och förklaringar
- Obligatorisk dad-joke

Else

▼ Innehållsförteckning

Beskrivning

“Annars” är ett nyckelord som används i programmeringsspråket Java för att skapa en alternativ väg i en kodblockstruktur. Det ger möjlighet att utföra en annan sekvens av kod om en “om”-sats utvärderas som falsk. Med andra ord kan “annars” användas för att hantera fall där “om”-villkoret inte är uppfyllt, och programmet kan utföra en annan uppsättning instruktioner istället. När en “om”-sats utvärderas som sann, utförs de kodinstruktioner som ligger inuti “om”-blocket. Om villkoret i “om”-satsen däremot utvärderas som falskt, kommer programmet att hoppa över kodblocket inom “om” och istället utföra kodblocket inom “annars”.

För att använda “annars” måste det alltid följas av en “om”. Det är också viktigt att notera att “annars” alltid kommer i slutet av en “om”-sats och kan inte användas utanför den.

Exempel

Här är ett exempel som visar hur “annars” kan användas för att kontrollera om en person är myndig eller inte baserat på deras ålder:

```
int ålder = 18;
if (ålder >= 18) {
    System.out.println("Du är myndig");
} else {
    System.out.println("Du är inte myndig");
}
```

I detta exempel är “age” satt till 18. Eftersom 18 är större än eller lika med 18, utvärderas “if”-villkoret som sant, och programmet skriver ut “Du är myndig”. Om vi ändrar värdet på “age” till 17 kommer “if”-villkoret att utvärderas som falskt, och programmet skriver ut “Du är inte myndig”. Med hjälp av “else” kan vi låta programmet välja olika vägar att följa beroende på villkoren.

Sammanfattning

“Else” är ett viktigt nyckelord inom Java som ger möjlighet att skapa en alternativ väg i kodblocket baserat på ett “if”-villkor. Om “if”-satsen utvärderas som sann utförs de kodinstruktioner som ligger inuti “if”-blocket, och om villkoret utvärderas som falskt utförs istället kodinstruktionerna inom “else”-blocket. Det ger programmerare möjlighet att hantera olika fall och göra beslut i sina program.

Termer och förklaringar

Term	Förklaring			
Else	Ett nyckelord i Java som används för att skapa en alternativ väg i kodblocket när "if"-villkoret är falskt.			
If	Ett nyckelord i Java som används för att skapa en villkorsbaserad kodblockstruktur. Om villkoret är sant, utförs kodinstruktionerna inom "if"-blocket		Code block	En grupp av kodinstruktioner som är grupperade tillsammans.
Condition	En logisk fråga som kan utvärderas som antingen sann eller falsk.			

Obligatorisk dad-joke

Varför gillar programmerare att använda "else"?

För att de inte gillar att vara i "if-nite"! ☐

Java för Jin23

Marcus Medina

Else if

- Användningsområden
- Kodexempel
 - Betygsberäkning
 - Validering
 - Menyval
 - Datum- och tidshantering
- Sammanfattning

Else if

Else if är en viktig konstruktion inom programmering som erbjuder flexibilitet och tydlighet när det gäller att hantera olika villkor och beslut. Det kan vara användbart i olika scenarier där vi behöver göra flera olika val baserat på olika förutsättningar.

Användningsområden

Else if kan tillämpas i olika situationer, inklusive:

1. **Betygsberäkning:** Om vi har ett betygssystem där vi behöver tilldela olika betyg baserat på en students poäng, kan vi använda Else if för att göra olika beslut baserat på poängintervallet.
2. **Validering:** Om vi behöver validera olika typer av data, som användarinput eller inmatade värden, kan vi använda Else if för att göra olika valideringsbeslut baserat på typen av data eller innehållet.
3. **Menyval:** Om vi har en meny med flera alternativ kan vi använda Else if för att hantera olika val baserat på användarens inmatning.
4. **Datum- och tidshantering:** Vi kan använda Else if för att hantera olika datum- och tidsscenarier, t.ex. att kontrollera om det är morgon, eftermiddag eller kväll och vidta olika åtgärder baserat på det.

Kodexempel

Här är några kodexempel som visar användningen av Else if i Java:

Betygsberäkning

```
int poäng = 75;
if (poäng >= 90) {
    System.out.println("A");
} else if (poäng >= 80) {
    System.out.println("B");
} else if (poäng >= 70) {
    System.out.println("C");
} else if (poäng >= 60) {
    System.out.println("D");
} else {
    System.out.println("F");
}
```

I detta exempel utvärderar vi en elevs poäng och skriver ut det motsvarande betyget beroende på poängintervallet.

Validering

Här utvärderar vi input från användaren

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Vänligen ange din ålder:");
        int age = scanner.nextInt();

        if (age < 0) {
            System.out.println("Åldern kan inte vara negativ.");
        } else if (age < 3) {
            System.out.println("Barnrumpa, du är för söt!");
        } else if (age < 18) {
            System.out.println("Ung och full av energi!");
        } else if (age < 40) {
            System.out.println("Medelålders och i sin bästa form!");
        } else if (age < 60) {
            System.out.println("Gamling, men fortfarande aktiv!");
        } else if (age < 100) {
            System.out.println("Jisses, du är gammal!");
        } else {
            System.out.println("Du är uråldrig! Respekt!");
        }

        scanner.close();
    }
}
```

Menyval

```
System.out.println("Välj ett alternativ:");
System.out.println("1. Visa saldo");
System.out.println("2. Gör en insättning");
System.out.println("3. Gör ett uttag");
int val = Integer.parseInt(System.console().readLine());
if (val == 1) {
    // Visa saldo
} else if (val == 2) {
    // Gör en insättning
} else if (val == 3) {
    // Gör ett uttag
} else {
    System.out.println("Ogiltigt val");
}
```

I detta exempel kan användaren välja ett alternativ från en meny genom att ange ett nummer. Beroende på användarens val kan olika handlingar utföras.

Datum- och tidshantering

```
import java.time.LocalDateTime;

LocalTime nu = LocalDateTime.now();
if (nu.getHour() < 12) {
    System.out.println("God morgon!");
} else if (nu.getHour() < 18) {
    System.out.println("God eftermiddag!");
} else {
    System.out.println("God kväll!");
}
```

I detta exempel används Else if för att kontrollera aktuell tid och skriva ut lämpliga hälsningar baserat på tiden på dygnet.

Sammanfattning

Else if är en viktig konstruktion inom programmering som erbjuder flexibilitet och tydlighet när det gäller att hantera olika villkor och beslut. Det kan vara användbart i olika scenarier där vi behöver göra flera olika val baserat på olika förutsättningar. Det är dock viktigt att vara medveten om begränsningarna och komplexiteten som kan uppstå när man använder Else if.

Obligatorisk dad-joke:

Varför gillar programmerare att använda Else if?

För att de har så många val i livet och villkorlig kärlek är bara en av dem! ☐

Java för Jin23

Marcus Medina

Ternary if

Exempel

Slutsats

Mer läsning

Termer

Obligatorisk pappaskämt

Ternary if

Ternary if, även känd som conditional operator, är en kompakt syntax i programmeringsspråket Java som tillåter oss att uttrycka en enkel if-sats på en rad. Det ger oss möjlighet att utvärdera ett villkor och välja en av två uttryck beroende på om villkoret är sant eller falskt. Syntaxen för ternary if består av tre delar: villkor, frågetecken och uttryck för sant och falskt. Det är ett kraftfullt verktyg som kan göra koden mer koncis och läsbar.

Exempel

Här är ett exempel som visar hur man använder ternary if i Java:

```
int age = 18;
String result = age >= 18 ? "Du är myndig" : "Du är inte myndig";
System.out.println(result);
```

I detta exempel tilldelas strängen "Du är myndig" till variabeln "result" om värdet av variabeln "age" är större eller lika med 18. Annars tilldelas strängen "Du är inte myndig". Sedan skrivs värdet av "result" ut till konsolen. Det är viktigt att notera att ternary if är ett uttryck och kan användas inuti andra uttryck eller tilldelningar. Det ger oss möjlighet att göra kompakt och läsbar kod för att hantera enkla villkor. En annan användning av ternary if är när vi redan har en boolean-variabel och vi behöver välja mellan två värden baserat på dess värde. Här är ett exempel:

```
boolean catIsCute = true;
String result = catIsCute ? "Katten är söt <3" : "Katten är inte söt :(";
```

I det här exemplet tilldelas strängen "Katten är söt <3" till variabeln "result" om värdet av "catIsCute" är sant (true). Annars tilldelas strängen "Katten är inte söt :(".

Ternary if är ett verktyg som kan göra koden mer läsbar och koncis i situationer där vi behöver göra en enkel villkorskontroll och tilldela olika värden beroende på resultatet.

Slutsats

Ternary if, eller conditional operator, är ett användbart verktyg inom programmering som gör det möjligt för oss att uttrycka en enkel if-sats på en rad. Det hjälper oss att göra koden mer koncis och läsbar genom att välja mellan två uttryck beroende på om ett villkor är sant eller falskt. Genom att använda ternary if kan vi undvika att skriva en längre if-sats när vi bara behöver hantera enkla villkor.

Ternary if använder följande syntax:

```
villkor ? uttryck om sant : uttryck om falskt
```

Där "villkor" är det uttryck som utvärderas, "?" är frågetecknet som markerar början på ternary if, "uttryck om sant" är det värde eller uttryck som tilldelas om villkoret är sant, och "uttryck om falskt" är det värde eller uttryck som tilldelas om villkoret är falskt.

Det är viktigt att notera att ternary if bara är lämplig för enkla villkor och enkla uttryck. Om du behöver hantera mer komplexa villkor eller flera uttryck kan det vara bättre att använda en vanlig if-sats.

Mer läsning

- Java Conditional Operator (?:)
- Java If-else statement

Termer

- Ternary if: En kompakt syntax i Java som tillåter oss att uttrycka en enkel if-sats på en rad.
- Conditional operator: En annan term för ternary if, som beskriver dess användning för att utvärdera och välja mellan två uttryck baserat på ett villkor.

Obligatorisk pappaskämt

Varför är ternary if alltid så bekymrad?

För att den alltid vill vara säker på att det är `true` eller `false`, inga “may-be”!

Java för Jin23

Marcus Medina

Variabler

Exempel

Variabler

En variabel är en behållare som används för att lagra data. I Java måste alla variabler deklareraras innan de kan användas. Detta innebär att du måste ange vilken typ av data som variabeln kommer att lagra.

graph LR A[Variabel] --> B[Deklarera] B --> C[Initiera]

Exempel

```
// Deklarera en variabel av typen int.  
int number = 1138;  
  
// Deklarera en variabel av typen String.  
String name = "Obi Wan Kenobi";  
  
// Deklarera en variabel av typen boolean.  
boolean isJedi = true;  
  
// Deklarera en variabel av typen double.  
double pi = 3.14;  
  
// Deklarera en variabel av typen char.  
char letter = 'J';
```

I detta exempel har vi deklarerat fem variabler. Den första är en int, vilket innebär att den kan lagra heltal. Den andra är en sträng (String), vilket innebär att den kan lagra text. Den tredje är en boolean, vilket innebär att den kan lagra sant eller falskt. Den fjärde är en double, vilket innebär att den kan lagra decimaltal. Den femte är en char, vilket innebär att den kan lagra ett tecken.

Java för Jin23

Marcus Medina

Typer

Primitiva typer

Referenstyper

Termer

TL;DR-sammanfattning

Typer

En typ är en klassificering av data som används för att bestämma vilken typ av värde en variabel kan lagra. I Java finns det olika typer av typer, inklusive primitiva typer och referenstyper.

Primitiva typer

Här är en lista över några vanliga primitiva typer i Java tillsammans med deras beskrivning, minsta värde, högsta värde och storlek i minnet:

Typ	Beskrivning	Minsta	Högsta	Storlek
boolean	Booleskt värde	false	true	1 byte
char	16-bitars Unicode-tecken	'000'	''	2 byte
byte	8-bitars heltal	-128	127	1 byte
short	16-bitars heltal	-32768	32767	2 byte
int	32-bitars heltal	-2147483648	2147483647	4 byte
long	64-bitars heltal	-9223372036854775808	9223372036854775807	8 byte
float	32-bitars flyttal med enkels precision	1.4E-45	3.4028235E+38	4 byte
double	64-bitars flyttal med dubbel precision	4.9E-324	1.79769313486232E+308	8 byte

Observera att storlekarna som anges ovan är standardstorlekar och kan variera beroende på implementationen och plattformen.

Referenstyper

Utöver de primitiva typerna har Java också referenstyper, som är typer som lagrar referenser till objekt. Här är några exempel på vanliga referenstyper i Java:

- **String**: En klass för att hantera textsträngar.
- **Array**: En typ för att skapa och hantera arrayer av andra typer.
- **Class**: En klass som representerar en typ vid körningstid.
- **Interface**: En typ som definierar en samling metoder som en klass kan implementera.
- **List**: En typ för att skapa och hantera listor av objekt.
- **Map**: En typ för att skapa och hantera mappningar av nyckel-värde-par.

Referenstyper lagrar inte själva värdet utan pekar på objektet i minnet där värdet finns lagrat. De ger oss möjligheten att hantera mer komplexa datastrukturer och skapa hierarkier av objekt.

Termer

Term	Förklaring
Array	En typ för att skapa och hantera arrayer av andra typer.
Boolesk	En typ med två möjliga värden: <code>true</code> och <code>false</code> .
Char	En 16-bitars Unicode-tecken.
Double	En 64-bitars flyttal med dubbel precision.
Float	En 32-bitars flyttal med enkels precision.
Int	En 32-bitars heltalstyp.
Interface	En typ som definierar en samling metoder som en klass kan implementera.
Klass	En mall för att skapa objekt.
List	En typ för att skapa och hantera listor av objekt.
Long	En 64-bitars heltalstyp.
Map	En typ för att skapa och hantera mappningar av nyckel-värde-par.
Referenstyp	En typ som lagrar en referens till ett objekt.
Short	En 16-bitars heltalstyp.
Stack	En del av minnet som används för att lagra metoder och lokala variabler.
String	En klass för att hantera textsträngar.
Variabel	En behållare som används för att lagra data.
Värdestyp	En typ som lagrar ett värde.
Wrapperklass	En klass som används för att "linda in" primitiva typer och ge dem extra funktionalitet.
Wrapperobjekt	Ett objekt som innehåller en referens till en primitiv typ.

TL;DR-sammanfattning

I Java finns det olika typer av typer, inklusive primitiva typer och referenstyper. Primitiva typer inkluderar `boolean`, `char`, `byte`, `short`, `int`, `long`, `float` och `double`. Dessa typer lagrar värden direkt. Referenstyper inkluderar `String`, `Array`, `Class`, `Interface`, `List` och `Map`. Dessa typer lagrar referenser till objekt och ger oss möjligheten att hantera mer komplexa datastrukturer.

Java för Jin23

Marcus Medina

Datastrukturer

- Beskrivning
- Array
- List
- Stack
- Queue
- Dictionary
- HashSet
- Ännu fler datastrukturer
- Referenser

Datastrukturer

▼ Innehållsförteckning

1. TOC

Beskrivning

Datastrukturer är organiserade sätt att lagra och hantera data i en dator. Det finns olika typer av datastrukturer, såsom arrayer, listor, träd, grafer och hashtabeller, var och en med sina egna unika egenskaper och användningsområden. Dessa datastrukturer kan användas för att effektivt söka, sortera, lagra och hämta data, och är en viktig del av programmering och datavetenskap. Datastrukturer är lika mångsidiga som superhjältar i DC- och Marvel-världen. Precis som hjältarna har de olika förmågor och användningsområden som gör dem unika och passande för olika situationer. Låt oss ta en titt på några av dessa datastrukturer och hur de kan användas i programmering.

Array

En array är en samling av element av samma datatyp som lagras i minnet i en sekventiell ordning. Varje element i arrayen kan nås med hjälp av ett index. Indexet representerar positionen för varje element i arrayen, där det första elementet har index 0, det andra elementet har index 1 och så vidare. Arrayer används för att lagra och hantera en samling av värden av samma typ, som till exempel en lista med tal eller strängar. Exempel:

```
String[] avengers = new String[3];
avengers[0] = "Iron Man";
avengers[1] = "Captain America";
avengers[2] = "Thor";
System.out.println(avengers[0]); // Output: Iron Man
```

List

En lista är en dynamisk samling av element av samma datatyp. Till skillnad från en array kan storleken på en lista ändras dynamiskt när element läggs till eller tas bort. Listor tillhandahåller också en mängd användbara metoder för att söka, sortera och manipulera elementen i listan. Listor används när du behöver en samling av element som kan ändras i storlek under körningstid.

```
List<int> numbers = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
numbers.add(6);
System.out.println(numbers.size()); // Output: 6
```

Stack

En stack är en datastruktur som använder principen LIFO (Last In, First Out), vilket innebär att det senast tillagda elementet är det första som tas bort. Du kan lägga till element på toppen av stacken och ta bort element från toppen. Stackar används när du behöver hantera data i omvänd ordning av deras inmatningssekvens. Exempel:

```
Stack<String> superheroes = new Stack<>();
superheroes.push("Spider-Man");
superheroes.push("Iron Man");
superheroes.push("Hulk");
System.out.println(superheroes.pop()); // Output: Hulk
```

Queue

En kö är en datastruktur som använder principen FIFO (First In, First Out), vilket innebär att det första tillagda elementet är det första som tas bort. Du kan lägga till element i slutet av kön och ta bort element från början. Kön används när du behöver hantera data i samma ordning som deras inmatningssekvens.

```
Queue<String> heroes = new LinkedList<>();
heroes.add("Captain America");
heroes.add("Black Widow");
heroes.add("Hawkeye");
System.out.println(heroes.poll()); // Output: Captain America
```

Dictionary

Ett dictionary är en datastruktur som lagrar data som en samling av nyckel-värde-par. Varje nyckel måste vara unik och kan användas för att hämta det tillhörande värdet. Dictionaries används när du behöver snabb åtkomst till värden baserat på en given nyckel.

```
HashMap<String, Integer> heroStats = new HashMap<>();
heroStats.put("Iron Man", 100);
heroStats.put("Captain America", 90);
heroStats.put("Thor", 95);
System.out.println(heroStats.get("Iron Man")); // Output: 100
```

HashSet

En HashSet är en datastruktur som lagrar unika element utan någon specifik ordning. HashSet använder en hashfunktion för att beräkna hashvärdet för varje element, vilket möjliggör snabb insättning, borttagning och sökning av element. HashSet används när du behöver lagra en samling av unika element och inte behöver behålla någon specifik ordning.

```
import java.util.HashSet;
public class Main {
    public static void main(String[] args) {
        HashSet<String> villains = new HashSet<String>();
        villains.add("Joker");
        villains.add("Loki");
        villains.add("Green Goblin");
        System.out.println(villains.size()); // Output: 3
    }
}
```

I exemplen ovan har vi använt namnen på några av de populära superhjältarna och skurkarna från DC och Marvel för att illustrera användningen av olika datastrukturer. Precis som superhjältar och skurkar kan vara olika och har olika förmågor, har också datastrukturerna olika egenskaper och användningsområden.

Ännu fler datastrukturer

Exemplen ovan är inte alla datastrukturer som finns. Artikeln fokuserar på några vanliga datastrukturer, men det finns fler datastrukturer som kan vara användbara inom programmering och datavetenskap. Här är några ytterligare exempel på datastrukturer som kan vara värda att undersöka:

- Länkad lista: En datastruktur där elementen är länkade tillsammans i en kedja.
- Träd: En hierarkisk datastruktur där varje element har en överordnad och noll eller flera underordnade element.
- Graf: En datastruktur som består av noder (vertices) och kanter (edges) som förbinder dessa noder.
- Heap: En speciell typ av trädstruktur där varje nod har ett värde större eller mindre än eller lika med sina underordnade noder.
- Trie: En speciell typ av trädstruktur som används för att effektivt lagra och söka efter ord och mönster.
- Hash-tabell: En datastruktur som använder en hashfunktion för att snabbt lagra och hämta värden baserat på en given nyckel.
- Graf: En datastruktur som består av noder (vertices) och kanter (edges) som förbinder dessa noder. Grafer används för att representera relationer och nätverk mellan olika objekt.—
- Heap: En speciell typ av trädstruktur där varje nod har ett värde större eller mindre än eller lika med sina underordnade noder. Heapar används ofta för att implementera prioritetssköer och sorteringsalgoritmer som heap sort.
- Trie: En speciell typ av trädstruktur som används för att effektivt lagra och söka efter ord och mönster. Tries används ofta i applikationer som textkomprimering, stavningskontroll och autokomplettering.
- Hash Table: En datastruktur som använder en hashfunktion för att snabbt lagra och hämta värden baserat på en given nyckel. Hash-tabeller är användbara för att implementera snabbt åtkomstbara datastrukturer som uppslagslistor och uppslagsböcker.
- Set: En datastruktur som lagrar unika element utan någon specifik ordning. Set kan användas för att utföra olika mängdoperationer som union, snitt och differens.
- Stack: En datastruktur som följer principen "last in, first out" (LIFO), vilket innebär att det senast tillagda elementet är det första som tas bort. Stackar används ofta för att hantera återuppringsinformation i funktioner eller för att utvärdera uttryck i matematiska uttryck.
- Queue: En datastruktur som följer principen "first in, first out" (FIFO), vilket innebär att det första tillagda elementet är det första som tas bort. Köer används ofta för att hantera processer och trådar eller för att implementera buffertar i kommunikationssystem. Dessa är bara några exempel på datastrukturer som du kan lära dig mer om. Varje datastruktur har sina egna unika egenskaper och användningsområden, och det kan vara värdefullt att utforska dem för att utöka din kunskap om datastrukturer och deras tillämpningar.

Referenser

- Wikipedia
- GeeksforGeeks
- TutorialsPoint
- Programiz
- Khan Academy
- Codecademy
- CodeWithHarry

Java för Jin23

Marcus Medina

Array övningar

1. Skapa en array
2. Tilldela värden till en array
3. Hämta värden från en array
4. Uppdatera värden i en array
5. Loopa igenom en array
6. Array av objekt
7. Tvådimensionell array
8. Sök efter ett värde i en array
9. Sortera en array
10. Summera värden i en array

Array övningar

Här finns några övningar för att träna på att använda arrayer i Java.

▼ Innehållsförteckning

1. Innehållsförteckning

1. Skapa en array

Skapa en array med namnet `numbers` som innehåller följande tal: 1, 2, 3, 4, 5.

▼ Lösning

```
int[] numbers = {1, 2, 3, 4, 5};
```

2. Tilldela värden till en array

Skapa en array med namnet `fruits` som kan hålla 3 fruktnamn. Tilldela värdena "apple", "banan" och "apelsin" till arrayen.

▼ Lösning

```
String[] fruits = {"apple", "banan", "apelsin"};
```

3. Hämta värden från en array

Använd indexering för att hämta det andra värdet från arrayen `numbers`. `int andraTalet = numbers[1];`

4. Uppdatera värden i en array

Uppdatera det första värdet i arrayen `fruits` till "päron".

▼ Lösning

```
fruits[0] = "päron";
```

5. Loopa igenom en array

Använd en for-loop för att skriva ut varje frukt i arrayen `fruits`.

▼ Lösning

```
for (int i = 0; i < fruits.length; i++) {  
    System.out.println(fruits[i]);  
}
```

6. Array av objekt

Skapa en array med namnet `characters` som kan hålla objekt av typen `Character`. Skapa två `Character`-objekt med namnen "Iron Man" och "Captain America" och placera dem i arrayen.

▼ Lösning

```
Character[] characters = new Character[2];  
characters[0] = new Character("Iron Man");  
characters[1] = new Character("Captain America");
```

7. Tvådimensionell array

Skapa en tvådimensionell array med namnet `matrix` som har 3 rader och 3 kolumner. Fyll arrayen med värdena 1, 2, 3, 4, 5, 6, 7, 8, 9.

▼ Lösning

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}};
```

8. Sök efter ett värde i en array

Skriv en metod med namnet `containsValue` som tar emot en array av heltal och ett heltal att söka efter. Metoden ska returnera `true` om värdet finns i arrayen, annars `false`.

▼ Lösning

```
public static boolean containsValue(int[] array, int value) {  
    for (int i = 0; i < array.length; i++) {  
        if (array[i] == value) {  
            return true;  
        }  
    }  
    return false;  
}
```

9. Sortera en array

Skriv en metod med namnet `sortArray` som tar emot en array av heltal och sorterar den i stigande ordning.

▼ Lösning

```
public static void sortArray(int[] array) {  
    Arrays.sort(array);  
}
```

10. Summera värden i en array

Skriv en metod med namnet `sumArray` som tar emot en array av heltal och returnerar summan av alla värden i arrayen.

▼ Lösning

```
public static int sumArray(int[] array) {  
    int sum = 0;  
    sum += array[i];  
    return sum;  
}
```

Lycka till med övningarna!

Java för Jin23

Marcus Medina

Plocka ut en del av en array och skapa en ny array av det.

Plocka ut en del av en array och skapa en ny array av det

Kodexempel

Resultat

Plocka ut en del av en array och skapa en ny array av det.

▼ Innehållsförteckning

1. Innehållsförteckning

Plocka ut en del av en array och skapa en ny array av det

I detta exempel visar vi hur du kan plocka ut en del av en array och skapa en ny array av det i Java. Vi skapar en ursprunglig array med 10 heltal och kopierar de fem första talen till en ny array.

Kodexempel

```
public class MainClass {  
    public static void main(String[] args) {  
        // Skapa en array med 10 heltal  
        int[] heltal = { 5, 2, 7, 1, 9, 3, 8, 4, 6, 10 };  
  
        // Skriv ut den ursprungliga arrayen  
        System.out.print("Siffror: ");  
        printArray(heltal);  
  
        // Skapa en ny array som innehåller de fem första talen i den  
        // ursprungliga arrayen  
        int[] femForsta = new int[5];  
        System.arraycopy(heltal, 0, femForsta, 0, 5);  
  
        // Skriv ut den nya arrayen  
        System.out.print("De fem första talen: ");  
        printArray(femForsta);  
  
        System.out.println();  
    }  
    private static void printArray(int[] array) {  
        for (int num : array) {  
            System.out.print(num + " ");  
        }  
    }  
}
```

Resultat

Siffror: 5 2 7 1 9 3 8 4 6 10

De fem första talen: 5 2 7 1 9

I det här exemplet skapar vi en array `heltal` med 10 heltal. Vi använder `System.arraycopy` för att kopiera de fem första talen från `heltal` till en ny array `femForsta`. Vi skriver sedan ut både den ursprungliga arrayen och den nya arrayen genom att använda metoden `printArray`.

Metoden `printArray` används för att skriva ut innehållet i en array. Den använder en förbättrad `for`-loop för att iterera över varje element i arrayen och skriva ut det.

Observera att detta bara är ett exempel på hur du kan lösa uppgiften. Det finns flera sätt att välja en del av en array i Java. Du kan också använda metoder som `Arrays.copyOfRange` eller använda en `for`-loop för att kopiera elementen manuellt.

Det är viktigt att notera att `arrayindex` i Java börjar på 0. I exemplet ovan kopierar vi elementen från index 0 till index 4, vilket motsvarar de fem första talen i arrayen.

Det är alltid bra att experimentera och utforska olika sätt att lösa problemet på egen hand. Genom att göra det kan du utöka din förståelse för arrayer och Java-programmering som helhet.

Jag hoppas att detta exempel har varit användbart för dig att förstå hur man plockar ut en del av en array och skapar en ny array av det i Java. Om du har fler frågor eller behöver ytterligare hjälp, tveka inte att fråga!

Java för Jin23

Marcus Medina

Summera element i en array

Beskrivning av övningen

Kodmall

Summera element i en array

▼ Innehållsförteckning

Beskrivning av övningen

Skriv en metod som tar emot en array av heltal och summerar alla element i arrayen. Returnera den resulterande summan. Det är bra om du kan lösa uppgiften utan att använda LINQ. Om du vill kan du också lösa uppgiften med hjälp av LINQ. Det är dock inte ett krav. Men det är alltid bra att vara bekant med LINQ. Även om du helst inte vill använda LINQ i denna uppgift kan det vara bra att lösa uppgiften med hjälp av LINQ också för att se hur det kan göras.

Kodmall

```
public class Main {  
    public static int sumArray(int[] numbers) {  
        // Implementera kod här  
    }  
    public static void main(String[] args) {  
        int[] numbers = { 1, 2, 3, 4, 5 };  
        int sum = sumArray(numbers);  
        System.out.println(sum);  
    }  
}
```

Förväntad output

15

Facit

▼ Klicka här för att se facit

```
int sum = 0;  
for (int i = 0; i < numbers.length; i++) {  
    sum += numbers[i];  
}  
return sum;
```

OBS! Detta kan också lösas med hjälp av den inbyggda metoden `sum` i klassen `Arrays`:

```
return Arrays.stream(numbers).sum();
```

Java för Jin23

Marcus Medina

Hitta det största elementet i en array

Kodmall

Förväntad output

Facit

Hitta det största elementet i en array

Skriv en metod som tar emot en array av heltal och returnerar det största elementet i arrayen.

Det är bra om du kan lösa uppgiften utan att använda LINQ. Om du vill kan du också lösa uppgiften med hjälp av LINQ. Det är dock inte ett krav. Men det är alltid bra att vara bekant med LINQ. Även om du helst inte vill använda LINQ i denna uppgift kan det vara bra att lösa uppgiften med hjälp av LINQ också för att se hur det kan göras.

Kodmall

```
public static int findLargestElement(int[] numbers) {  
    // Implementera kod här  
}  
  
// Exempelanvändning  
int[] numbers = { 5, 8, 2, 11, 3 };  
int largest = findLargestElement(numbers);  
System.out.println(largest);
```

Förväntad output

11

Facit

▼ Här är lösningen

```
int largest = numbers[0];  
for (int i = 1; i < numbers.length; i++) {  
    if (numbers[i] > largest) {  
        largest = numbers[i];  
    }  
}  
return largest;
```

OBS! Detta kan också lösas med hjälp av det inbyggda metoden `max()` i klassen `Arrays`:

```
return Arrays.stream(numbers).max().getAsInt();
```

Java för Jin23

Marcus Medina

Invertera en array

Beskrivning av övningen

Kodmall

Förväntad output

Facit

Invertera en array

▼ Innehållsförteckning

Beskrivning av övningen

Skriv en metod som tar emot en array av strängar och inverterar ordningen på elementen i arrayen. Metoden ska returnera den resulterande inverterade arrayen.

Kodmall

```
public class Main {  
    public static String[] reverseArray(String[] words) {  
        // Implementera kod här  
    }  
    public static void main(String[] args) {  
        String[] words = { "Katt", "Hund", "Kanin", "Capybara" };  
        String[] reversed = reverseArray(words);  
        System.out.println(String.join(", ", reversed));  
    }  
}
```

Förväntad output

Capybara, Kanin, Hund, Katt

Facit

▼ Klicka här för att se facit

```
public static String[] reverseArray(String[] words) {  
    String[] reversed = new String[words.length];  
    int index = 0;  
    for (int i = words.length - 1; i >= 0; i--) {  
        reversed[index] = words[i];  
        index++;  
    }  
    return reversed;  
}
```

OBS! Detta kan också lösas med hjälp av metoden `reverse()` i klassen `Collections`:

```
List<String> list = Arrays.asList(words);  
Collections.reverse(list);  
return list.toArray(new String[list.size()]);
```

I Java kan du också använda Arrays-klassen för att arbeta med arrayer. I det här exemplet har vi en metod `reverseArray` som tar emot en array av strängar och returnerar en ny array där ordningen av elementen har vänts. Vi använder `stream()`-metoden för att skapa en ström av elementen i arrayen. Sedan använder vi metoden `reversed()` för att vända på ordningsföljden av elementen i strömmen. Slutligen använder vi metoden `toArray()` för att konvertera strömmen till en ny array.

```
public static String[] reverseArray(String[] words) {  
    return Arrays.stream(words)  
        .reversed()  
        .toArray(String[]::new);  
}
```

Java för Jin23

Marcus Medina

Arrayer

Exempel

Slutsats

Obligatorisk dad-joke

Arrayer

En array är en grundläggande datastruktur inom programmering som tillåter lagring av en samling av element av samma datatyp. Den används för att organisera och hantera större mängder data på ett strukturerat sätt.

För att deklarerar och skapa en array i Java använder vi följande syntax:

```
dataType[] arrayName = new dataType[arrayLength];
```

Här är en förklaring av de olika delarna av arraydeklarationen:

- `dataType`: Anger den datatyp som elementen i arrayen kommer att ha, t.ex. `int`, `String`, `boolean`, osv.
- `arrayName`: Namnet på arrayen, som du kan välja själv.
- `arrayLength`: Antalet element som arrayen ska ha.

Exempelvis kan vi deklarerar och skapa en array av heltal med 5 element på följande sätt:

```
int[] numbers = new int[5];
```

Efter att en array har skapats kan vi tilldela värden till dess enskilda element och få tillgång till elementen genom att använda deras index. Indexeringen i en array börjar alltid på 0, vilket innebär att det första elementet i arrayen har index 0, det andra elementet har index 1, och så vidare.

Här är ett exempel som tilldelar värden till några element i vår tidigare skapade array och sedan skriver ut dessa värden:

```
numbers[0] = 10;  
numbers[1] = 20;  
numbers[2] = 30;  
numbers[3] = 40;  
numbers[4] = 50;
```

```
System.out.println(numbers[0]); // Output: 10  
System.out.println(numbers[2]); // Output: 30  
System.out.println(numbers[4]); // Output: 50
```

Det finns olika metoder och egenskaper som kan användas tillsammans med arrayer i Java för att underlätta deras hantering och bearbetning. Till exempel kan vi använda egenskapen `length` för att få antalet element i en array:

```
System.out.println(numbers.length); // Output: 5
```

Det är viktigt att komma ihåg att storleken på en array inte kan ändras efter att den har skapats. Om vi behöver lägga till eller ta bort element i en samling på ett dynamiskt sätt kan det vara mer lämpligt att använda andra datastrukturer, som `ArrayList` i Java.

Exempel

Här är ett exempel som visar hur man kan använda en array för att lagra och manipulera en samling av namn:

```
String[] heroes = new String[3];
heroes[0] = "Batman";
heroes[1] = "Spider-Man";
heroes[2] = "Wonder Woman";

System.out.println(heroes[1]); // Output: Spider-Man

heroes[1] = "Iron Man";
System.out.println(heroes[1]); // Output: Iron Man
```

I det här exemplet deklarerar vi en array av strängar och tilldelar värden till några av elementen. Vi använder sedan indexering för att komma åt och uppdatera ett av elementen i arrayen.

Detta är bara en grundläggande introduktion till arrayer i Java. Det finns mycket mer att lära sig om hur man hanterar och bearbetar arrayer, inklusive användning av loopar för att iterera över elementen och olika metoder för att söka, sortera och filtrera arrayer. Men förhoppningsvis ger denna introduktion dig en bra startpunkt för att förstå grunderna i arrayhantering.

Slutsats

Arrayer är en viktig datastruktur inom programmering som används för att organisera och hantera samlingar av element av samma datatyp. Genom att använda indexering kan vi komma åt och manipulera enskilda element i en array. Det är viktigt att komma ihåg att storleken på en array är fastställd vid skapandet och inte kan ändras i efterhand. Om du behöver en dynamiskt storlek anpassningsbar samling, kan andra datastrukturer som `ArrayList` vara mer lämpliga att använda.

För att bli mer bekant med användningen av arrayer och utforska avancerade funktioner och tekniker, rekommenderas det att du fortsätter din läsning och experimenterar med kodexempel. Det finns många resurser tillgängliga online, inklusive dokumentation och handledningar, som kan hjälpa dig att fördjupa din förståelse och behärskning av arrayer i Java.

Lycka till med ditt fortsatta lärande och utforskande av arrayer!

Obligatorisk dad-joke

Varför älskar programmerare att använda arrayer?

För att de känner sig som riktiga "array-stokrater"!

Java för Jin23

Marcus Medina

Dictionary (HashMap)

Beskrivning

Exempel

Slutsats

TL;DR

Dictionary (HashMap)

▼ Innehållsförteckning

1. TOC

Beskrivning

En Dictionary är en datastruktur som tillåter oss att lagra och hämta värden baserat på nycklar. Den fungerar på ett liknande sätt som en telefonbok där vi kan slå upp ett namn (nyckel) för att få fram ett telefonnummer (värde). Dictionary är en kraftfull datastruktur inom programmering som erbjuder snabb åtkomst och effektiva sökningar. I java finns det flera olika klasser som kan användas för att implementera en Dictionary, t.ex. HashMap, TreeMap och LinkedHashMap. Vi ska använda HashMap.

Exempel

Här är ett exempel på hur man kan använda en Dictionary i Java:

```

import java.util.HashMap;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        // Skapa en HashMap för att lagra hjältarnas namn och deras
        // hemvärld
        Map<String, String> heroes = new HashMap<>();

        // Lägg till värden i HashMap
        heroes.put("Superman", "DC");
        heroes.put("Spider-Man", "Marvel");
        heroes.put("Batman", "DC");

        // Hämta hemvärlden för en specifik hjälte
        String supermanHomeWorld = heroes.get("Superman");
        System.out.println("Hemvärld för Superman: " +
            supermanHomeWorld);

        // Uppdatera hemvärlden för en hjälte
        heroes.put("Spider-Man", "DC");

        // Kontrollera om en hjälte finns i HashMap
        if (heroes.containsKey("Batman")) {
            System.out.println("Batman finns i HashMap.");
        }

        // Ta bort en hjälte från HashMap
        heroes.remove("Superman");

        // Loopa genom alla hjältarnas namn och hemvärldar i HashMap
        for (Map.Entry<String, String> entry : heroes.entrySet()) {
            String heroName = entry.getKey();
            String homeWorld = entry.getValue();
            System.out.println("Hjälte: " + heroName + ", Hemvärld: " +
                homeWorld);
        }
    }
}

```

I det här exemplet används klassen HashMap från java.util-paketet för att implementera en HashMap i Java. Vi använder put-metoden för att lägga till värden i HashMap och get-metoden för att hämta värdet för en specifik nyckel. Vi kan också använda containsKey-metoden för att kontrollera om en nyckel finns i HashMap och remove-metoden för att ta bort en nyckel och dess tillhörande värde. Genom att använda en for-each-loop och entrySet-metoden kan vi loopa genom alla nycklar och värden i HashMap och utföra önskade operationer.

Detta är bara ett grundläggande exempel på hur man kan använda en HashMap i Java. Det finns mycket mer att utforska och lära sig om denna kraftfulla datastruktur.

Slutsats

En HashMap är en värdefull datastruktur som tillåter oss att lagra och hämta värden baserat på nycklar. Den erbjuder snabb åtkomst och effektiva sökningar, vilket gör den till en populär datastruktur inom programmering. Genom att använda en HashMap kan vi effektivt hantera och organisera data i våra program.

För att lära dig mer om HashMap och hur den kan användas i Java-programmering, rekommenderas att du utforskar dokumentationen och exempelkod på Oracles officiella webbplats eller andra resurser som erbjuder fördjupad information om ämnet.

TL;DR

En HashMap är en datastruktur som låter oss lagra och hämta värden baserat på nycklar. Den ger snabb åtkomst och effektiva sökningar, vilket gör den till ett kraftfullt verktyg inom programmering.

Obligatorisk dad-joke: Varför älskar programmerare att använda dictionaries? För att de alltid vill ha en "key" till framgång! ``

Jag har rättat formateringen och justerat några delar av din artikel. Var noga med att dubbelkolla innehållet och ändra tillbaka eventuella delar som inte stämmer överens med din ursprungliga avsikt.

Java för Jin23

Marcus Medina

Loopar

 Typer av loopar

 Mer delar av loopar

Loopar

Loopar är en viktig del av programmering. De används för att upprepa en viss uppsättning instruktioner eller handlingar ett visst antal gånger eller tills ett specifikt villkor uppfylls. Loopar gör det möjligt att automatisera och effektivisera repetitiva uppgifter i koden. I denna artikel kommer vi att utforska olika typer av loopar som används inom programmering och fokusera på deras användning i språket Java. Vi kommer att lära oss hur man skapar loopar, vilka villkor som kan användas för att kontrollera loopens beteende och vilka försiktighetsåtgärder man bör ta för att undvika oändliga loopar.

Typer av loopar

Det finns flera typer av loopar som används i Java. Vi kommer att titta närmare på tre av de vanligaste looparna: `for`, `while` och `do-while`.

Mer delar av loopar

Det finns också andra typer av loopar och looprelaterade koncept i Java, till exempel `break` och `continue` satser som kan användas för att styra loopens beteende. Det är också möjligt att använda rekursion för att uppnå loop-liknande beteende genom att en funktion anropar sig själv upprepade gånger.

Java för Jin23

Marcus Medina

Loopar i Java

- Introduktion
- Fördelar
- Begränsningar
- Användningsområden
- Typer av loopar i Java
- Exempel
- Slutsats
- TL;DR
- Termer
- Obligatorisk dad joke

Loopar i Java

Loopar är ett mycket användbart verktyg inom Java-programmering för att automatisera processer, förbättra prestanda och lösa komplexa problem. Genom att använda lämpliga loopar kan vi iterera över listor, utföra beräkningar och hantera användarinmatning. Det är viktigt att vara medveten om begränsningarna och utmaningarna vid användning av loopar och implementera dem korrekt för att undvika potentiella problem.

▼ Innehållsförteckning

1. TOC

Introduktion

Loopar är en viktig del av programmering som används för att upprepa ett block av kod tills ett visst villkor är uppfyllt. Det finns flera olika typer av loopar som kan användas i Java, inklusive for, while, do-while och foreach.

Fördelar

Loopar är ett effektivt sätt att automatisera upprepade processer. De kan också användas för att förbättra prestandan och effektivisera kod. Loopar kan också hjälpa till att förenkla komplexa problem och göra det möjligt att lösa dem på ett enkelt sätt.

Begränsningar

Loopar kan vara tidskrävande och kräva mycket processorresurser. Om loopen inte är korrekt skriven, kan den också leda till att programmet kraschar. Det är viktigt att förstå och vara medveten om eventuella utmaningar eller negativa aspekter som kan uppstå vid användning av loopar.

Användningsområden

Loopar används ofta för att iterera igenom en lista av objekt eller för att upprepa en process tills ett visst villkor är uppfyllt. De kan också användas för att utföra beräkningar, hantera inmatning från användaren eller för att skapa komplexa algoritmer.

Typer av loopar i Java

Loop	Beskrivning
For	En loop som körs ett visst antal gånger.
While	En loop som körs så länge som villkoret är sant.
Do While	En loop som körs minst en gång. Den körs sedan så länge som villkoret är sant.
Foreach	En loop som itererar över en lista av objekt.

Exempel

Här är ett exempel på en for-loop som används för att iterera igenom en lista:

```
int[] list = {1, 2, 3, 4, 5};
for (int i = 0; i < list.length; i++) {
    System.out.println(list[i]);
}
```

Ovanstående loop kommer att skriva ut alla element i listan.

Slutsats

Loopar är ett mycket användbart verktyg för att automatisera processer och förenkla komplexa problem. De kan också användas för att förbättra prestandan och effektivisera kod. Det är viktigt att förstå begreppet och vara medveten om eventuella utmaningar eller negativa aspekter som kan uppstå vid användning av loopar.

TL;DR

Loopar är ett mycket användbart verktyg för att automatisera processer och förenkla komplexa problem. De kan användas för att iterera igenom en lista, utföra beräkningar, hantera användarinmatning eller skapa algoritmer. Det är viktigt att förstå begreppet och vara medveten om eventuella utmaningar eller negativa aspekter som kan uppstå vid användning av loopar.

Termer

Term	Förklaring
Algoritm	En algoritm är lista av kommandon i steg för steg- beskrivning, för att lösa ett problem.
Do While	En loop som körs minst en gång. Den körs sedan så länge som villkoret är sant.
For	En loop som körs ett visst antal gånger.
Foreach	En loop som itererar över en lista av objekt.
While	En loop som körs så länge som villkoret är sant.
Loop	En struktur i programmering som gör att en viss kod kan köras upprepade gånger tills ett visst villkor är uppfyllt.
Villkor	Ett uttryck som utvärderas till sant eller falskt.
Iteration	En enskild körning av kod inuti en loop.
Objekt	En instans av en klass.
Klass	En mall för ett objekt.
Metod	En funktion som tillhör ett objekt.
Variabel	En variabel är en behållare för ett värde.
Attribut	En variabel som tillhör ett objekt.

Obligatorisk dad joke

Varför var looparna så bra på att gå på fest?

För att de alltid kunde "loop-a" runt dansgolvet! ☐

Java för Jin23

Marcus Medina

For loopar

For-loop

Fördelar

Nackdelar

For loopar

For-loop

En for-loop används när vi vill upprepa en uppsättning instruktioner ett känt antal gånger. Syntaxen för en for-loop är följande:

```
for (initialisering; villkor; iteration) {  
    // Kod som ska upprepas  
}
```

Här är en förklaring av de olika delarna i en for-loop:

- **initialisering:** Här initieras en räknare eller en variabel som används för att kontrollera antalet iterationer. Detta görs vanligtvis genom att tilldela ett startvärde till räknaren.
- **villkor:** Här definieras villkoret som kontrollerar om loopen ska fortsätta att upprepas eller avslutas. Om villkoret är sant, fortsätter loopen att köras. Om villkoret är falskt, avslutas loopen och programmet fortsätter med koden efter loopen.
- **iteration:** Här specificeras hur räknaren eller variabeln ska ändras vid varje iteration. Vanligtvis ökar eller minskar man värdet på räknaren med ett visst steg.

Här är ett exempel på en for-loop som skriver ut talen 1 till 5:

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

I exemplet ovan initieras räknaren (variabeln) `i` till 1. Sedan kontrolleras villkoret `i <= 5`. Om villkoret är sant, skrivs värdet på räknaren ut och räknaren ökar med ett steg. Detta upprepas tills villkoret är falskt, dvs. när räknaren når värdet 6.

```
'''text 1 2 3 4 5 '''
```

Fördelar

For-loopar har flera fördelar jämfört med andra typer av loopar:

- De är enkla att använda och förstå.
- De är effektiva eftersom de inte kräver några extra variabler eller villkor för att kontrollera antalet iterationer.
- De är säkrare än andra typer av loopar eftersom de inte kräver någon manuell hantering av index eller iterationsspecifikationer.

Nackdelar

For-loopar har också några nackdelar jämfört med andra typer av loopar:

- De är inte lika flexibla som andra typer av loopar eftersom de kräver att du känner till antalet iterationer i förväg.

- De är inte lika läsbara som andra typer av loopar eftersom de kräver att du känner till antalet iterationer i förväg.
- Man kan få problem om man lägger till eller tar bort element i samlingen under iterationen.

Java för Jin23

Marcus Medina

Foreach-loop

Fördelar

Nackdelar

Foreach-loop

En foreach-loop används för att iterera över en samling av objekt eller värden och utföra en handling för varje element i samlingen. Syntaxen för en foreach-loop är följande:

```
for (var element : samling) {  
    // Kod som ska utföras för varje element  
}
```

Här är en förklaring av de olika delarna i en foreach-loop:

- `element`: En variabel som används för att representera varje element i samlingen när loopen itererar över den.
- `samling`: Den samling av objekt eller värden som ska itereras över, t.ex. en lista, ett fält eller en array.

Här är ett exempel på en foreach-loop som skriver ut varje element i en lista:

```
List<Integer> numbers = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));  
  
for (int number : numbers) {  
    System.out.println(number);  
}
```

Resultatet av koden ovan kommer att vara:

```
1  
2  
3  
4  
5
```

Foreach-loopar är särskilt användbara när du vill arbeta med varje element i en samling utan att behöva oroa dig för att hantera indexer och iterationsspecifikationer manuellt.

Fördelar

Foreach-loopar har flera fördelar jämfört med andra typer av loopar:

- De är enkla att använda och förstå.
- De är säkrare än andra typer av loopar eftersom de inte kräver någon manuell hantering av indexer eller iterationsspecifikationer.
- De är mer läsbara än andra typer av loopar eftersom de inte kräver någon manuell hantering av indexer eller iterationsspecifikationer.

Nackdelar

Foreach-loopar har också några nackdelar:

- De kan inte användas för att iterera över en samling baklänges.
- Om du har värden i samlingen kan du inte ändra dem medan du itererar över dem. Du måste kopiera värdena till en annan samling först.

Java för Jin23

Marcus Medina

Nästlade loopar

Nästlade loopar

Nästlade loopar är loopar som finns inuti en annan loop.

Detta innebär att den inre loopen upprepas lika många gånger som den yttre loopen bestämmer.

Om tänker dig att du ska säga "A B C" tre gånger.

Du säger då "ABC ABC ABC"

I kodform skulle detta bli

```
char[] letters = {'A', 'B', 'C'};

for(int i = 0; i < 3; i++) {
    foreach(char letter in letters) {
        System.out.print(letter);
    }
    system.out.print(' ');
}
```

med resultatet

ABC ABC ABC

De används när vi behöver upprepa en uppsättning instruktioner i flera dimensioner. Till exempel kan vi ha en loop som representerar rader och en inre loop som representerar kolumner för att bearbeta en matris. Här är ett exempel på en nästlad loop:

```
for (int row = 1; row <= 3; row++) {
    for (int col = 1; col <= 3; col++) {
        System.out.println("Row: " + row + ", Column: " + col);
    }
}
```

med följande resultat:

```
Row: 1, Column: 1
Row: 1, Column: 2
Row: 1, Column: 3
Row: 2, Column: 1
Row: 2, Column: 2
Row: 2, Column: 3
Row: 3, Column: 1
Row: 3, Column: 2
Row: 3, Column: 3
```

När nästlade loopar används är det viktigt att vara medveten om prestanda och hur många iterationer som utförs. Nästlade loopar kan öka tidskomplexiteten för ett program och leda till längre exekveringstider om de inte används effektivt.

Loopar är kraftfulla verktyg som hjälper till att automatisera upprepningsuppgifter och utföra operationer på en samling av objekt eller värden. Genom att välja rätt typ av loop och använda den på rätt sätt kan vi skapa mer effektiv och strukturerad kod.

Java för Jin23

Marcus Medina

While

- Fördelar
- Begränsningar
- Användningsområden
- Exempel
- Sammanfattning

While

While-loopar är en typ av loop inom programmering som används för att upprepa en kodsekvens så länge ett visst villkor är sant. Det är en grundläggande kontrollstruktur som tillåter oss att utföra repeterade handlingar utan att behöva skriva samma kod flera gånger. I en while-loop utvärderas villkoret före varje iteration, och om det är sant körs loopen och kodblocket inuti utförs. När villkoret blir falskt avslutas loopen och programmet fortsätter med den efterföljande koden.

Fördelar

While-loopar erbjuder flera fördelar och används i olika situationer:

1. **Upprepa kod:** En while-loop tillåter oss att upprepa en kodsekvens ett godtyckligt antal gånger baserat på ett villkor. Det är särskilt användbart när vi behöver utföra samma handlingar eller beräkningar igen och igen tills ett visst kriterium är uppfyllt.
2. **Enkelhet:** While-loopar är relativt enkla att förstå och implementera. De har en tydlig struktur och kräver inte avancerade koncept eller komplicerade syntaxregler.
3. **Effektivitet:** Genom att använda en while-loop kan vi undvika att upprepa samma kod manuellt. Istället kan vi använda en loop för att automatisera upprepningen och minska kodupprepning, vilket sparar tid och minskar risken för fel.
4. **Anpassningsbarhet:** Med en while-loop kan vi enkelt ändra villkoret som avgör när loopen ska avslutas. Det ger oss flexibilitet att anpassa loopen efter olika scenarier och ändra beteendet med hjälp av villkor.
5. **Möjlighet till oändlig loop:** Genom att använda en lämpligt konstruerad while-loop kan vi skapa en oändlig loop, vilket innebär att loopen fortsätter att köras tills ett avbrytningsvillkor uppfylls. Detta kan vara användbart i vissa situationer där vi vill att en kodsekvens ska köras kontinuerligt tills vi explicit bryter ut ur loopen.

Begränsningar

Trots sina fördelar har while-loopar vissa begränsningar att vara medveten om:

1. **Risk för oändlig loop:** Om villkoret i en while-loop alltid är sant kan loopen köra för evigt och leda till att programmet fryser eller inte svarar. Det är viktigt att vara försiktig när man skapar while-loopar och se till att det finns ett sätt att bryta ut ur loopen när det behövs.
2. **Uppdatering av villkor:** För att undvika oändliga loopar eller loopar som aldrig körs måste vi se till att uppdatera villkoret i while-loopen på rätt sätt. Annars kan loopen antingen inte köras alls eller köras för evigt utan att uppfylla det avsedda syftet.

3. **Risk för buggar:** Om vi inte är noggranna när vi skriver villkoret i en while-loop kan det leda till buggar och felaktigt beteende i vårt program. Det är viktigt att noggrant testa och validera villkoret för att undvika sådana problem.

Användningsområden

While-loopar kan användas i en rad olika situationer och scenarier, inklusive:

1. **Inmatningsvalidering:** Vi kan använda en while-loop för att validera användarinput och be användaren att ange korrekta värden tills det görs.
2. **Iteration över en samling:** Med en while-loop kan vi iterera över en samling av objekt och utföra en viss åtgärd för varje objekt tills ett visst villkor är uppfyllt.
3. **Uppprepning av en kodsekvens:** Vi kan använda en while-loop för att upprepa en kodsekvens tills ett visst villkor är sant. Det kan vara användbart när vi vill utföra en uppgift eller en beräkning ett visst antal gånger eller tills ett specifikt kriterium är uppfyllt.
4. **Simulering av händelser:** Med en while-loop kan vi simulera händelser och uppdatera tillståndet på olika delar av vårt program tills ett visst villkor är uppfyllt.

Exempel

Här är ett exempel som visar hur man använder en while-loop i Java:

```
public class WhileExample {  
    public static void main(String[] args) {  
        int count = 0;  
        while (count < 5) {  
            System.out.println("Count: " + count);  
            count++;  
        }  
    }  
}
```

I detta exempel använder vi en while-loop för att skriva ut värdet på variabeln count tills count når värdet 5. För varje iteration ökar vi värdet på count med 1 och skriver sedan ut värdet. Loopen kommer att fortsätta tills count når värdet 5.

Output: Count: 0 Count: 1 Count: 2 Count: 3 Count: 4

Detta är bara ett exempel på hur man kan använda en while-loop. Det finns många fler möjligheter och variationer beroende på dina specifika behov och scenarier.

Sammanfattning

While-loopar är en viktig kontrollstruktur inom programmering som låter oss upprepa en kodsekvens så länge ett villkor är sant. De erbjuder enkelhet, effektivitet och anpassningsbarhet i våra program. Det är viktigt att vara medveten om riskerna med oändliga loopar och att se till att uppdatera villkoret på rätt sätt för att undvika buggar. Med rätt användning kan while-loopar vara kraftfulla verktyg för att hantera upprepade uppgifter och logik i våra program.

Fortsätt utforska och experimentera med while-loopar och andra loopar inom programmering för att bygga robusta och effektiva program.

Java för Jin23

Marcus Medina

Do While

Beskrivning
Exempel
Sammanfattning
Termer

Do While

Do While är en loop som körs minst en gång. Den körs sedan så länge som villkoret är sant.

▼ Innehållsförteckning

1. TOC

Beskrivning

Do While är en loopstruktur i Java som liknar While-loopen. Skillnaden är att i en Do While-loop kontrolleras villkoret efter varje iteration, vilket innebär att loopen alltid körs minst en gång. Detta är användbart när du vill att en viss kod ska utföras minst en gång innan villkoret kontrolleras.

Exempel

För att förstå hur en Do While-loop fungerar, låt oss titta på följande kodexempel:

```
int i = 0;
do {
    System.out.println(i);
    i++;
} while (i < 10);
```

I detta exempel deklarerar vi en variabel `i` och tilldelar den värdet 0. Sedan har vi en Do While-loop som kontrollerar om `i` är mindre än 10. Inuti loopen skriver den ut värdet av `i` och ökar sedan värdet med 1. Loopen fortsätter att köras så länge som `i` är mindre än 10.

Med följande output:

```
0
1
2
3
4
5
6
7
8
9
```

Medan villkoret i Do While-loopen är sant, körs koden inuti loopen. När villkoret blir falskt, avslutas loopen och programmet fortsätter med resten av koden efter loopen.

I detta specifika exempel kommer loopen att köra 10 gånger och skriva ut värdena från 0 till 9.

Sammanfattning

Do While är en loopstruktur i Java som körs minst en gång och sedan fortsätter att köra så länge som villkoret är sant. Det är användbart när du behöver utföra en viss kod minst en gång innan villkoret kontrolleras.

Termer

Term	Förklaring
Do While-loop	En loopstruktur i Java som körs minst en gång och sedan fortsätter att köra så länge som villkoret är sant.
Loop	En struktur i programmering som gör att en viss kod kan köras upprepade gånger tills ett visst villkor är uppfyllt.
Iteration	En enskild körning av kod inuti en loop.
Villkor	Ett uttryck som avgör om en loop ska fortsätta köras eller inte.

Java för Jin23

Marcus Medina

Rekursiva loopar
Rekursion i Java
Obligatorisk dad-joke

Rekursiva loopar

Rekursion är en teknik där en funktion kallar sig själv för att utföra en uppgift. Detta kan vara användbart när du har en uppgift som kan delas upp i mindre liknande deluppgifter. Ett vanligt exempel på rekursion är beräkningen av det n:te Fibonacci-talet. Här är en rekursiv funktion som beräknar Fibonacci-talet:

```
int Fibonacci(int n) {  
    if (n <= 1)  
        return n;  
    else  
        return Fibonacci(n - 1) + Fibonacci(n - 2);  
}
```

Du kan sedan anropa funktionen med önskat värde för att få det motsvarande Fibonacci-talet:

```
int result = Fibonacci(6);  
System.out.println(result); // Resultatet blir 8
```

Bästa sättet att förstå det är att köra det i debugläge och stega ett steg i taget. Skriv om koden såhär:

```
int Fibonacci(int n) {  
    if (n <= 1)  
        return n;  
    else  
        return  
            Fibonacci(n - 1) +  
            Fibonacci(n - 2);  
}
```

Vi försöker förklara flödet

1. Först anropas metoden med värdet 6.
2. Eftersom 6 inte är mindre än eller lika med 1, går vi vidare till else-blocket.
3. Metoden anropar sig själv två gånger med de två föregående talen i Fibonacci-sekvensen, det vill säga `Fibonacci(6 - 1)` och `Fibonacci(6 - 2)`.
4. Nu fortsätter vi att evaluera dessa anrop separat.
5. Första anropet `Fibonacci(6 - 1)` utvärderas till `Fibonacci(5)`.
6. För att beräkna `Fibonacci(5)` måste vi fortsätta att anropa metoden med mindre värden tills vi når basfallet.
7. Detta resulterar i en serie anrop `Fibonacci(4)`, `Fibonacci(3)`, `Fibonacci(2)`, och `Fibonacci(1)`.
8. När vi når basfallet `Fibonacci(1)` returneras värdet 1.
9. Därefter fortsätter vi att evaluera anropen på vägen tillbaka. `Fibonacci(2)` returneras också med värdet 1.
10. Nu kan vi beräkna `Fibonacci(3)` genom att addera `Fibonacci(2)` och `Fibonacci(1)`, vilket ger 2.
11. Vi fortsätter att evaluera anropen och beräkna värdena för `Fibonacci(4)` och `Fibonacci(5)`.
12. Slutligen, när vi har beräknat `Fibonacci(6)` genom att addera `Fibonacci(5)` och `Fibonacci(4)`, returneras värdet 8.

Så resultatet av att anropa `Fibonacci(6)` är 8 enligt definitionen av Fibonacci-sekvensen.

Det är värt att notera att den här implementationen är rekursiv och kan vara ineffektiv för stora värden på n på grund av att den utför många redundanta beräkningar. Det finns alternativa iterativa eller memoiserade implementationer som kan förbättra prestandan för Fibonacci-sekvensen.

Sätt en breakpoint på raden `return n`; och kör programmet. Stega igenom koden och se vad som händer.

Rekursion i Java

Rekursion är en cool teknik som används inom programmering för att lösa problem genom att bryta ner dem i mindre delproblem. Det innebär att en funktion kan anropa sig själv för att lösa ett problem. Det är dock viktigt att ha en stoppvillkor för att undvika oändlig rekursion. Här är ett exempel på rekursion i Java:

```
public class RekursionExempel {
    public static void main(String[] args) {

        int resultat = summera(5);
        System.out.println("Summan är: " + resultat);
    }

    public static int summera(int n) {
        if (n == 1) {
            return 1;
        } else {
            return n + summera(n - 1);
        }
    }
}
```

I det här exemplet använder vi en rekursiv funktion `summera` för att beräkna summan av alla heltal från 1 till det givna talet n . Funktionen har en stoppvillkor som säger att om n är lika med 1, returnera 1. Annars anropar funktionen sig själv med $n-1$ och lägger till n i resultatet. När vi kör programmet kommer det att skriva ut summan av alla heltal från 1 till 5, vilket är 15.

Rekursion kan vara svårt att förstå i början, men med övning och erfarenhet kan du behärska denna kraftfulla teknik. Fortsätt att utforska och experimentera med rekursion för att utveckla dina programmeringskunskaper!

Om du vill ha mer inspiration kan du titta på några av följande filmer:

- Source code
- Happy Death day
- Happy Death day 2U
- Groundhog day
- Edge of tomorrow

Obligatorisk dad-joke

Varför gillar programmerare att använda rekursion i skämt?

För att förstå rekursion i skämt måste du först förstå rekursion i skämt.

Java för Jin23

Marcus Medina

Evig loop

Fördelar

Nackdelar

Exempel

Exempel på oändlig loop

Evig loop

En oändlig loop är en loop som inte har något villkor för att avslutas och kommer att köra kontinuerligt tills programmet avbryts. Det kan vara användbart i vissa situationer, till exempel när du skapar ett program som körs i en slinga och kontinuerligt utför några uppgifter i bakgrunden. Här är ett exempel på en oändlig loop:

```
while (true) {  
    // Kod som körs kontinuerligt  
}
```

Observera att en oändlig loop kan leda till att programmet hänger sig om det inte finns något sätt att avbryta loopen manuellt.

Detta gör man när man testar eller när man har en brytning i loopen som gör att den avslutas.

Fördelar

Oändliga loopar har flera fördelar jämfört med andra typer av loopar:

- De är enkla att använda och förstå.
- De är effektiva eftersom de inte kräver några extra variabler eller villkor för att kontrollera antalet iterationer.
- De är säkrare än andra typer av loopar eftersom de inte kräver någon manuell hantering av index eller iterationsspecifikationer.

Nackdelar

Oändliga loopar har flera nackdelar jämfört med andra typer av loopar:

- De kan vara svåra att felsöka eftersom de inte har något villkor för att avsluta loopen.
- De kan leda till att programmet hänger sig om det inte finns något sätt att avbryta loopen manuellt.
- De kan vara svåra att felsöka eftersom de inte har något villkor för att avsluta loopen.

Exempel

Här är ett exempel på en oändlig loop som skriver ut talen 1 till 5:

```
while (true) {  
    System.out.println("1");  
    System.out.println("2");  
    System.out.println("3");  
    System.out.println("4");  
    System.out.println("5");  
}
```

Resultatet av koden ovan kommer att vara:

```
1
2
3
4
5
1
2
3
osv...
```

Exempel på oändlig loop

Här är ett exempel på en meny som körs i en oändlig loop. Fördelen med detta är att vi inte behöver veta hur många gånger användaren vill upprepa loopen, utan vi kan låta användaren avsluta loopen när den är klar.

```
while (true) {
    System.out.println("Välj ett alternativ:");
    System.out.println("1. Visa alla produkter");
    System.out.println("2. Lägg till en produkt");
    System.out.println("3. Ta bort en produkt");
    System.out.println("4. Avsluta programmet");

    int choice = scanner.nextInt();

    // Observera att switch break inte bryter loopen i sig
    // same name different shit... eller vad man ska säga
    bool breakLoop=false;
    switch (choice) {
        case 1:
            // Visa alla produkter
            break;
        case 2:
            // Lägg till en produkt
            break;
        case 3:
            // Ta bort en produkt
            break;
        case 4:
            // Avsluta programmet
            breakLoop=true;
            break;
        default:
            System.out.println("Felaktigt val");
            break;
    }

    if (breakLoop) break; // bryter loopen
}
```

Java för Jin23

Marcus Medina

Loop kontroller

- Break
- Continue
- Return
- yield

Loop kontroller

För att få mer kontroll på loopar kan vi använda oss av loop kontroller. Dessa är break, continue och return.

Break

break används för att avbryta en loop. Detta kan vara användbart om vi vill avbryta en loop innan den är klar.

```
int max = 5;
for(int i = 0; i < 10; i++) {
    if(i >= max) {
        break;
    }
    System.out.println(i);
}
```

Resultatet av koden ovan kommer att vara:

```
0
1
2
3
4
```

Loopen är en loop som ska gå upp till tio, men vi styr den med break så att den bara går upp till fem som vi angett som maxvärde.

Continue

continue används för att hoppa över en iteration i en loop. Detta kan vara användbart om vi vill hoppa över en iteration i en loop.

```
int mod=3;
for(int i = 0; i < 10; i++) {
    if(i % mod == 0) {
        continue;
    }
    System.out.println(i);
}
```

Resultatet av koden ovan kommer att vara:

```
1
2
4
5
7
8
```

Med mod variabeln kan vi styra hur många iterationer som ska hoppas över. I exemplet ovan hoppar vi över var tredje iteration. Continue används ofta tillsammans med en if-sats. Den gör att loopens hoppas över allt följande kod i loopens och går direkt till nästa iteration.

Return

return används för att avsluta en metod. Detta kan vara användbart om vi vill avsluta en metod innan den är klar.

```
public static void main(String[] args) {  
    int max = 5;  
    for(int i = 0; i < 10; i++) {  
        if(i >= max) {  
            return;  
        }  
        System.out.println(i);  
    }  
}
```

Resultatet av koden ovan kommer att vara:

```
0  
1  
2  
3  
4
```

Hela metoden avslutas när vi når return och vi kommer inte till System.out.println(i); som ligger efter.

yield

yield används för att avsluta en iterator. Detta kan vara användbart om vi vill avsluta en iterator innan den är klar.

```
public static void main(String[] args) {  
    int max = 5;  
    for(int i = 0; i < 10; i++) {  
        if(i >= max) {  
            yield;  
        }  
        System.out.println(i);  
    }  
}
```

Resultatet av koden ovan kommer att vara:

```
0  
1  
2  
3  
4
```

Hela iteratorn avslutas när vi når yield och vi kommer inte till System.out.println(i); som ligger efter. Vad är då skillnaden mellan detta och break? Jo, break avslutar bara loopens, medan yield avslutar hela iteratorn. Detta är användbart om vi vill avsluta en iterator innan den är klar.

Vi kollar på ett bättre exempel:

```
// kalla på en metod som yieldar

public static void main(String[] args) {
    System.out.println("Start");
    System.out.println(GetNumber());
    System.out.println(GetNumber());
    System.out.println(GetNumber());
    System.out.println(GetNumber());
    System.out.println("End");
}

public static int GetNumber() {
    for(int i = 0; i < 10; i++) {
        if(i >= 5) {
            yield;
        }
        System.out.println(i);
    }
    return 0;
}
```

Resultatet av koden ovan kommer att vara:

```
Start
0
1
2
3
4
End
```

Vad händer om vi byter ut yield mot break?

```
// kalla på en metod som yieldar

public static void main(String[] args) {
    System.out.println("Start");
    System.out.println(GetNumber());
    System.out.println(GetNumber());
    System.out.println("End");
}

public static int GetNumber() {
    for(int i = 0; i < 10; i++) {
        if(i >= 5) {
            break;
        }
        System.out.println(i);
    }
    return 0;
}
```

Resultatet av koden ovan kommer att vara:

```
Start
0
1
2
3
4
0
1
2
3
4
End
```

Yield ger oss alltså en möjlighet att avsluta en iterator innan den är klar. Detta är användbart vi vill anropa en metod men bara få en del av resultatet åt gången. Visst är det coolt.

Tänk dig att du har en lista med namn men vill bara arbeta med en åt gången utan att behöva kapsla in din kod i en jätteloop. Då kan du använda yield.

Det är ett sätt att slippa ifrån loopar, vilket man gärna gör i funktionell programmering.

Java för Jin23

Marcus Medina

Objektorienterad programmering (OOP)

När du har läst detta kommer du att kunna

Innehållsförteckning

Introduktion

Fördelar

Begränsningar

Tillämpningsområden

Exempel

Slutsats

TL;DR

Objektorienterad programmering (OOP)

I denna avsnitt kommer vi att titta på ämnet Objektorienterad programmering (OOP).

När du har läst detta kommer du att kunna

- Förstå och förklara vad Objektorienterad programmering (OOP) är och dess relevans inom programmering.
- Diskutera fördelar och begränsningar med Objektorienterad programmering (OOP).
- Identifiera olika användningsområden där Objektorienterad programmering (OOP) kan tillämpas.
- Förstå och tolka ett kodexempel som använder Objektorienterad programmering (OOP).
- Känna dig inspirerad att utforska mer om ämnet och lära dig mer!

Innehållsförteckning

- Introduktion
- Fördelar
- Begränsningar
- Användningsområden
- Exempel
- Slutsats
- TL;DR

Introduktion

Välkommen till världen av Objektorienterad programmering (OOP)! Det är en spännande grej inom programmering som handlar om att organisera kod på ett sätt som liknar hur vi tänker och interagerar med saker i den verkliga världen. Vi skapar objekt som har egenskaper och beteenden, precis som riktiga grejer vi kan pilla på och få dem att göra saker!

Fördelar

Så, varför är OOP så coolt? Jo, det finns faktiskt några riktigt häftiga fördelar med det. Kolla in:

- **Modularitet:** Med OOP kan vi dela upp vår kod i små, självständiga objekt. Det gör det enklare att bygga och underhålla vår kod och ger oss möjlighet att återanvända objekten på olika ställen. Precis som att bygga med lego!—

- **Återanvändbarhet:** Vi kan använda samma objekt om och om igen i olika delar av vår kod eller till och med i olika projekt. Det sparar tid och minskar mängden krånglig kod. Det är bra för oss och bra för vår planet!
- **Lätt att förstå och ändra:** Om vi behöver göra ändringar i en del av koden påverkar det inte resten av systemet. Vi kan fixa buggar eller göra förbättringar utan att oroa oss för att allt annat ska sluta fungera. Det är som att bygga ett hus med olika moduler som kan bytas ut eller uppgraderas utan att påverka resten av huset.
- **Abstraktion:** Genom att använda abstraktion kan vi förenkla komplexa system genom att bara visa de viktigaste detaljerna för användaren. Det är som att köra en bil, vi behöver inte veta hur motorn fungerar för att kunna köra bilen.
- **Flexibilitet:** OOP ger oss möjlighet att skapa hierarkier av objekt och använda koncept som arv och polymorfism. Det ger oss flexibilitet att skapa olika typer av objekt och hantera dem på ett enkelt sätt. Det är som att ha en verktygslåda med olika verktyg som vi kan använda för att lösa olika problem.

Begränsningar

Nu ska vi vara ärliga här, OOP har också sina begränsningar och utmaningar. Det är viktigt att vara medveten om dem:

- **Inlärningskurva:** OOP kan vara lite knepigt att lära sig i början. Det finns en del koncept och termer att förstå, men det är värt det! När du väl har lärt dig grunderna kommer du att kunna skapa fantastiska saker!—
- **Prestandaöverväganden:** Ibland kan OOP leda till viss prestandaförlust eftersom det kan finnas några "bakom kulisserna" -operationer som påverkar kodens hastighet. Detta är dock vanligtvis inget att oroa sig för i de flesta fall.
- **Designkomplexitet:** Om OOP-koncept inte används på rätt sätt kan det leda till överdriven designkomplexitet och göra koden svårare att förstå och underhålla. Det är viktigt att använda OOP på ett sätt som gör koden enklare och inte mer komplicerad.

Tillämpningsområden

OOP är inte bara ett fancy koncept som bara fungerar i laboratoriemiljö. Det har faktiskt många praktiska tillämpningar! Här är några exempel:

- **Applikationsutveckling:** Om vi bygger applikationer med komplexa datastrukturer och beteenden kan OOP vara till stor hjälp. Det hjälper oss att organisera koden och göra den lättare att hantera.
- **Spelutveckling:** Har du någonsin velat bygga ditt eget spel? OOP är vägen att gå! Det hjälper oss att modellera spelobjekt, hantera spellogik och skapa interaktiva spelupplevelser.
- **Webbutveckling:** Inom webbutveckling kan OOP hjälpa oss att skapa återanvändbara och skalbara komponenter. Det hjälper oss också att implementera designmönster som gör vår kod renare och mer effektiv.
- **Simuleringar:** OOP används ofta inom simuleringar för att modellera och interagera med simulerade enheter och beteenden. Det kan vara allt från vetenskapliga simuleringar till spelutveckling.
- **Databashantering:** Till och med databashantering kan dra nytta av OOP. Vi kan abstrahera databasåtkomst och använda objekt för att enkelt kommunicera med vår databas.

Exempel

Nu ska vi titta på ett konkret exempel för att se OOP i aktion! Vi ska skapa en klass som representerar en bil i Java:

```

public class Bil {
    private String märke;
    private String modell;
    private int årsmodell;
    public Bil(String märke, String modell, int årsmodell) {
        this.märke = märke;
        this.modell = modell;
        this.årsmodell = årsmodell;
    }
    public String getMärke() {
        return märke;
    }
    public void setMärke(String märke) {
    }
    public String getModel() {
        return modell;
    }
    public void setModell(String modell) {
    }
    public int getÅrsmodell() {
        return årsmodell;
    }
    public void setÅrsmodell(int årsmodell) {
    }
    public void köra() {
        System.out.println("Bilen kör.");
    }
    public void stanna() {
        System.out.println("Bilen stannar.");
    }
}

```

I det här exemplet har vi en klass som heter "Bil". Den har tre privata instansvariabler: "märke", "modell" och "årsmodell". Vi har också en konstruktor som tar emot dessa tre variabler och tilldelar dem till de motsvarande instansvariablerna.

Vi har också getter- och setter-metoder för varje instansvariabel för att kunna få och sätta värden på dem. Dessutom har vi två metoder, "köra" och "stanna", som bara skriver ut meddelanden till konsolen.

Nu kan vi skapa objekt av klassen "Bil" och använda dess metoder:

```

public class Main {
    public static void main(String[] args) {
        Bil bil = new Bil("Volvo", "V70", 2022);
        System.out.println("Märke: " + bil.getMärke());
        System.out.println("Modell: " + bil.getModel());
        System.out.println("Årsmodell: " + bil.getÅrsmodell());
        bil.köra();
        bil.stanna();
    }
}

```

I detta exempel skapar vi en instans av klassen "Bil" med hjälp av konstruktorn och tilldelar den till variabeln "bil". Sedan använder vi getter-metoderna för att få värdena på instansvariablerna och skriver ut dem till konsolen. Slutligen anropar vi metoder för att köra och stanna bilen.

```

public class Car {
    private String brand;
    private String color;
    private int speed;
    public Car(String brand, String color) {
        this.brand = brand;
        this.color = color;
        this.speed = 0;
    }
    public void accelerate(int value) {
        speed += value;
    }
    public void brake(int value) {
        speed -= value;
    }
    public int getSpeed() {
        return speed;
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car("Volvo", "blå");
        myCar.accelerate(20);
        System.out.println(myCar.getSpeed()); // Output: 20
        myCar.brake(10);
        System.out.println(myCar.getSpeed()); // Output: 10
    }
}

```

I detta exempel skapar vi en bilklass (Car) med egenskaper och metoder för att accelerera, bromsa och hämta hastigheten. I main-metoden skapar vi en instans av Car och testar dess metoder.

Slutsats

All right! Du har precis fått en introduktion till Objektorienterad programmering (OOP). Det är en viktig del av programmeringsvärlden och något som du definitivt bör utforska mer! Med OOP kan du organisera din kod på ett sätt som gör den lättare att förstå, underhålla och återanvända. Tänk på fördelarna och användningsområdena vi har diskuterat och låt det inspirera dig att ta dig an OOP och skapa fantastiska saker! Lycka till!

TL;DR

OOP är en programmeringsmetodik som hjälper oss att organisera vår kod genom att skapa objekt med egenskaper och beteenden. Det ger oss modularitet, återanvändbarhet och lättare underhåll. OOP kan användas inom applikationsutveckling, spelutveckling, webbutveckling, simuleringar och databashantering. Ta en titt på vårt exempel med en bilklass och kom igång med OOP! Nu är det dags att dyka djupare in i världen av OOP och skapa magi med din kod!

Java för Jin23

Marcus Medina

Klasser och Objekt inom programmering med Java

TL;DR

När du läst detta ska du kunna

Introduktion

Vad är Klasser och Objekt?

Klassers namn

Fördelar

Begränsningar

Användningsområden

Exempelkod

Output

Slutsats

Obligatorisk dad joke

Klasser och Objekt inom programmering med Java

En artikel som utforskar ämnet Klasser och Objekt inom programmering med Java.

▼ Innehållsförteckning

TL;DR

Klasser och Objekt är centrala koncept inom objektorienterad programmering med Java. Klasser fungerar som mallar för att skapa objekt med definierade egenskaper och beteenden. Genom att använda Klasser och Objekt kan vi skapa modulära och återanvändbara program, hantera komplexitet och modellera relationer mellan olika entiteter. Det finns fördelar som modularitet och abstraktion, men också begränsningar som inlärningskurva och designkomplexitet. Klasser och Objekt kan tillämpas inom olika områden som applikationsutveckling, spelutveckling och simuleringar.

När du läst detta ska du kunna

- Förstå och förklara vad Klasser och Objekt är och deras relevans inom programmering med Java.
- Diskutera fördelar och begränsningar med Klasser och Objekt.
- Identifiera olika användningsområden där Klasser och Objekt kan tillämpas.
- Förstå och tolka ett kodexempel som använder Klasser och Objekt i Java.
- Sammanfatta viktiga insikter och rekommendationer för vidare läsning.

Introduktion

Välkommen till denna artikel som kommer att utforska konceptet Klasser och Objekt inom programmering med Java. Klasser och Objekt är grundläggande byggstenar inom objektorienterad programmering (OOP) och spelar en central roll för att skapa modulära och strukturerade program. Genom att förstå och behärska dessa koncept kan du skapa kraftfulla och flexibla program som är lättare att underhålla och återanvända.

Vad är Klasser och Objekt?

Klasser och Objekt är centrala koncept inom objektorienterad programmering med Java som hjälper oss att organisera och strukturera vår kod på ett mer logiskt sätt.

En **klass** kan betraktas som en mall eller ritning som definierar hur ett objekt av den klassen ska se ut och agera. Klassen innehåller definitioner av attribut (också kallade egenskaper) och metoder som beskriver objektets egenskaper och beteenden.

Ett **objekt** är en specifik förekomst av en klass. Det kan ses som en konkret entitet med egenskaper som är definierade av klassen och som kan utföra de metoder som klassen tillhandahåller.

En analogi som ofta används för att förklara klasser och objekt är att klassen är som en ritning för att skapa ett hus och objektet är själva huset som skapas enligt ritningen. Ritningen definierar husets egenskaper (antalet rum, färg, storlek, etc.) och metoder (öppna dörren, tända lampan, etc.), medan själva huset är den specifika instansen som skapas baserat på ritningen.

Klassers namn

Klasser kallas för olika namn beroende på hur de används. Här är den uppdaterade tabellen med basklass, subklass och några namn för olika designmönster:

Term	Förklaring	Andra namn	
Klass	En klass är som en ritning för att bygga något. Det berättar vilka delar och funktioner som något ska ha när det skapas.		
Abstrakt klass	En abstrakt klass är som en mall där man bara får en idé om hur något ska se ut och fungera, men inte exakt hur det ska göras.		
Gränssnitt	Ett gränssnitt är som en överenskommelse där man bestämmer vilka saker man behöver kunna göra för att passa in i en viss grupp.	Interface	
Konkret klass	En konkret klass är som en färdig produkt som kan användas direkt, precis som en leksak som man kan leka med direkt när man köper den.		
Basklass	En basklass är som en övergripande klass som innehåller gemensam funktionalitet och egenskaper som är ärvt av flera subklasser.	Superklass	
Subklass	En subklass är som en specialiserad klass som ärver funktionalitet och egenskaper från en basklass och kan lägga till eller ändra dem.	Underklass	
Instans	En instans är som en specifik sak som man skapar baserat på ritningen eller mallen i en klass.	Objekt	
Modell	En modell är som en beskrivning av hur något ser ut och fungerar, t.ex. hur man ska spara information i en databas.		
Typ	En typ är som en kategori som används för att organisera och hantera olika sorters saker, som olika typer av mätningar eller enheter.		
Objekt	Ett objekt är som en grej som man kan röra vid och använda. Det är det konkreta resultatet av att använda en ritning eller mall.		
Entitet	En entitet är som en specifik grej som man vill hålla reda på och spara information om, som en person eller en produkt i en databas.		
Helper	En helper är som en bästa vän som alltid är där för att hjälpa till med små uppgifter och göra saker lite enklare för dig.	Hjälpklass, Verktøygsklass	

Term	Förklaring	Andra namn	
Utility	En utility är som en verktygslåda full med användbara saker som hjälper dig att lösa specifika problem eller göra svåra saker enklare.		
Komponent	En komponent är som en del av något större, som en pusselbit som passar in i en större bild och har en specifik funktion.		
Enhetsobjekt	Ett enhetsobjekt är som en representation av en specifik enhet eller apparat som kan utföra vissa uppgifter eller ha viss funkt	ionalitet.	
Verktyg	Ett verktyg är som en hjälpande hand som underlättar och effektiviserar olika uppgifter eller processer.		
Modul	En modul är som en självständig del av ett större system, som kan kopplas in och användas för att utföra specifika uppgifter.		
Singleton	Singleton är ett designmönster som används för att se till att endast en instans av en klass skapas och att den kan nå globalt.		
Fabrik	Fabrik är ett designmönster som används för att skapa objekt utan att avslöja den konkreta implementationen och istället använda en gemensam gränssnitt.	Factory	
Byggare	Byggare är ett designmönster som används för att skapa objekt stegvis och möjliggör olika sätt att bygga upp komplexa objekt.	Builder	
Strategi	Strategi är ett designmönster som används för att välja och använda en av flera möjliga algoritmer eller beteenden vid körningstid.	Strategy	
Observatör	Observatör är ett designmönster som används för att etablera en publika/prenumerera-mekanism där flera objekt kan lyssna på händelser från ett annat objekt.	Observer	

Som du ser har klasser många namn, man kan säga att det är en klass för varje tillfälle. Men inget att oroa sig för, du kommer att lära dig mer om dem allt eftersom du fortsätter att lära dig om programmering. Ju mer man använder klasser, desto bättre lär man sig namnen.

Fördelar

Användningen av Klasser och Objekt erbjuder flera fördelar inom programmering med Java:

1. **Modularitet och återanvändbarhet:** Klasser möjliggör modulär kod genom att separera olika delar av programmet i olika klasser. Detta gör det enklare att hantera och underhålla koden samt möjliggör återanvändning av kod genom att skapa nya objekt baserat på en befintlig klass.
2. **Abstraktion och hantering av komplexitet:** Genom att använda klasser kan vi abstrahera bort detaljer och fokusera på de väsentliga egenskaperna och beteendena hos objektet. Detta hjälper till att hantera komplexitet och gör koden mer läsbar och underhållbar.
3. **Kapsling och informationsskydd:** Klasser möjliggör att vi kan begränsa åtkomsten till objektets egenskaper och metoder. Detta främjar informationsskydd och hjälper till att undvika oavsiktliga ändringar av objektets tillstånd.

4. **Hantering av relationer mellan objekt:** Genom att använda klasser och objekt kan vi definiera och hantera relationer mellan olika objekt. Detta möjliggör att vi kan skapa mer realistiska och flexibla modeller av den verkliga världen i våra program.

Begränsningar

Det finns också vissa begränsningar eller utmaningar med att använda Klasser och Objekt i Java:

1. **Inlärningskurva:** Konceptet med Klasser och Objekt kan vara svårt att förstå i början, särskilt för nybörjare inom programmering. Att förstå sambandet mellan klasser och objekt och att behärska olika OOP-koncept kan ta tid och övning.
2. **Prestandaöverhead:** Objektorienterad kod kan vara lite mer resurskrävande än procedurorienterad kod på grund av det extra lagringsutrymmet som krävs för att hålla reda på objektens tillstånd och beteenden. Detta kan vara en faktor att ta hänsyn till när det krävs maximal prestanda i en applikation.
3. **Designkomplexitet:** Att designa och planera klasser och objekt kan vara en utmaning. Att hitta rätt abstraktionsnivå, definiera korrekta relationer och undvika överkomplicerade hierarkier kan vara svårt och kräver erfarenhet och bra designprinciper.

Användningsområden

Klasser och Objekt kan tillämpas i en mängd olika scenarier inom programmering med Java. Här är några exempel på användningsområden:

1. **Applikationsutveckling:** Klasser och Objekt används i stor utsträckning vid utveckling av applikationer, oavsett om det är webbapplikationer, mobilappar eller skrivbordsprogram. Genom att strukturera koden med hjälp av klasser och objekt blir programmet mer organiserat och lättare att underhålla.
2. **Spelutveckling:** Inom spelutveckling används Klasser och Objekt för att skapa olika spelobjekt, karaktärer, världar och mycket mer. Genom att använda objektorienterad programmering kan spelutvecklare skapa komplexa och interaktiva spelvärldar.
3. **Simuleringar:** Simuleringsprogram och modelleringsverktyg kan dra nytta av Klasser och Objekt för att representera och simulera olika entiteter och processer. Genom att använda objekt för att modellera olika aspekter av systemet kan simuleringar bli mer realistiska och flexibla.
4. **Databashantering:** Vid databashantering används ofta objektorienterade koncept för att modellera och hantera data. Objekt kan representera tabeller, rader och kolumner i en databas och möjliggöra en mer flexibel och hanterbar databasstruktur.

Exempelkod

För att illustrera användningen av Klasser och Objekt, låt oss tänka oss att vi bygger ett program för att hantera en biblioteksdatabase. Vi kan använda Klasser och Objekt för att representera olika entiteter i biblioteket, till exempel böcker och medlemmar. Här är ett kodexempel i Java:

```

public class Book {
    private String title;
    private String author;
    private int year;

    public Book(String title, String author, int year) {
        this.title = title;
        this.author = author;
        this.year = year;
    }

    public void displayInfo() {
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
        System.out.println("Year: " + year);
    }
}

```

Nu har vi skapat en klass för böcker som innehåller information om titel, författare och år. Vi kan sedan skapa ett objekt av klassen och använda dess metoder för att visa informationen om boken:

```

public class Member {
    private String name;
    private int age;
    private List<Book> borrowedBooks;

    public Member(String name, int age) {
        this.name = name;
        this.age = age;
        this.borrowedBooks = new ArrayList<>();
    }

    public void borrowBook(Book book) {
        borrowedBooks.add(book);
    }

    public void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Borrowed Books:");
        for (Book book : borrowedBooks) {
            System.out.println("- " + book.getTitle());
        }
    }
}

```

Här har vi skapat en klass för medlemmar som innehåller information om namn, ålder och en lista över lånade böcker. Vi kan sedan skapa ett objekt av klassen och använda dess metoder för att visa informationen om medlemmen:

Nu kan vi använda klasserna för att skapa ett bibliotekssystem:

```

// Exempel på användning av Klasser och Objekt
Book book = new Book("The Catcher in the Rye", "J.D. Salinger", 1951);
Member member = new Member("John Doe", 30);
member.borrowBook(book);

book.displayInfo();
member.displayInfo();

```

Vi kan skapa ett objekt av varje klass och använda deras metoder för att visa informationen om boken och medlemmen.

I exemplet skapar vi en bok, sedan skapar vi en person som heter John Doe och är 30 år gammal. Vi lånar sedan boken till John Doe och visar informationen om boken och medlemmen.

När vi har båda klasserna instansierade kan vi nu tala om för boken att visa sin information och för medlemmen att visa sin information. Vi kan också lägga till en bok till medlemmens lånade böcker och sedan visa informationen om medlemmen igen för att se att boken har lagts till.

Output

Title: The Catcher in the Rye
Author: J.D. Salinger
Year: 1951
Name: John Doe
Age: 30
Borrowed Books:
- The Catcher in the Rye

Så kan vi använda klasserna, Book och Member, som representerar en bok och en medlem i biblioteket. Vi kan skapa objekt av varje klass och använda deras metoder för att visa informationen om boken och medlemmen.

Slutsats

Klasser och Objekt är grundläggande koncept inom objektorienterad programmering med Java som hjälper oss att organisera och strukturera vår kod på ett mer logiskt och effektivt sätt. Genom att använda Klasser och Objekt kan vi skapa modulära och återanvändbara program, hantera komplexitet och modellera relationer mellan olika entiteter.

Det är viktigt att förstå både fördelarna och begränsningarna med att använda Klasser och Objekt och att kunna identifiera olika användningsområden där dessa koncept kan tillämpas. Genom att behärska Klasser och Objekt kan du utveckla välstrukturerad och lättunderhållen kod.

För vidare läsning och fördjupning rekommenderas att studera OOP-principer, designmönster och mer avancerade koncept som arv och polymorfism.

Obligatorisk dad joke

Varför ville objektet inte gå på festen?
För att det inte hade någon klass!

Java för Jin23

Marcus Medina

Klasskomposition

Fördelar

Begränsningar

Exempel på användning

Kodexempel

Output

Slutsats

Klasskomposition

Klasskomposition är en teknik inom objektorienterad programmering (OOP) där mindre klasser kombineras för att bygga större och mer komplexa klasser. Genom att använda klasskomposition kan vi skapa mer modulära, flexibla och underhållbara lösningar.

Fördelar

Att använda klasskomposition i din kod kan erbjuda flera fördelar:

- **Återanvändbarhet:** Genom att kombinera mindre och specialiserade klasser kan vi skapa återanvändbar kod. De mindre klasserna kan användas i olika sammanhang och kan även användas i andra större klasser om det behövs.
- **Flexibilitet:** Genom att använda klasskomposition blir koden mer flexibel. Vi kan lägga till eller ta bort komponenter genom att lägga till eller ta bort de mindre klasserna som ingår i den större klassen.
- **Tydlighet och underhållbarhet:** Genom att dela upp funktionaliteten i mindre och självständiga klasser blir koden lättare att förstå och underhålla. Varje klass har sitt eget ansvarsområde och kan ändras separat utan att påverka resten av systemet.

Begränsningar

Det finns också vissa begränsningar att tänka på när man använder klasskomposition:

- **Komplexitet:** Klasskomposition kan öka komplexiteten i koden om den inte används på ett genomtänkt sätt. Det är viktigt att noggrant planera och designa klassrelationerna för att undvika onödig komplexitet.
- **Beroenden:** Om en klass är beroende av en annan klass genom klasskomposition, kan ändringar i den andra klassen påverka funktionaliteten hos den första klassen. Det är viktigt att vara medveten om beroendena mellan klasserna och att hantera dem på ett lämpligt sätt.
- **Prestanda:** Klasskomposition kan påverka prestandan hos programmet eftersom det kan innebära fler objekt och därmed mer minnesanvändning och bearbetningstid. Det är viktigt att överväga prestandaaspekterna när man använder klasskomposition och att optimera koden vid behov.

Exempel på användning

Klasskomposition kan tillämpas i en mängd olika scenarier. Här är några exempel på användningsområden där klasskomposition är användbart:

- **Bil och motor:** En bilklass kan innehålla en instans av en motorklass för att hantera bilens motorfunktionalitet.
- **Dator och processor:** En datorklass kan innehålla en instans av en processor för att hantera beräkningsfunktionalitet.
- **Bank och konton:** En bankklass kan innehålla flera instanser av en kontoklass för att hantera olika bankkonton.
- **Spel och karaktärer:** Ett spel kan ha en spelklass som innehåller flera instanser av en karaktärsklass för att hantera spelkaraktärer.

Kodexempel

Låt oss nu titta på ett kodexempel för att förstå hur klasskomposition kan tillämpas i praktiken. I det här exemplet har vi två klasser: Person och Address. Klassen Person representerar en person och innehåller en instans av klassen Address för att hantera adressrelaterad information.

```
public class Address {
    private String street;
    private String city;
    private String state;
    private String zipCode;

    public Address(String street, String city, String state, String zipCode)
    {
        this.street = street;
        this.city = city;
        this.state = state;
        this.zipCode = zipCode;
    }

    // Getters and setters

    public String getStreet() {return street;}
    public String getCity() {return city;}
    public String getState() {return state;}
    public String getZipCode() {return zipCode;}
    public void setStreet(String street) {this.street = street;}
    public void setCity(String city) {this.city = city;}
    public void setState(String state) {this.state = state;}
    public void setZipCode(String zipCode) {this.zipCode = zipCode;}

    @Override
    public String toString() {
        return "Address{" +
            "street='" + street + '\'' +
            ", city='" + city + '\'' +
            ", state='" + state + '\'' +
            ", zipCode='" + zipCode + '\'' +
            '}';
    }
}
```

Nu har vi en klass som representerar en adress. Den innehåller attribut för gatuadress, stad, län och postnummer. Klassen har också en konstruktor som tar emot dessa attribut och en metod toString() som returnerar en strängrepresentation av adressen.

```

public class Person {
    private String name;
    private int age;
    private Address address;

    public Person(String name, int age, Address address) {
        this.name = name;
        this.age = age;
        this.address = address;
    }

    public String getAddressDetails() {
        return "Address: " + address.getStreet() + ", " + address.getCity() + ",
            " + address.getState() + ", " + address.getZipCode();
    }
}

```

Här har vi en klass som representerar en person. Den innehåller attribut för namn, ålder och en instans av klassen Address för att hantera adressuppgifter. Klassen har också en metod getAddressDetails() som returnerar en strängrepresentation av personens adress.

```

// Användning av klasskomposition
Address address = new Address("123 Main St", "Cityville", "Stateville",
    "12345");
Person person = new Person("John Doe", 25, address);
System.out.println(person.getAddressDetails());

```

I det här exemplet har vi en Address-klass som innehåller attribut för gatuadress, stad, delstat och postnummer. Klassen Person innehåller attribut för namn, ålder och en instans av Address-klassen för att hantera personens adressuppgifter. Klassen Person har också en metod getAddressDetails() som returnerar en strängrepresentation av personens adress.

För att använda dessa klasser kan vi skapa en instans av Person och tilldela en instans av Address till dess address-attribut. Vi kan sedan använda getAddressDetails()-metoden för att få personens adressuppgifter.

```

Address address = new Address("123 Main St", "Cityville", "Stateville",
    "12345");
Person person = new Person("John Doe", 25, address);
System.out.println(person.getAddressDetails());

```

Output

```
Address: 123 Main St, Cityville, Stateville, 12345
```

I det här kodexemplet skapar vi en instans av Address med adressuppgifterna "123 Main St", "Cityville", "Stateville" och "12345". Vi skapar sedan en instans av Person

med namnet "John Doe", ålder 25 och den tidigare skapade adressinstansen. Slutligen skriver vi ut personens adressuppgifter med hjälp av getAddressDetails()-metoden.

Slutsats

Klasskomposition är en kraftfull teknik inom objektorienterad programmering som låter oss skapa mer flexibla, återanvändbara och underhållbara lösningar. Genom att kombinera mindre och specialiserade klasser kan vi bygga större och mer komplexa system utan att offra tydlighet och prestanda. Det är viktigt att förstå fördelarna och begränsningarna med klasskomposition för att kunna använda det på bästa sätt.

För vidare läsning och fördjupning i ämnet rekommenderas att utforska designprinciper och mönster inom objektorienterad programmering, såsom "SOLID-principerna" och "komposition över arv". Genom att utveckla din kunskap om dessa koncept kan du bli en skickligare programmerare och bygga mer effektiva och flexibla program.

Java för Jin23

Marcus Medina

Inkapsling

- Vad är inkapsling?
- Begränsningar med inkapsling
- Användningsområden för inkapsling i Java
- Exempel på inkapsling i Java
- Ett annat exempel
- Sammanfattning
- Obligatorisk dad-joke

Absolut! Här är hela artikeln om inkapsling i Java:

Inkapsling

Inkapsling är en viktig princip inom objektorienterad programmering som handlar om att kombinera data och metoder inom en klass och kontrollera åtkomsten till dem. Det främjar säkerhet, modularitet, återanvändbarhet och kodunderhåll. Inkapsling kan tillämpas i dataklasser, API-design och användas tillsammans med arv och polymorfism.

▼ Innehållsförteckning

1. Innehållsförteckning

Vad är inkapsling?

Inkapsling innebär att kombinera data och metoder inom en klass och kontrollera åtkomsten till dem. Genom att använda inkapsling kan vi definiera vilka medlemmar som är tillgängliga utanför klassen och hur de kan manipuleras. Detta möjliggör en tydlig separation mellan implementationen av en klass och dess användning i andra delar av programmet.

Inkapsling har flera fördelar inom programmering:

1. **Modularitet:** Inkapsling möjliggör att klasser kan vara självständiga enheter med en tydlig gränssnitt för att interagera med andra delar av programmet. Detta underlättar utveckling, underhåll och återanvändbarhet av kod, eftersom det gör det lättare att isolera och ändra specifika delar av koden utan att påverka resten av programmet.
2. **Säkerhet:** Genom att använda inkapsling kan vi begränsa åtkomsten till viss data och funktionalitet. Detta gör det möjligt att skydda känslig information och förhindra oavsiktlig manipulation av data. Genom att använda privata medlemmar kan vi säkerställa att vissa operationer bara kan utföras inom klassen eller av utvalda metoder.
3. **Kodunderhåll:** Inkapsling främjar en bättre struktur och organisation av koden. Genom att gruppera relaterade data och funktioner inom en klass blir det lättare att förstå och ändra koden. Detta underlättar underhåll och felsökning, eftersom det minimerar risken för biverkningar och felaktiga ändringar i andra delar av programmet.

Begränsningar med inkapsling

Även om inkapsling har många fördelar finns det också vissa begränsningar att vara medveten om:

1. **Komplexitet:** Inkapsling kan introducera en viss komplexitet i koden, särskilt när det gäller att hantera olika åtkomstnivåer och beroenden mellan klasser. Det är viktigt att noga planera och organisera klasser för att undvika överdriven komplexitet och onödiga beroenden.
2. **Prestanda:** Vissa åtkomstmodifierare, som `private`, kan medföra en viss prestandaförlust när det gäller att komma åt data och utföra operationer. Detta beror på att åtkomst till privata medlemmar kräver ytterligare overhead i form av metoanrop eller reflektion. Det är dock viktigt att notera att prestandaförlusten oftast är försumbar och att inkapslingens fördelar oftast överväger den.

Användningsområden för inkapsling i Java

Inkapsling kan tillämpas på olika sätt inom Java-programmering. Här är några vanliga användningsområden:

1. **Klasser och objekt:** Inkapsling används i stor utsträckning för att definiera klasser och objekt i Java. Genom att använda åtkomstmodifierare, som `public`, `private` och `protected`, kan vi definiera vilka medlemmar som är tillgängliga utanför klassen och hur de kan manipuleras.
2. **Egenskaper (Getters och Setters):** Genom att använda getters och setters kan vi kontrollera åtkomsten till objektens data och möjliggöra läsning och skrivning av privata medlemmar på ett kontrollerat sätt. Genom att använda getters och setters kan vi implementera validering och logik för att säkerställa korrekt användning av data.
3. **Gränssnitt (Interfaces):** Inkapsling används för att definiera gränssnitt i Java. Ett gränssnitt definierar en uppsättning metoder som en klass kan implementera. Genom att använda gränssnitt kan vi separera definitionen av en klass från dess implementation och möjliggöra en mer flexibel och modular kodstruktur.
4. **Nedarvning (Inheritance):** Inkapsling används också i samband med nedarvning för att definiera och kontrollera åtkomsten till medlemmar i en basclass. Genom att använda åtkomstmodifierare kan vi definiera vilka medlemmar som ärver och vilka som inte ärver till en underklass.

Exempel på inkapsling i Java

För att ge en bättre förståelse för inkapsling i Java kan vi titta på ett exempel på en klass som använder inkapsling:

```

public class BankAccount {
    private double balance;

    public BankAccount(double initialBalance) {
        balance = initialBalance;
    }

    public void deposit(double amount) {
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
        } else {
            System.out.println("Otillräckliga medel.");
        }
    }

    public double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {

        BankAccount account = new BankAccount(1000);
        System.out.println("Nuvarande saldo: " + account.getBalance());
        account.deposit(500);
        System.out.println("Efter insättning: " + account.getBalance());
        account.withdraw(200);
        System.out.println("Efter uttag: " + account.getBalance());
    }
}

```

I detta exempel har vi en klass BankAccount som representerar en bankräkning. Klassen har en privat medlemsvariabel balance som håller reda på kontots saldo. Klassen har också metoder för att sätta in pengar (deposit), ta ut pengar (withdraw) och hämta saldot (getBalance).

I Main-klassen skapar vi en instans av BankAccount och använder dess metoder för att utföra insättningar och uttag på bankkontot.

Ett annat exempel

Vi kan också titta på ett annat exempel på inkapsling i Java:

```

public class Superhero {
    private String name; // Privat medlemsvariabel för superhjältens namn
    private String powers; // Privat medlemsvariabel för superhjältens krafter

    // Konstruktör för att skapa en instans av Superhero med namn och krafter
    public Superhero(String name, String powers) {
        this.name = name;
        this.powers = powers;
    }

    // Getter-metod för att hämta superhjältens namn
    public String getName() {
        return name;
    }

    // Getter-metod för att hämta superhjältens krafter
    public String getPowers() {
        return powers;
    }

    // Setter-metod för att sätta superhjältens krafter
    public void setPowers(String powers) {
        this.powers = powers;
    }

    // Metod för att visa superhjältens namn och krafter
    public void displaySuperhero() {
        System.out.println("Namn: " + name);
        System.out.println("Krafter: " + powers);
    }
}

public class Main {
    public static void main(String[] args) {
        // Skapa en instans av Superhero med namnet "Spider-Man" och krafterna
        // "Wall-crawling, superhuman strength"
        Superhero superhero = new Superhero("Spider-Man", "Wall-crawling,
        superhuman strength");

        // Visa superhjältens namn och krafter med hjälp av displaySuperhero-
        // metoden
        superhero.displaySuperhero();

        // Uppdatera superhjältens krafter med hjälp av setPowers-metoden
        superhero.setPowers("Web-slinging, spider-sense");

        // Visa superhjältens uppdaterade krafter med hjälp av getPowers-metoden
        System.out.println("Uppdaterade krafter: " + superhero.getPowers());
    }
}

```

I detta exempel har vi en klass Superhero som representerar en superhjälte. Klassen har två privata medlemsvariabler, name och powers, för att hålla superhjältens namn och krafter. Här är förklaringar för varje del av koden:

- I konstruktorn Superhero tar vi emot namn och krafter som parametrar och tilldelar dem till de privata medlemsvariablerna med hjälp av this-referensen.
- getName är en getter-metod som returnerar superhjältens namn.
- getPowers är en getter-metod som returnerar superhjältens krafter.
- setPowers är en setter-metod som tar emot en ny kraft som parameter och uppdaterar den privata medlemsvariabeln powers med den nya kraften.
- displaySuperhero är en metod som visar superhjältens namn och krafter genom att skriva ut dem till konsolen.
- I Main-klassens main-metod skapar vi en instans av Superhero med namnet "Spider-Man" och krafterna "Wall-crawling, superhuman strength".
- Vi anropar displaySuperhero-metoden på den skapade instansen för att visa superhjältens namn och krafter.
- Sedan använder vi setPowers-metoden för att uppdatera superhjältens krafter till "Web-slinging, spider-sense".
- Till sist använder vi getPowers-metoden för att hämta och skriva ut de uppdaterade krafterna till konsolen.

På så sätt kan du skapa och manipulera en instans av Superhero-klassen med hjälp av inkapsling i Java.

Jag hoppas att detta förtydligar exemplet för dig! Låt mig veta om det finns något mer jag kan hjälpa dig med.

Sammanfattning

Inkapsling är en viktig princip inom objektorienterad programmering som möjliggör att data och funktioner som hör samman hålls tillsammans inom en enhet, kallad en klass. Genom att använda inkapsling kan vi definiera vilka medlemmar som är tillgängliga utanför klassen och hur de kan manipuleras. Detta främjar modularitet, återanvändbarhet och säkerhet i programkoden.

Inkapsling har flera fördelar, såsom modularitet, säkerhet och kodunderhåll. Det finns dock också vissa begränsningar, såsom ökad komplexitet och eventuell prestandaförlust. Inkapsling kan tillämpas på olika sätt inom Java-programmering, inklusive klasser, getters och setters, gränssnitt och nedarvning.

Genom att förstå inkapslingens koncept och användningsområden kan du skapa välstrukturerad och lättunderhållen kod i Java. Lycka till med dina programmeringsprojekt!

Obligatorisk dad-joke

Varför älskar programmerare att använda inkapsling?

För att de inte vill läcka sina privata medlemmar!

Java för Jin23

Marcus Medina

// Arv är en princip inom OOP där en klass kan ärva egenskaper och beteenden från en annan överordnad klass, vilket gör det möjligt att återanvända kod och skapa hierarkier av klasser.

// Superklassen är den överordnade klassen som fungerar som en mall eller ritning för de nedärvande klasserna. Subklasserna ärver och kan utöka funktionaliteten från superklassen. // Exempel: // Superklassen "Fordon" kan ha egenskaper och beteenden som är gemensamma för alla fordon, t.ex. hastighet och färg. // Subklasserna "Bil" och "Motorcykel" kan ärva dessa egenskaper och beteenden från "Fordon" och lägga till sina egna unika egenskaper och beteenden, t.ex. antal dörrar och typ av motor. // På så sätt kan vi återanvända kod och undvika att duplicera samma kod i flera klasser. Vi kan också strukturera och organisera vår kod på ett modulärt sätt, vilket förbättrar underhållbarheten och läsbarheten. // Arv möjliggör också polymorfism, vilket innebär att en instans av en subklass kan användas där en instans av superklassen förväntas. Detta gör det möjligt att skriva generell kod som kan hantera olika typer av objekt. // En metod som tar emot en parameter av typen "Fordon" kan användas för att hantera både "Bil" och "Motorcykel" objekt, eftersom de båda är subklasser till "Fordon". // Arv kan vara en kraftfull teknik för att organisera och strukturera kod, men det kan också leda till problem om det inte används på rätt sätt. Det är viktigt att tänka på följande begränsningar och rekommendationer när du använder arv: // - Arv bör endast användas när det finns en verklig "är ett" relation mellan superklassen och subklassen. Om relationen bara är en "har ett" relation, kan det vara mer lämpligt att använda komposition istället för arv. // - Arv kan leda till en hierarki av klasser som kan bli svår att underhålla och förstå. Det är viktigt att noggrant planera och designa hierarkin för att undvika överdriven komplexitet. // - Om en subklass behöver ändra eller utöka funktionaliteten från superklassen, kan det vara mer lämpligt att använda interfaces eller abstrakta klasser istället för arv. Detta möjliggör flexibilitet och undviker problem med klasshierarkin. // - Det är viktigt att använda rätt namngivningskonventioner och vara konsekvent i hela klasshierarkin för att underlätta förståelsen och läsbarheten. // - Det är också viktigt att vara medveten om eventuell kodupprepning i klasshierarkin och undvika det genom att extrahera gemensam funktionalitet till separata metoder eller klasser. // - Slutligen är det alltid bra att dokumentera klasshierarkin och dess relationer för att underlätta förståelsen och underhållningen av koden. // Med dessa rekommendationer kan arv vara en kraftfull teknik för att organisera och strukturera kod inom OOP. # Arv

Arv kan tillämpas i olika situationer och användningsområden inom programmering. Här är några exempel: 1. Skapa en hierarki av klasser där varje klass representerar en mer specialiserad version av den överordnade klassen. Till exempel kan vi ha en superklass "Djur" och subklasser som "Hund", "Katt" och "Fågel" som ärver och utökar funktionaliteten från "Djur". 2. Återanvända kod genom att extrahera gemensam funktionalitet till en överordnad klass och låta subklasserna ärva den. På så sätt undviker vi duplicering av kod och håller vår kod baserad på principen "Don't Repeat Yourself" (DRY). 3. Implementera polymorfism genom att använda superklasser och subklasser. Polymorfism möjliggör att en instans av en subklass kan behandlas som en instans av superklassen, vilket gör att vi kan skriva generell kod som fungerar med olika typer av objekt. 4. Skapa en modellering av verkliga scenarier där hierarkier och relationer mellan olika objekt är viktiga. Till exempel kan vi modellera en hierarki av fordon där vi har en superklass "Fordon" och subklasser som "Bil", "Motorcykel" och "Buss". Det finns många andra användningsområden för arv beroende på de specifika kraven och designen av programmet. Det är viktigt att noggrant planera och designa hierarkin för att få ut det bästa av arv och undvika problem eller överdriven komplexitet. ## Kodexempel: Arv i Java

Låt oss nu titta på ett kodexempel som visar hur arv kan användas i Java. Vi kommer att använda exemplet med hierarkin av fordon som nämndes tidigare.

```
// Superklassen "Fordon"
class Fordon {
    private int hastighet;
```

```

private int hastighet;
private String färg;
public Fordon(int hastighet, String färg) {
    this.hastighet = hastighet;
    this.färg = färg;
}
public int getHastighet() {
    return hastighet;
}
public String getFärg() {
    return färg;
}
}
// Subklassen "Bil" ärver från "Fordon"
class Bil extends Fordon {
    private int antalDörrar;
    public Bil(int hastighet, String färg, int antalDörrar) {
        super(hastighet, färg);
        this.antalDörrar = antalDörrar;
    }
    public int getAntalDörrar() {
        return antalDörrar;
    }
}
// Subklassen "Motorcykel" ärver från "Fordon"
class Motorcykel extends Fordon {
    private String typAvMotor;
    public Motorcykel(int hastighet, String färg, String typAvMotor) {
        this.typAvMotor = typAvMotor;
    }
    public String getTypAvMotor() {
        return typAvMotor;
    }
}
// Genom arv kan en hierarki av klasser skapas där gemensamma egenskaper
och beteenden
// placeras i överordnade klasser och specialiserade egenskaper och
beteenden läggs till
// i de nedärvande klasserna. Detta möjliggör återanvändning av kod och
effektivisering av
// utvecklingsprocessen.
// Fördelar
// Arv erbjuder flera fördelar inom programmering:
// 1. Kodåteranvändning: Genom att använda arv kan du återanvända kod
från överordnade
// klasser i de nedärvande klasserna. Detta minskar behovet av att
skriva samma kod
// flera gånger och förbättrar därmed kodens underhållbarhet och
läsbarhet.
// 2. Modulär design: Arv bidrar till en modulär design av program.
Genom att dela upp
// funktionaliteten i olika klasser och använda arv kan du separera
olika ansvarsområden
// och skapa en hierarki av klasser som är enklare att förstå och
hantera.
// 3. Kodens struktur: Arv kan förbättra kodens struktur och
organisering. Genom att placera
// gemensam funktionalitet i överordnade klasser och specialisera den
i nedärvande klasser
// blir koden mer lättläslig och intuitiv.
// 4. Utbytbarhet: Arv möjliggör att objekt av en nedärvande klass kan
användas där objekt
// av en överordnad klass förväntas. Detta skapar möjligheten att
behandla objekt på ett
// enhetligt sätt och gör koden mer flexibel och skalbar.
// Begränsningar
// Trots sina fördelar har arv vissa begränsningar och kompromisser:
// 1. Tätt kopplade klasser: Genom att använda arv skapas en tät
koppling mellan överordnade
// och nedärvande klasser. Om du gör ändringar i överordnade klasser
kan det påverka alla
// nedärvande klasser, vilket kan vara komplicerat att hantera och
underhålla.
// 2. Brist på flexibilitet: Arv kan begränsa flexibiliteten i en
kodbas. Om hierarkin av
// klasser inte är korrekt utformad kan det bli svårt att lägga till
eller ändra
// funktionalitet på ett smidigt sätt.
// 3. Ökad komplexitet: När hierarkin av klasser blir djup och komplex
kan det bli svårt att
// förstå och hantera koden. Det är viktigt att noggrant planera och
organisera
// klasshierarkin för att undvika överflödig komplexitet.
// Användningsområden
// Arv kan tillämpas i olika scenarier och användningsområden inom
programmering. Här är

```

```

programmering. Här är
// några exempel:
// 1. GUI-ramverk: I grafiska användargränssnittsramverk används arv för
att skapa hierarkier
//    av användargränssnittskomponenter. Till exempel kan en överordnad
klass "Komponent"
//    innehålla grundläggande egenskaper och beteenden, medan nedärvande
klasser som "Knapp"
//    och "Textfält" specialiserar funktionaliteten.
// 2. Spelprogrammering: I spelutveckling kan arv användas för att skapa
en hierarki av
//    spelobjekt. Till exempel kan en överordnad klass "Spelobjekt"
innehålla gemensamma
//    egenskaper och beteenden, medan nedärvande klasser som "Fiende"
och "Spelare"
//    specialiserar funktionaliteten för specifika spelkaraktärer.

```

För att visa hur arv fungerar i Java kan vi använda följande kodexempel: // En överordnad klass "Karaktär" som representerar en generell spelkaraktär. class Karaktär { String namn; int hälsa; public Karaktär(String namn, int hälsa) { this.namn = namn; this.hälsa = hälsa; public void attack() { System.out.println(namn + " attackerar!"); // En nedärvande klass "Fiende" som specialiserar funktionaliteten för en fiende i spelet. class Fiende extends Karaktär { int styrka; public Fiende(String namn, int hälsa, int styrka) { super(namn, hälsa); this.styrka = styrka; System.out.println(namn + " attackerar med styrka " + styrka + "!"); // En nedärvande klass "Spelare" som specialiserar funktionaliteten för spelaren i spelet. class Spelare extends Karaktär { int nivå; public Spelare(String namn, int hälsa, int nivå) { this.nivå = nivå; System.out.println(namn + " attackerar med nivå " + nivå + "!"); public class ArvExempel { public static void main(String[] args) { // Skapar en fiende och en spelare. Fiende fiende = new Fiende("Ondskans mästare", 100, 10); Spelare spelare = new Spelare("Hjälten", 100, 5); // Anropar attack-metoden för fienden och spelaren. fiende.attack(); spelare.attack(); I detta kodexempel används arv för att skapa en hierarki av spelkaraktärer. Den överordnade klassen "Karaktär" innehåller gemensamma egenskaper och beteenden för alla spelkaraktärer, medan nedärvande klasser som "Fiende" och "Spelare" specialiserar funktionaliteten för specifika spelkaraktärer. Båda nedärvande klasserna har en egen implementation av attack-metoden som anropas när de attackerar. När vi kör programmet skapar vi en instans av Fiende och en instans av Spelare och anropar— 3.

Databashanterare: I en databashanterare kan arv användas för att skapa en hierarki av databasobjekt. Till exempel kan en överordnad klass "DatabaseObject" innehålla generella funktioner för att hantera databasoperationer, medan nedärvande klasser som "Table" (Tabell) och "Query" (Fråga) specialiserar funktionaliteten för specifika databasentiteter. Detta är bara några exempel på användningsområden där arv kan tillämpas inom programmering. Principen om arv kan vara användbar i olika typer av program och system, oavsett om det är grafiska användargränssnitt, spel eller databashantering. ## Exempelkod - Arv i en berättelse

För att bättre förstå arv kan vi titta på ett kodexempel som illustrerar användningen av arv genom en berättelse. Anta att vi bygger ett spel där vi har olika typer av karaktärer, inklusive fiender och hjältar. Vi kan använda arv för att skapa en hierarki av karaktärsklasser. // Definiera överordnad klass Karaktär public class Karaktär { public String Namn; public int Hälsa; // Definiera nedärvande klass Fiende public class Fiende extends Karaktär { public void Attackera() { // Implementera attacklogik för fiender // Definiera nedärvande klass Hjärte public class Hjärte extends Karaktär { public void Försvara() { // Implementera försvarlogik för hjältar // Användning av arv i spellogik Fiende fiende = new Fiende(); fiende.Namn = "Ond skurk"; fiende.Hälsa = 100; fiende.Attackera(); Hjärte hjärte = new Hjärte(); hjärte.Namn = "Modig hjärte"; hjärte.Hälsa = 100; hjärte.Försvara(); I det här exemplet skapas en överordnad klass Karaktär som har egenskaper för Namn (Name) och Hälsa (Health). Därefter skapas två nedärvande klasser, Fiende (Enemy) och Hjärte (Hero), som båda ärver från Karaktär och lägger till sina egna specifika metoder (Attackera och Försvara). I spellogiken skapar vi en instans av Fiende och sätter dess egenskaper (Namn och Hälsa) och använder sedan dess specifika metod Attackera. På samma sätt skapar vi en instans av Hjärte, sätter dess egenskaper och använder dess specifika metod Försvara. Genom att använda arv kan vi enkelt skapa en hierarki av karaktärsklasser och använda deras specifika metoder och egenskaper baserat på deras roll i spelet. I detta kodexempel har vi en överordnad klass Karaktär som innehåller gemensamma egenskaper för både fiender och hjältar. Genom att ärva från Karaktär kan vi definiera specialiserad funktionalitet för fiender och hjältar i deras respektive nedärvande klasser Fiende och Hjärte. Vi kan sedan skapa instanser av dessa klasser och använda deras unika funktioner, som Attackera() för fiender och Försvara() för hjältar. ### Utdata

(fiende.Attackera() skriver ut något här) (hjärte.Försvara() skriver ut något här) Detta är bara ett enkelt exempel som visar hur arv kan användas för att skapa hierarkier av klasser och dela funktionalitet mellan dem. I praktiken kan arv vara mycket mer kraftfullt och komplext i sina tillämpningar. ## Slutsats

Arv är en viktig princip inom objektorienterad programmering som möjliggör återanvändning av kod och skapar hierarkier av klasser. Genom att använda arv kan vi strukturera och organisera vår kod på ett modulärt sätt, vilket förbättrar underhållbarheten och läsbarheten. I denna artikel har vi utforskat arvets fördelar, inklusive kodåteranvändning, modulär design och strukturerad kod. Vi har också diskuterat dess begränsningar och utmaningar, såsom tät koppling och ökad komplexitet. Arv kan tillämpas inom olika områden inom programmering, från grafiska användargränssnittsramverk till spelprogrammering och databashanterare. Genom att förstå och behärska arv kan du utveckla effektivare och mer flexibla program. För att fördjupa dina kunskaper rekommenderar vi att du fortsätter läsa om arv, utforskar mer avancerade koncept som abstrakt arv och gränssnitt och experimenterar med att använda arv i dina egna programmeringsprojekt. ## TL;DR Arv i Java:

Arv i Java är en princip som möjliggör att en klass kan ärva egenskaper och beteenden från en annan klass. Detta möjliggör kodåteranvändning och skapar en hierarki av klasser. För att implementera arv i Java används nyckelordet "extends". En klass kan ärva från en annan klass genom att använda extends-nyckelordet och ange namnet på den överordnade klassen. Klassen som ärver kallas subklassen och den överordnade klassen kallas superklassen. Här är ett exempel som visar hur arv fungerar i Java: // Superklass class Djur { public void sägLjud() { System.out.println("Djuret säger ljudet"); } // Subklass som ärver från superklassen Djur class Hund extends Djur { System.out.println("Hunden säger voff"); } public class Main { Hund hund = new Hund(); hund.sägLjud(); // Output: Hunden säger voff I exemplet ovan har vi en superklass "Djur" och en subklass "Hund". Subklassen "Hund" ärver egenskaper och beteenden från superklassen "Djur". Subklassen kan också överlagra metoder från superklassen för att ändra deras beteende. I main-metoden skapar vi en instans av subklassen "Hund" och anropar metoden "sägLjud()". Eftersom subklassen har överlagrat metoden, kommer den att visa det specifika ljudet för hundar. Arv möjliggör också flera nivåer av hierarkier. En subklass kan i sin tur fungera som superklass för en annan subklass. På så sätt kan vi bygga mer komplexa strukturer av klasser och utnyttja arvets fördelar. Det är viktigt att notera att Java inte stöder flerfaldig arv, vilket innebär att en klass inte kan ärva från flera superklasser samtidigt. Java använder istället gränssnitt (interface) för att uppnå liknande funktionalitet. Gränssnitt låter en klass implementera flera gränssnitt samtidigt, vilket ger flexibilitet och kodåteranvändning.

Java för Jin23

Marcus Medina

Komposition över arv

Vad är det för något?

Varför är det viktigt?

Fördelar

Begränsningar

Enkel förklaring

Exempel 1

Exempel 2

Sammanfattning

Komposition över arv

Vad är det för något?

Komposition över arv är ett designmönster inom objektorienterad programmering som förespråkar att "sammansättning" (komposition) bör användas istället för "arv" (inheritance) för att uppnå återanvändbarhet och flexibilitet i kod. Detta mönster är baserat på principen om att klasser bör sammansättas av andra klasser snarare än att ärva från dem.

Varför är det viktigt?

Komposition över arv är ett viktigt koncept inom objektorienterad programmering eftersom det hjälper oss att skapa flexibla och återanvändbara klasser. Genom att använda komposition kan vi enkelt kombinera olika klasser för att skapa nya klasser med olika beteenden och egenskaper. Detta gör det möjligt för oss att skapa modulära och flexibla program som är lätta att underhålla och utöka.

Fördelar

Komposition över arv erbjuder flera fördelar och används i olika situationer:

1. **Flexibilitet:** Genom att använda komposition kan vi enkelt kombinera olika klasser för att skapa nya klasser med olika beteenden och egenskaper. Detta gör det möjligt för oss att skapa modulära och flexibla program som är lätta att underhålla och utöka.
2. **Återanvändbarhet:** Genom att använda komposition kan vi återanvända befintlig kod och undvika att duplicera kod. Detta gör det möjligt för oss att skapa modulära och flexibla program som är lätta att underhålla och utöka.
3. **Moduläritet:** Genom att använda komposition kan vi skapa modulära program som är lätta att underhålla och utöka. Detta gör det möjligt för oss att skapa modulära och flexibla program som är lätta att underhålla och utöka.
4. **Minskad beroende:** Genom att använda komposition kan vi minska beroendet mellan klasser och undvika att duplicera kod. Detta gör det möjligt för oss att skapa modulära och flexibla program som är lätta att underhålla och utöka.

Begränsningar

Komposition över arv har vissa begränsningar att vara medveten om:

1. **Ökad komplexitet:** Genom att använda komposition kan vi öka komplexiteten i vårt program. Detta kan göra det svårare att förstå och underhålla koden.
2. **Mer kod att skriva:** Genom att använda komposition kan vi behöva skriva mer kod för att uppnå samma resultat som med arv. Detta kan göra det svårare att förstå och underhålla koden.

Enkel förklaring

Man skulle förenkla det hela med att säga att vi simulerar "arv av flera klasser" och undvika att duplicera kod. Detta gör det möjligt för oss att skapa modulära och flexibla program som är lätta att underhålla och utöka.

Exempel 1

Känns det rörigt? Lugnt, vi kollar på ett exempel... Vi har faktiskt använt detta innan...

kolla koden nedan

```

class Pet
{
    private String name;
    private int age;
    private String breed;

    public Pet(String name, int age, String breed)
    {
        this.name = name;
        this.age = age;
        this.breed = breed;
    }

    public String getName(){ return name; }
    public int getAge(){ return age; }
    public String getBreed(){ return breed; }
    public void setName(String name){ this.name = name; }
    public void setAge(int age){ this.age = age; }
    public void setBreed(String breed){ this.breed = breed; }
}

class Person
{
    private String name;
    private int age;

    public Person(String name, int age)
    {
        this.name = name;
        this.age = age;
    }

    public String getName(){ return name; }
    public int getAge(){ return age; }
    public void setName(String name){ this.name = name; }
    public void setAge(int age){ this.age = age; }
}

class HumanPet
{
    private Person person;
    private Pet pet;

    public HumanPet(Person person, Pet pet)
    {
        this.person = person;
        this.pet = pet;
    }

    public Person getPerson(){ return person; }
    public Pet getPet(){ return pet; }
    public void setPerson(Person person){ this.person = person; }
    public void setPet(Pet pet){ this.pet = pet; }
}

```

Här hade vi kunnat ärva Person eller Pet men vi väljer självklart komposition över arv. Oavsett variant skulle det orsakat problem. Så nu kan vi ärv HumanPet och få med oss både Person och Pet... om vi nu vill det.

```

class Family
{
    private HumanPet father;
    private HumanPet mother;
    private HumanPet[] child;
    private Pet[] familyPets;

    public Family(HumanPet father, HumanPet mother, HumanPet[] child,
        Pet[] familyPets)
    {
        this.father = father;
        this.mother = mother;
        this.child = child;
        this.familyPets = familyPets;
    }

    public HumanPet getFather(){ return father; }
    public HumanPet getMother(){ return mother; }
    public HumanPet getChild(){ return child; }
    public Pet[] getFamilyPets(){ return familyPets; }
    public void setFather(HumanPet father){ this.father = father; }
    public void setMother(HumanPet mother){ this.mother = mother; }
    public void setChild(HumanPet child){ this.child = child; }
    public void setFamilyPets(Pet[] familyPets){ this.familyPets =
        familyPets; }
}

```

Nu kan vi skapa en familj där alla har var sitt husdjur.

Exempel 2

För att förstå hur man implementerar "Komposition över Arv" i praktiken, låt oss titta på ett exempel med Java-kod. Anta att vi har ett program som hanterar karaktärer i en Star Wars-värld. Vi har två klasser: Character och Weapon.

```

public class Character {
    private String name;
    private Weapon weapon;

    public Character(String name, Weapon weapon) {
        this.name = name;
        this.weapon = weapon;
    }

    public void attack() {
        if (weapon != null) {
            weapon.use();
        } else {
            System.out.println(name + " has no weapon.");
        }
    }
}

```

Ovan skapade vi karaktärer, vapen och använde dem i vår applikation. Vi skapade en instans av Character och tilldelade den ett vapen. Sedan anropade vi attack()-metoden för att se hur karaktären använder sitt tilldelade vapen.

Nu ska vi skapa vapen.

```

public interface Weapon {
    void use();
}

public class Lightsaber implements Weapon {
    @Override
    public void use() {
        System.out.println("Swinging lightsaber!");
    }
}

public class Blaster implements Weapon {
    @Override
    public void use() {
        System.out.println("Shooting blaster!");
    }
}

```

I det här exemplet representerar `Character` en karaktär i Star Wars och `Weapon` representerar ett vapen som kan användas av karaktären. Istället för att använda arv för att definiera olika typer av karaktärer med specifika vapen, använder vi komposition för att sammansätta karaktärer med vapen.

En karaktär har ett namn och en referens till ett vapen. I `Character`-klassen finns en metod `attack()` som anropar `use()`-metoden för det tilldelade vapnet. Om karaktären inte har något vapen skrivs ett meddelande ut.

Vi har också en `Weapon`-gränssnitt som definierar en enda metod `use()`. Både `Lightsaber`- och `Blaster`-klasserna implementerar detta gränssnitt och tillhandahåller implementationer för `use()`-metoden.

För att använda detta i vår applikation kan vi skapa instanser av `Character` och `Weapon` och sätta ihop dem genom att tilldela ett vapen till en karaktär.

```

public class Main {
    public static void main(String[] args) {
        Weapon lightsaber = new Lightsaber();
        Character jedi = new Character("Luke Skywalker", lightsaber);
        jedi.attack(); // Output: Swinging Lightsaber!

        Weapon blaster = new Blaster();
        Character bountyHunter = new Character("Boba Fett", blaster);
        bountyHunter.attack(); // Output: Shooting blaster!

        Character padawan = new Character("Ahsoka Tano", null);
        padawan.attack(); // Output: Ahsoka Tano has no weapon.
    }
}

```

I `Main`-klassen skapar vi instanser av `Lightsaber` och `Blaster` som representerar vapen. Sedan skapar vi instanser av `Character` och tilldelar respektive vapen till dem. Genom att anropa `attack()`-metoden för varje karaktär kan vi se hur de använder sina tilldelade vapen.

Detta exempel visar hur vi kan använda komposition för att sammansätta klasser och uppnå flexibilitet och återanvändbarhet. Istället för att ha en hierarki av klasser med olika kombinationer av vapen, kan vi enkelt kombinera olika karaktärer och vapen genom att ändra vilka objektinstanser som tilldelas till varje karaktär.

Sammanfattning

I denna artikel har vi utforskat konceptet "Komposition över Arv" och dess betydelse inom programmering. Vi har sett fördelarna med att använda komposition, såsom flexibilitet, modularitet, återanvändbarhet och minskat beroende. Vi har också identifierat begränsningar som ökad komplexitet och mer kod att skriva. Genom ett kodexempel har vi sett hur man kan implementera komposition i praktiken med hjälp av Java.

För vidare läsning rekommenderas att utforska olika designmönster och principer inom objektorienterad programmering, såsom SOLID-principerna och *Design Patterns*-boken av Erich Gamma m.fl.

Java för Jin23

Marcus Medina

Åtkomstmoderator

Åtkomstmoderator

En åtkomstmodifikator är en modifierare som används för att definiera åtkomsten till en klassmedlem. Detta innebär att du kan bestämma vilka klassmedlemmar som ska vara tillgängliga för andra klasser och vilka som ska vara privata för klassen.

Det finns fyra huvudsakliga åtkomstmodifikatorer i Java:

1. **public**: Tillåter åtkomst till en klassmedlem från alla andra klasser.
2. **private**: Tillåter åtkomst till en klassmedlem endast från den klass där den är deklarerad.
3. **protected**: Tillåter åtkomst till en klassmedlem endast från den klass där den är deklarerad och alla dess underklasser.
4. **ingen åtkomstmodifikator**: Om ingen åtkomstmodifikator anges, är standardåtkomsten "package-private". Det innebär att klassmedlemmen är tillgänglig endast inom samma paket.
5. **final**: Tillåter inte att en klassmedlem ändras efter att den har deklarerats.
6. **package-private** (default): Tillåter åtkomst till en klassmedlem endast inom samma paket.
7. **static**: Tillåter åtkomst till en klassmedlem utan att skapa en instans av klassen.
8. **abstract**: Tillåter inte att en klassmedlem har en implementation.
9. **transient**: Tillåter inte att en klassmedlem sparas när ett objekt serialiseras.
10. **volatile**: Tillåter inte att en klassmedlem cachas av JVM.
11. **synchronized**: Tillåter inte att flera trådar samtidigt ändrar en klassmedlem.
12. **native**: Tillåter inte att en klassmedlem implementeras i Java.
13. **sealed**: Tillåter inte att en klassmedlem ärver från en annan klass.

Exempel:

```
public class Person {  
    public String name; // Tillgänglig från alla klasser  
    private int age; // Endast tillgänglig från Person-klassen  
    protected String address; // Tillgänglig från Person-klassen och  
        dess underklasser  
    String phoneNumber; // Endast tillgänglig inom samma paket som  
        Person-klassen  
}
```

I exemplet ovan är name tillgänglig från alla klasser, age är endast tillgänglig från Person-klassen, address är tillgänglig från Person-klassen och dess underklasser, och phoneNumber är endast tillgänglig inom samma paket som Person-klassen.

Java för Jin23

Marcus Medina

Private

Beskrivning

Exempel

Private

Privat är en åtkomstmodifikator som gör att en klassmedlem endast är tillgänglig för den klass där den är deklarerad.

▼ Innehållsförteckning

1. TOC

Beskrivning

Vi kan använda privat för att göra en klassmedlem endast tillgänglig för den klass där den är deklarerad. Detta är användbart om vi vill att en klassmedlem endast ska vara tillgänglig för den klass där den är deklarerad och inte för någon annan klass.

Exempel

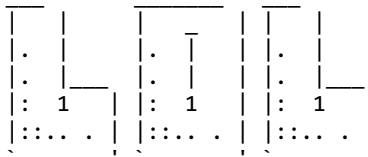
Låt oss titta på ett exempel där vi använder privat för att göra en klassmedlem endast tillgänglig för den klass där den är deklarerad:

```
public class Person {  
    private String name;  
    private int age;  
}
```

I detta exempel har vi en klass som heter Person. Vi har också två egenskaper, name och age. Båda är privata, vilket innebär att de endast är tillgängliga för klassen Person. Även om name och age borde vara tillgängliga för alla klasser som länkas till denna, kommer de inte att vara tillgängliga för någon annan klass på grund av att de är privata.

Detta innebär att vi inte kan använda dem i klassen Person själv heller. Vi kan inte ens använda dem i en konstruktor i Person-klassen. Detta beror på att privata medlemmar endast är tillgängliga för den klass där de är deklarerade.

Jaja det är ju bara ett exempel. Det får räcka som Dad-Joke för den här artikeln.



Java för Jin23

Marcus Medina

Protected

Beskrivning

Exempel

Förklaring

Obligatorisk Dad-Joke

Protected

Protected är en åtkomstmodifierare som gör att en klass, metod eller egenskap är tillgänglig för klassen den är deklarerad i och alla klasser som ärver från den.

▼ Innehållsförteckning

1. TOC

Beskrivning

När vi arbetar med polymorfism kan det vara bra att göra en metod eller egenskap tillgänglig för alla klasser som ärver från en klass. Detta gör vi genom att använda protected. Protected är som private för alla klasser, utom den som ärver. Klasser kan inte ärva privata medlemmar, så detta är det bästa alternativet.

Exempel

```
// Klassen Person ärver inte från någon annan klass
// Eller ja... den ärver från Objekt, det gör alla klasser :)
public class Person {
    protected String name;
}

// Klassen Student ärver från Person och kan därför använda "name"
public class Student extends Person {
    public Student(String name) {
        this.name = name;
    }
}

// Klassen Student kan nu använda "name" och skriva ut det
public void printInfo() {
    System.out.println("Student Info: " + name);
}
```

Förklaring

I exemplet ovan är "name" propertyn i klassen Person skyddad för alla klasser förutom de som ärver från den, vilket gör att klassen Student kan lägga till ett namn och skriva ut det, medan inga andra klasser kan göra det.

Obligatorisk Dad-Joke

Varför var det nyckelordet "protected" så nervöst hela tiden?

För att det alltid ville hålla sitt privata liv... väl, "protected"! ☐

Java för Jin23

Marcus Medina

Public

- Beskrivning
- Exempel
- Förklaring
- Getters och setters?
- Exempel
- Sammanfattning

Public

Public is an access modifier that allows a class, method, or property to be accessible to all classes.

▼ Table of Contents

1. TOC

Beskrivning

Med public kan vi göra en klass, metod eller egenskap tillgänglig för alla klasser. Detta är bra om vi vill dela kod mellan flera klasser i olika projekt.

Exempel

```
public class Person {  
    public String name;  
    public int age;  
}
```

Förklaring

I exemplet ovan är klassen Person tillgänglig för alla klasser i alla projekt. Detta gör att vi kan skapa en ny instans av Person i en annan klass och lägga till ett namn och ålder.

Getters och setters?

Varför har man Getters och Setters istället för publika variabler?

Namn	Fördel	Nackdel
Getter	Du kan göra mer än bara returnera en variabel. Du kan till exempel validera värdet innan du returnerar det. Du kan returnera resultat av data i din klass istället för en specifik variabel.	Du måste skriva mer kod.
Setter	Du kan göra mer än bara tilldela en variabel. Du kan till exempel validera värdet innan du tilldelar det. Om värdet inte är bra för din klass kan kasta ett felmeddelande eller bara vägra att spara värdet	Du måste skriva mer kod.
Public	Du behöver inte skriva mer kod.	Du kan inte göra mer än att tilldela eller returnera en variabel.

Exempel

```
// Setter som inte accepterar negativa tal
public void setAge(int age) {
    if (age < 0) {
        throw new IllegalArgumentException("Age cannot be negative");
    }
    this.age = age;
}

// Getter som returnerar åldern på en person baserat på en annan
// variabel (Date)
public int getAge(Date date) {
    return date.getYear() - this.birthDate.getYear();
}
```

Sammanfattning

Public är en åtkomstmodifikator som gör att en klass, metod eller egenskap är tillgänglig för alla klasser. Detta är bra om vi vill dela kod mellan flera klasser i olika projekt. Att dela metoder till externa klasser är ett bra sätt för att få klasser att samarbeta.

Java för Jin23

Marcus Medina

Static

Beskrivning
Exempel
Obligatorisk Dad-joke

Static

En static är en åtkomstmodifikator som gör att en klassmedlem tillhör klassen och inte objektet.

▼ Innehållsförteckning

Beskrivning

Vi kan använda static för att göra en klassmedlem tillhör klassen och inte objektet. Detta är användbart om vi vill dela data mellan olika klasser utan att behöva skapa flera instanser av en klass.

När koden kompileras kommer alla statiska medlemmar att tilldelas ett gemensamt minnesutrymme som delas av alla instanser av klassen. Detta gör att vi kan dela data mellan olika klasser utan att behöva skapa flera instanser av en klass. Detta är användbart om vi vill dela data mellan olika klasser utan att behöva skapa flera instanser av en klass.

mindmap root((Minnet)) Instanser av klasser Instansvariabler Instansmetoder
Statiskt Statiska variabler Statiska metoder

I detta diagram representerar "Minne" den allmänna minnesplatsen där programmet körs.

- Instanser av klasser representerar alla instanser av klasser som skapas under körningen.
- Statiskt representerar alla statiska variabler och metoder som skapas under körningen.

Observera att diagrammet bara visar en förenklad representation av hur en statisk metod och instansvariabler lagras i minnet vid kompileringen. Det kan finnas fler faktorer och detaljer som påverkar minnesorganisationen beroende på den specifika implementationen och miljön där programmet körs.

Hoppas att detta diagram hjälper till att illustrera hur en statisk metod läggs i minnet! Låt mig veta om du har fler frågor.

Exempel

```
public class UserSettings {  
    public static String UserName;  
    public static int Password;  
    public static boolean DarkMode = true;  
}
```

I detta exempel har vi en klass som heter UserSettings. Vi har också tre statiska egenskaper: UserName, Password och DarkMode. Dessa egenskaper är tillgängliga för alla klasser i samma projekt. Men om en annan klass i ett annat projekt länkar till detta projekt kommer de inte att kunna använda UserSettings-klassen.

Det är viktigt att notera att i Java kan vi inte ha statiska egenskaper med automatisk implementering som i C#. Istället måste vi använda publika statiska variabler och få tillgång till dem direkt genom klassnamnet.

För att använda `UserSettings`-klassen kan vi tilldela värden till de statiska egenskaperna innan vi använder dem:

```
UserSettings.UserName = "JohnDoe";  
UserSettings.Password = 123456;  
UserSettings.DarkMode = true;
```

Och sedan kan vi använda dessa värden i andra delar av koden:

```
System.out.println("Användarnamn: " + UserSettings.UserName);  
System.out.println("Lösenord: " + UserSettings.Password);  
System.out.println("Mörkt läge: " + UserSettings.DarkMode);
```

Detta ger oss möjligheten att dela data mellan olika klasser utan att behöva skapa flera instanser av `UserSettings`-klassen. Vi kan också ändra värdena i `UserSettings`-klassen och ha tillgång till de uppdaterade värdena överallt där vi använder klassen.

Detta är bara en av många användningar av `static` i Java. Det kan också användas för att skapa statiska metoder och block, vilket vi kommer att utforska i andra artiklar.

Obligatorisk Dad-joke

Varför var den statiska metoden så dålig på att socialisera?

För att den aldrig kunde få någon att "dynamiskt" intressera sig för den! ☐

Java för Jin23

Marcus Medina

Polymorfism
Beskrivning

Polymorfism

Polymorfism kommer från grekiskans poly = många och morphe = form. Det är en av de viktigaste egenskaperna i objektorienterad programmering.

▼ Innehållsförteckning
1. TOC

Beskrivning

Polymorfism är en egenskap som gör att objekt som ärver från samma familj av en klass eller ett interface kan hanteras som om de vore samma typ. Detta ger oss möjligheten att skapa en lista av olika objekt från samma familj och ändå kunna hantera dem på samma sätt. Polymorfism ger oss många fördelar:

- Möjlighet att byta ut delar av programmet utan att behöva ändra resten av programmet.
- Möjlighet att skapa en metod som kan hantera olika typer av objekt utan att behöva skapa en metod för varje typ av objekt.
- Möjlighet att skapa plugins till programmet utan att behöva ändra i programmet, med mera.

Java för Jin23

Marcus Medina

Här är den korrigerade och förbättrade versionen av artikeln med kommentarer:

Interfaces

Gränssnitt är ett kraftfullt verktyg i Java som ger oss möjlighet att skapa flexibla och återanvändbara komponenter i våra program. Genom att använda gränssnitt kan vi implementera polymorfism och separera implementation och användning av komponenter.

Beskrivning

Ett gränssnitt är en typ som definierar en uppsättning metoder, egenskaper och händelser som en klass kan implementera. En klass som implementerar ett gränssnitt måste implementera alla dess medlemmar. Därför säger man att gränssnitt är ett kontrakt.

Exempel

Låt oss titta på ett exempel där vi skapar ett gränssnitt som heter `Animal`:

```
```java
interface Animal {
 String getName();
 void eat();
 void sleep();
 void shit();
}
```

I detta exempel har vi ett gränssnitt som heter `Animal`. Vi har också tre metoder och en egenskap. Observera att gränssnitt inte kan ha fält, men de kan ha egenskaper!

```
class Cat implements Animal {
 private String name;

 public Cat(String name) {
 this.name = name;
 }

 public String getName() {
 return name;
 }

 public void eat() {
 System.out.println(name + " is eating.");
 }

 public void sleep() {
 System.out.println(name + " is sleeping.");
 }

 public void shit() {
 System.out.println(name + " is taking a shit.");
 }
}
```

I detta exempel har vi en klass som heter `Cat` och den implementerar gränssnittet `Animal`. Vi har en privat egenskap för namnet på katten och en konstruktor för att sätta namnet. Vi implementerar också alla metoder från gränssnittet `Animal`.

Nu kan vi skapa en instans av katten och använda dess metoder:

```

public class Main {
 public static void main(String[] args) {
 Cat cat = new Cat("Whiskers");
 cat.eat();
 cat.sleep();
 cat.shit();
 }
}

```

Output:

```

Whiskers is eating.
Whiskers is sleeping.
Whiskers is taking a shit.

```

I detta exempel skapar vi en instans av katten med namnet "Whiskers" och använder sedan dess metoder för att få katten att äta, sova och göra sina behov. Resultatet skrivs ut i konsolen.

Detta är ett grundläggande exempel på hur gränssnitt kan användas för att implementera polymorfism och separera implementationen av komponenter från deras användning. Genom att använda gränssnitt kan vi skapa flexibla och återanvändbara komponenter i våra program.

Gränssnitt är ett viktigt koncept inom objektorienterad programmering och ger oss möjlighet att skapa en enhetlig och modulär kodstruktur. Genom att definiera gränssnitt kan vi även underlätta samarbete mellan olika utvecklare genom att specificera vilka metoder som förväntas implementeras.

Användningen av gränssnitt är vanligt förekommande i Java och är en viktig del av språkets design. Genom att använda gränssnitt kan vi skapa mer flexibla och underhållbara program.

Kommentarer: - Animal: Gränssnittet Animal definierar fyra metoder: getName(), eat(), sleep() och shit(). Dessa metoder måste implementeras av alla klasser som implementerar gränssnittet Animal. - Cat: Klassen Cat implementerar gränssnittet Animal och tillhandahåller implementationen av de fyra metoderna. Den har också en privat egenskap name för att lagra namnet på katten. - Main: Huvudklassen Main skapar en instans av Cat med namnet "Whiskers" och använder sedan kattens metoder för att simulera kattens beteende.

Genom att använda gränssnitt kan vi skapa en gemensam plattform för olika klasser och möjliggöra utbytbarhet och polymorfism. Gränssnitt ger oss flexibilitet och en tydlig struktur i vår kod.

Detta var en introduktion till gränssnitt i Java. Förhoppningsvis har du fått en grundläggande förståelse för hur man skapar och använder gränssnitt i dina Java-program. Fortsätt utforska och experimentera med gränssnitt för att bygga mer flexibla och återanvändbara komponenter i dina program!

# Java för Jin23

Marcus Medina

## Plugins

Fördelar med att använda Plugins

Hur man implementerar ett pluginsystem med gränssnitt (interfaces)

Steg 1: Definiera ett gränssnitt för Plugins

Steg 2: Implementera Plugins

Steg 3: Ladda och köra Plugins

Slutsats

Användningsområden för Pluginsystem

## Plugins

Plugins är en kraftfull mekanism för att lägga till ny funktionalitet i en applikation. De ger en möjlighet att utöka och anpassa applikationen utan att behöva modifiera dess kärnfunktionalitet. I denna artikel kommer vi att utforska hur man implementerar ett pluginsystem med hjälp av gränssnitt (interfaces) i Java.

## Fördelar med att använda Plugins

Att använda ett pluginsystem i din applikation kan ge flera fördelar:

- **Modularitet:** Ett pluginsystem främjar en modulär design där olika delar av funktionaliteten kan isoleras i separata plugins. Detta gör det enkelt att lägga till, ändra eller ta bort specifika funktioner utan att påverka resten av applikationen.
- **Utbytbarhet:** Genom att använda ett pluginsystem kan användare eller utvecklare enkelt byta ut eller uppgradera plugins utan att påverka kärnfunktionaliteten i applikationen. Detta möjliggör snabb anpassning och utveckling av applikationen efter behov.
- **Återanvändbarhet:** Plugins kan vara återanvändbara och användas i olika applikationer. Detta sparar tid och resurser genom att undvika att bygga samma funktionalitet från grunden i varje applikation.
- **Flexibilitet:** Ett pluginsystem ger möjlighet att anpassa applikationen efter olika användares eller kunders krav och preferenser. Användare kan välja vilka plugins de vill använda och konfigurera applikationen enligt sina behov.

## Hur man implementerar ett pluginsystem med gränssnitt (interfaces)

För att implementera ett pluginsystem med hjälp av gränssnitt i Java följer vi följande steg:

### Steg 1: Definiera ett gränssnitt för Plugins

Vi börjar med att definiera ett gränssnitt (interface) som beskriver den gemensamma funktionaliteten som alla plugins måste implementera. Detta gränssnitt kommer att fungera som en kontrakt för att säkerställa att alla plugins följer samma struktur och regler.

```
public interface Plugin {
 void run();
}
```

I det här exemplet har vi definierat ett `Plugin`-gränssnitt med en metod `run()`. Alla plugins i vårt system måste implementera denna metod.

## Steg 2: Implementera Plugins

Nästa steg är att implementera olika plugins genom att skapa klasser som implementerar det Plugin-gränssnitt som vi har definierat. Varje pluginsklass kommer att innehålla den specifika funktionaliteten för pluginet.

```
public class MarvelPlugin implements Plugin {
 @Override
 public void run() {
 System.out.println("Kör Marvel-pluginet...");
 // Implementera Marvel-specifik funktionalitet här
 }
}

public class DCPlugin implements Plugin {
 @Override
 public void run() {
 System.out.println("Kör DC-pluginet...");
 // Implementera DC-specifik funktionalitet här
 }
}

public class StarWarsPlugin implements Plugin {
 @Override
 public void run() {
 System.out.println("Kör Star Wars-pluginet...");
 // Implementera Star Wars-specifik funktionalitet här
 }
}
```

I det här exemplet har vi implementerat tre olika plugins: MarvelPlugin, DCPlugin och StarWarsPlugin. Varje pluginsklass innehåller sin egen implementation av run()-metoden.

## Steg 3: Ladda och köra Plugins

Det sista steget är att ladda och köra plugins i vår applikation. För att göra detta kan vi använda Java Reflection API för att dynamiskt ladda klasser och skapa instanser av dem.

```
import java.util.ArrayList;
import java.util.List;

public class Application {
 private List<Plugin> plugins;

 public Application() {
 plugins = new ArrayList<>();
 }

 public void loadPlugins() {
 // Ladda alla plugins från en viss mapp eller konfiguration
 // Till exempel kan vi lägga till MarvelPlugin, DCPlugin och
 // StarWarsPlugin i plugins-listan
 plugins.add(new MarvelPlugin());
 plugins.add(new DCPlugin());
 plugins.add(new StarWarsPlugin());
 }

 public void runPlugins() {
 for (Plugin plugin : plugins) {
 plugin.run();
 }
 }

 public static void main(String[] args) {
 Application app = new Application();
 app.loadPlugins();
 app.runPlugins();
 }
}
```

I det här exemplet har vi skapat en `Application`-klass som har en lista av `Plugin`-instanser. Vi kan använda `loadPlugins()`-metoden för att ladda alla plugins som vi vill använda i vår applikation. Sedan kan vi använda `runPlugins()`-metoden för att köra alla plugins i ordning.

## Slutsats

Ett pluginsystem är en kraftfull mekanism som möjliggör enkel tillägg av ny funktionalitet till en applikation. Genom att använda gränssnitt kan vi definiera gemensam funktionalitet som alla plugins måste implementera, vilket ger en flexibel och modular arkitektur. Genom att följa stegen som beskrivs ovan kan vi implementera ett pluginsystem i Java och dra nytta av dess fördelar.

## Användningsområden för Pluginsystem

Pluginsystem kan vara användbara i en mängd olika scenarier och applikationer. Här är några exempel på användningsområden:

- **Textredigerare:** Ett pluginsystem kan användas för att lägga till nya funktioner och redigeringsfunktion

er i en textredigerare, som stavningskontroll, syntaxhöjning eller automatiserade uppgifter.

- **Webbläsare:** Webbläsare kan dra nytta av pluginsystem för att lägga till stöd för olika webbt tekniker, tillägg för annonserhantering eller användarskript.
- **Grafikprogram:** Grafikprogram kan använda plugins för att lägga till nya filter, effekter eller verktyg för att utöka funktionaliteten.
- **E-handelsplattformar:** Plugins kan användas för att lägga till betalningsportaler, fraktoalternativ eller integration med externa tjänster i en e-handelsplattform.

Dessa är bara några exempel, och det finns många andra områden där pluginsystem kan vara användbara.

# Java för Jin23

Marcus Medina

Abstrakta klasser

Beskrivning

Exempel

Referenser

## Abstrakta klasser

Abstrakta klasser är klasser som innehåller både kod och abstrakta metoder. De fungerar som en kombination av ett interface och en klass.

▼ Innehållsförteckning

1. TOC

### Beskrivning

En abstrakt klass används för att andra klasser ska kunna ära från den. Det går inte att skapa objekt av en abstrakt klass, utan den används endast för att andra klasser ska kunna ära från den.

### Exempel

Vi skapar en klass kallad Shape, i den har vi metoden area(). Det ser enkelt ut men det är effektivt.

```

// Abstrakt klass för geometriska former
public abstract class Shape {
// Abstrakt metod för att beräkna arean
public abstract double area();
}

// Underklass för cirkel som ärver från Shape
public class Circle extends Shape {
private double radius; // Radien på cirkeln

// Konstruktör för att skapa en cirkel med given radie
public Circle(double radius) {
this.radius = radius;
}

// Getter för att hämta radien på cirkeln
public double getRadius() {
return radius;
}

// Setter för att sätta radien på cirkeln
public void setRadius(double radius) {
this.radius = radius;
}

// Överskuggning av area-metoden för att beräkna arean av cirkeln
@Override
public double area() {
return Math.PI * radius * radius;
}
}

// Underklass för rektangel som ärver från Shape
public class Rectangle extends Shape {
private double width; // Bredden på rektangeln
private double height; // Höjden på rektangeln

// Konstruktör för att skapa en rektangel med given bredd och höjd
public Rectangle(double width, double height) {
this.width = width;
this.height = height;
}

// Getter för att hämta bredden på rektangeln
public double getWidth() {
return width;
}

// Setter för att sätta bredden på rektangeln
public void setWidth(double width) {
this.width = width;
}

// Getter för att hämta höjden på rektangeln
public double getHeight() {
return height;
}

// Setter för att sätta höjden på rektangeln
public void setHeight(double height) {
this.height = height;
}

// Överskuggning av area-metoden för att beräkna arean av rektangeln
@Override
public double area() {
return width * height;
}
}

```

Vi kan nu skapa en array av typen Shape och lägga in objekt av typen Circle och Rectangle i den. Vi kan sedan loopa igenom arrayen och anropa area-metoden på varje objekt.

```
// Skapar en array av typen Shape
Shape[] shapes = new Shape[2];

// Lägger in ett objekt av typen Circle i arrayen
shapes[0] = new Circle(5);

// Lägger in ett objekt av typen Rectangle i arrayen
shapes[1] = new Rectangle(5, 10);

// Loopar igenom arrayen och anropar area-metoden på varje objekt
for (Shape shape : shapes) {
 System.out.println(shape.area());
}
```

Detta gör att vi kan använda oss av polymorfism för att beräkna arean av olika geometriska former. Coolt va!

## Referenser

- Abstrakta klasser
- Abstract class W3Schools

# Java för Jin23

Marcus Medina

## Exempel

Beskrivning  
Fördelar  
Begränsningar  
Användningsområden  
Kodexempel  
Termer  
Slutsats  
Referenser

## Exempel

I den här artikeln ska vi titta på hur man skapar en abstrakt klass med abstrakta och virtuella metoder i Java.

För detta behöver vi Maven libraryn JSOUP 1.14.3 eller nyare.

org.jsoup jsoup 1.14.3

## Beskrivning

En abstrakt klass kan innehålla vanliga metoder, abstrakta och virtuella metoder. Vi kommer att använda den här mallen för att skapa en abstrakt klass som hanterar webbscrapning för olika sidor. Vi använder även följande Java-paket:

- JSoup för att hantera HTML-dokument.

## Fördelar

När du använder abstrakta klasser kan du dra nytta av följande fördelar:

1. **Återanvändbarhet:** Abstrakta klasser kan fungera som grund för andra klasser och möjliggöra återanvändning av kod.
2. **Moduläritet:** Genom att använda abstrakta metoder kan du separera implementationen av en metod från själva klassen, vilket leder till en modulär design.
3. **Flexibilitet:** Abstrakta klasser kan användas som bas för olika implementationer och ge flexibilitet i utvecklingen.

## Begränsningar

Det finns några begränsningar att vara medveten om när du använder abstrakta klasser:

1. **Enkel arv:** En klass kan bara ära från en enda abstrakt klass, vilket kan begränsa möjligheterna till hierarkiska strukturer.
2. **Ingen direkt instansiering:** Eftersom en abstrakt klass inte kan instansieras direkt måste den äras och implementeras i en konkret klass för att användas.

## Användningsområden

Abstrakta klasser är användbara i olika scenarier, inklusive:

1. **Framtida utbyggbarhet:** Genom att definiera abstrakta metoder kan du planera för framtida utökning och implementation av specifika beteenden.

2. **Grundläggande mallar:** Abstrakta klasser kan fungera som grund för att skapa mallar eller ramverk med fördefinierad funktionalitet.
3. **Plugin-arkitektur:** Genom att använda abstrakta klasser kan du skapa en flexibel arkitektur för att lägga till och hantera plugins i din applikation.

## Kodexempel

Här är en implementation av en abstrakt klass för webbskrapning i Java:

```
import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.jsoup.nodes.Element;
import org.jsoup.select.Elements;

import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public abstract class WebScraper {
 private String url = "";
 private String title = "";
 private String description = "";
 private String tags = "";
 private Document htmlDocument = null;

 public String getUrl() {
 return url;
 }

 public void setUrl(String url) {
 this.url = url;
 }

 public String getTitle() {
 return title;
 }

 public void setTitle(String title) {
 this.title = title;
 }

 public String getDescription() {
 return description;
 }

 public void setDescription(String description) {
 this.description = description;
 }

 public String getTags() {
 return tags;
 }

 public void setTags(String tags) {
 this.tags = tags;
 }

 public Document getHtmlDocument() {
 return htmlDocument;
 }

 public void setHtmlDocument(Document htmlDocument) {
 this.htmlDocument = htmlDocument;
 }

 public String getHtml() {
 return htmlDocument != null ? htmlDocument.outerHtml() : "";
 }

 public String getText() {
 return htmlDocument != null ? htmlDocument.text() : "";
 }

 public String getHtmlDocument(String url) {
```

```

setUrl(url);
try {
htmlDocument = Jsoup.connect(url).get();
title = htmlDocument.title();
description =
 htmlDocument.select("meta[name=description]").attr("content");
tags = htmlDocument.select("meta[name=keywords]").attr("content");
} catch (IOException e) {
e.printStackTrace();
}
return getHtml();
}

public String getDivById(String id) {
Element div = htmlDocument != null ? htmlDocument.getElementById(id) :
 null;
return div != null ? div.outerHtml() :;
}

public List<String> getDivByClass(String className) {
Elements divs = htmlDocument != null ?
 htmlDocument.getElementsByClass(className) : null;
List<String> divHtmlList = new ArrayList<>();
if (divs != null) {
for (Element div : divs) {
divHtmlList.add(div.outerHtml());
}
}
return divHtmlList;
}

public List<String> getElementByClass(String element, String className)
{
Elements elements = htmlDocument != null ? htmlDocument.select(element +
 "." + className) : null;
List<String> elementHtmlList = new ArrayList<>();
if (elements != null) {
for (Element el : elements) {
elementHtmlList.add(el.outerHtml());
}
}
return elementHtmlList;
}

public List<String> getImages() {
List<String> imageUrls = new ArrayList<>();
if (htmlDocument != null) {
Elements images = htmlDocument.select("img");
for (Element image : images) {
String imageUrl = image.attr("src");
if (imageUrl.endsWith(".jpg") || imageUrl.endsWith(".png")) {
if (!imageUrl.startsWith("http:") && !imageUrl.startsWith("https:")) {
imageUrl = url.trim() + imageUrl;
}
imageUrls.add(imageUrl);
}
}
}
return imageUrls;
}

public abstract void scrape(String url);
}

```

Nu ska vi göra en implementation av denna klass för att hämta innehållet från en specifik sida.

```

import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.jsoup.nodes.Element;
import org.jsoup.select.Elements;

import java.io.IOException;
import java.util.List;

public class GetKittens extends WebScraper {
 public String

 downloadKitten() {
 scrape("https://www.pinterest.com/katesaidy/cute-kitten-pics/");
 String imgFolder = System.getProperty("user.home") + "/Pictures";
 String filename = imgFolder + "/Daily kitten.jpg";
 File file = new File(filename);
 if (file.exists()) {
 return filename;
 } else {
 return "";
 }
 }

 @Override
 public List<String> getImages() {
 List<String> imageUrls = super.getImages();

 // IMG SRC fungerade inte på Pinterest så vi söker med regex istället
 // För att göra det mer genomskinnligt så överridar vi bara GetImages
 if (getHtmlDocument() != null) {
 String html = getHtml();
 String regex = "(['\"])([^'\"]+\\.\\.(jpg|png|bmp|gif))\\1";
 Pattern pattern = Pattern.compile(regex);
 Matcher matcher = pattern.matcher(html);
 while (matcher.find()) {
 System.out.println(matcher.group(2));
 imageUrls.add(matcher.group(2));
 }
 }
 return imageUrls;
 }

 // Här är scrape-metoden som vi måste implementera
 @Override
 public void scrape(String url) {
 System.out.println("Downloading page");
 getHtmlDocument(url);
 System.out.println("Getting image list");
 List<String> images = getImages();
 System.out.println("Removing crap images");
 images.removeIf(pic -> pic.contains("/images/user/"));
 if (images.isEmpty()) {
 return;
 }

 // Hämtar en slumpmässig bild från listan
 Random random = new Random();
 System.out.println("Selecting random image");
 String randomImage = images.get(random.nextInt(images.size()));
 System.out.println("Downloading image");
 System.out.println(randomImage);
 try {
 URL imageUrl = new URL(randomImage);
 BufferedImage image = ImageIO.read(imageUrl);
 System.out.println("Saving image");
 String imgFolder = System.getProperty("user.home") + "/Pictures";
 String filename = imgFolder + "/Daily kitten.jpg";
 File file = new File(filename);
 ImageIO.write(image, "jpg", file);
 } catch (IOException e) {
 e.printStackTrace();
 }
 }
}

```

Jag hoppas att du gillar exemplet och att det är till hjälp för dig. Kattbilder är alltid en hit! Ha det roligt med webbscraping och experimentera med olika sidor att hämta innehåll från. Lycka till! ☐

## Termer

Här är en lista över några termer som används i koden:

- **WebScraper:** En abstrakt klass som hanterar webbscrapning och definierar olika metoder för att hämta och bearbeta webbsidor.
- **url:** En variabel som representerar adressen till den webbsida som ska webbscrapas.
- **title:** En variabel som representerar titeln på webbsidan.
- **description:** En variabel som representerar sidans egna beskrivning.
- **tags:** En variabel som representerar meta-nyckelord från webbsidan.
- **htmlDocument:** En variabel som representerar det HTML-dokument som hämtas från webbsidan.
- **html:** En metod som returnerar HTML-dokumentet som en sträng.
- **text:** En metod som returnerar all text på webbsidan utan HTML-taggar.
- **getHtmlDocument:** En metod som öppnar en webbsida och hämtar dess HTML-dokument.
- **getDivById:** En metod som hämtar innehållet i en div med en specifik id från webbsidan.
- **getDivByClass:** En metod som hämtar innehållet i alla divar med en specifik klass från webbsidan.
- **getElementByClass:** En metod som hämtar innehållet i alla element med en specifik klass från webbsidan.
- **getImages:** En metod som hämtar URL:er till alla bilder (IMG SRC-länkar) på webbsidan.
- **scrape:** En abstrakt metod som definierar hur webbsidan ska webbscrapas.
- **GetKittens:** En klass som ärver från **WebScraper** och implementerar **scrape**-metoden för att hämta kattbilder från en specifik sida.
- **Abstrakt klass:** En klass som inte kan instansieras direkt utan måste ärvas och implementeras i en konkret klass.
- **Virtuell metod:** En metod som kan överskridas i en subklass.
- **Paket:** En samling av Java-klasser och resurser som kan användas för att tillhandahålla specifik funktionalitet.
- **Grafik:** Bilder, ikoner och andra visuella element som används på en webbsida.
- **HTML-dokument:** En textfil som innehåller HTML-kod för att skapa en webbsida.
- **Överskrida:** Att skriva om en metod i en subklass.
- **Utöka:** Att lägga till funktionalitet i en subklass.
- **Klassimplementation:** En klass som ärver från en abstrakt klass och implementerar dess abstrakta metoder.
- **Återanvändbar kod:** Kod som kan återanvändas i olika delar av ett projekt.
- **Underhållbar kod:** Kod som är lätt att förstå och underhålla.

## Slutsats

I den här artikeln har vi utforskat användningen av abstrakta klasser och metoder i Java. Vi har sett hur man kan skapa en abstrakt klass med abstrakta och virtuella metoder för att hantera webbscrapning av olika sidor. Vi har också diskuterat användningen av JSoup för att underlätta hanteringen av HTML-dokument.

En abstrakt klass ger oss en bra grundstruktur för att implementera specifik funktionalitet och samtidigt möjliggöra flexibilitet genom att tillåta subklasser att överskrida och utöka funktionaliteten genom att implementera abstrakta metoder. Detta hjälper oss att skapa återanvändbar och underhållbar kod.

Vi har också sett en konkret implementation av den abstrakta klassen för att hämta kattbilder från en specifik sida. Genom att använda olika metoder som `getImages` och `getHtmlDocument`, kan vi hämta och bearbeta bilder från webbsidan.

Det är viktigt att nämna att det finns många andra sätt att använda abstrakta klasser och metoder i Java. Det är en kraftfull mekanism som kan användas för att skapa flexibla och modulära applikationer. Jag hoppas att du har lärt dig något nytt och att du kommer att använda denna kunskap i dina egna projekt. Lycka till! ☐

## Referenser

- JSoup
- Java
- Wikipedia

# Java för Jin23

Marcus Medina

delegater

## delegater

# Java för Jin23

Marcus Medina

Exempel

## Exempel

klass som innehåller en metod för att utföra en beräkning med hjälp av delegater. ||  
Addera | En metod som tar två heltal som parametrar och returnerar deras summa.  
|| Subtrahera | En metod som tar två heltal som parametrar och returnerar deras  
differens. || Multiplicera | En metod som tar två heltal som parametrar och returnerar  
deras produkt. || Dividera | En metod som tar två heltal som parametrar och  
returnerar deras kvot. |

En delegat i C# är en typ som representerar referenser till metoder med en specifik signatur. Delegater används ofta för att implementera händelsehantering och callback-funktioner. En delegat kan användas för att skapa en referens till en metod och sedan användas för att anropa den metoden. Här är ett exempel på hur man använder en delegat i C#:

```
// Definiera en delegat med namnet CalcParams som tar två heltal som
// parametrar och returnerar inget.
delegate void CalcParams(int a, int b);

public class Calculator {
 // Metod som utför en beräkning med hjälp av delegaten CalcParams.
 public void PerformCalculation(int a, int b, CalcParams calculation) {
 calculation(a, b);
 }

 public static void main(String[] args) {
 // Skapa en instans av Calculator-klassen.
 Calculator calculator = new Calculator();
 // Skapa en instans av delegaten CalcParams som refererar till metoden
 // Addera.
 CalcParams addDelegate = new CalcParams(calculator.Addera);
 // Anropa PerformCalculation-metoden och skicka med delegaten Addera.
 calculator.PerformCalculation(5, 3, addDelegate);
 }
}
```

I det här exemplet definierar vi en delegat med namnet CalcParams som tar två heltal som parametrar och returnerar inget. Sedan skapar vi en instans av Calculator-klassen och en instans av delegaten CalcParams som refererar till metoden Addera. Vi anropar sedan PerformCalculation-metoden på instansen av Calculator-klassen och skickar med delegaten Addera. När PerformCalculation-metoden anropas, kommer den att utföra beräkningen genom att anropa delegaten med de angivna parametrarna.

Metoden Addera adderar två heltal och skriver ut resultatet till konsolen. Detta är ett exempel på hur man använder en delegat i C#. I Java kan man använda liknande koncept med hjälp av gränssnitt och lambda-uttryck. Nedan följer ett exempel på hur man skulle implementera samma funktionalitet i Java:

```

// Definiera ett gränssnitt med namnet CalcParams som har en metod som
// tar två heltal som parametrar och returnerar inget.
interface CalcParams {
void calculate(int a, int b);

public class Calculator {
// Metod som utför en beräkning med hjälp av gränssnittet CalcParams.
public void performCalculation(int a, int b, CalcParams calculation) {
calculation.calculate(a, b);

public class Main {
public static void main(String[] args) {
// Skapa en Lambda-uttryck som implementerar gränssnittet CalcParams och
// adderar två heltal.
CalcParams addLambda = (int a, int b) -> {
int sum = a + b;
System.out.println("Summan av " + a + " och " + b + " är " + sum);
};
// Anropa performCalculation-metoden och skicka med Lambda-uttrycket.
calculator.performCalculation(5, 3, addLambda);
}
}

```

I det här exemplet definierar vi ett gränssnitt med namnet CalcParams som har en metod calculate som tar två heltal som parametrar och returnerar inget. Sedan skapar vi en instans av Calculator-klassen och en lambda-uttryck som implementerar gränssnittet CalcParams och adderar två heltal. Vi anropar sedan performCalculation-metoden på instansen av Calculator-klassen och skickar med lambda-uttrycket. När performCalculation-metoden anropas, kommer den att utföra beräkningen genom att anropa calculate-metoden på lambda-uttrycket med de angivna parametrarna.

Detta är ett exempel på hur man kan implementera samma funktionalitet med hjälp av gränssnitt och lambda-uttryck i Java.

# Java för Jin23

Marcus Medina

Events

## Events

I denna översikt kommer vi att utforska konceptet händelser i C#. Händelser är en funktion som låter oss reagera på händelser som inträffar under körningen av vårt program. ## Vad är händelser?

Händelser kan ses som en mekanism för att anropa metoder när något specifikt händer i programmet. Istället för att använda if-satser eller liknande för att kontrollera och reagera på olika scenarier kan vi definiera händelser som triggas när en händelse uppstår. En händelse kan vara något som att en knapp klickas, en fil ändras, en timer slår till eller ett objekt uppdateras. Genom att använda händelser kan vi separera logiken för att hantera händelsen från den del av koden som utlöser händelsen. ## Användningsområden för händelser

Händelser kan vara till nytta i en mängd olika scenarier, inklusive: 1. Grafiska användargränssnitt (GUI): I GUI-applikationer kan händelser användas för att hantera händelser som knappklickar, musinmatning eller fönsterfokusändringar. 2. Spelprogrammering: I spel kan händelser användas för att hantera händelser som spelarens interaktion, karaktärsdöd eller nivåuppggradering. 3. Multitrådad programmering: I flertrådade applikationer kan händelser användas för att synkronisera och kommunicera mellan olika trådar. 4. Kommunikation med externa enheter: Händelser kan användas för att hantera händelser som inträffar vid kommunikation med externa enheter eller system, t.ex. sensorer eller nätverksanslutningar. ## Kodexempel

För att ge dig en bättre förståelse för hur händelser fungerar i C# låt oss titta på ett exempel:

```
import java.util.ArrayList;
import java.util.List;
// Definierar en händelsehanterare
interface EventHandler {
 void handleEvent(String event);
}
// Definierar en händelsekälla
class EventSource {
 private List<EventHandler> eventHandlers = new ArrayList<>();
 // Registrerar en händelsehanterare
 public void registerEventHandler(EventHandler eventHandler) {
 eventHandlers.add(eventHandler);
 }
 // Utlöser händelsen
 public void triggerEvent(String event) {
 for (EventHandler eventHandler : eventHandlers) {
 eventHandler.handleEvent(event);
 }
 }
 // En exempelklass som implementerar EventHandler
 class ExampleEventHandler implements EventHandler {
 @Override
 public void handleEvent(String event) {
 System.out.println("Händelse mottagen: " + event);
 }
 }
 public class Main {
 public static void main(String[] args) {
 // Skapar en händelsekälla
 EventSource eventSource = new EventSource();
 // Skapar en händelsehanterare
 EventHandler eventHandler = new ExampleEventHandler();
 // Registrerar händelsehanteraren hos händelsekällan
 eventSource.registerEventHandler(eventHandler);
 // Utlöser en händelse
 eventSource.triggerEvent("En händelse har inträffat");
 }
 }
}
```

I detta exempel definierar vi en händelsehanterare genom att skapa ett gränssnitt `EventHandler` som har en metod `handleEvent`. Vi definierar också en händelsekälla `EventSource` som kan registrera händelsehanterare och utlösa händelser. Vi skapar en exempelklass `ExampleEventHandler` som implementerar `EventHandler` och skriver ut en meddelande när en händelse mottas. I `Main`-klassen skapar vi en instans av `EventSource`, en instans av `ExampleEventHandler` och registrerar händelsehanteraren hos händelsekällan. Sedan utlöser vi en händelse genom att anropa `triggerEvent`-metoden på händelsekällan. När händelsen utlöses kommer `ExampleEventHandler` att reagera genom att skriva ut ett meddelande. Detta är bara ett grundläggande exempel på hur händelser kan användas i Java. Det finns många andra möjligheter och användningsområden för händelser i Java-programmering.

```

````java import java.util.*; public class Hero { private
List<LevelUpListener> levelUpListeners = new ArrayList<>(); private
List<DiedListener> diedListeners = new ArrayList<>(); private int level
= 1; private int XP = 0; public int getLevel() { return level; public
int getXP() { return XP; public int getMaxXP() { return level * 100;
public void increaseXP(int amount) { XP += amount; if (XP >= getMaxXP())
{ level++; onLevelUp(); public void die() { onDied(); public void
addLevelUpListener(LevelUpListener listener) {
levelUpListeners.add(listener); public void addDiedListener(DiedListener
listener) { diedListeners.add(listener); private void onLevelUp() { for
(LevelUpListener listener : levelUpListeners) { listener.onLevelUp();
private void onDied() { for (DiedListener listener : diedListeners) {
listener.onDied(); public interface LevelUpListener { void onLevelUp();
public interface DiedListener { void onDied(); public class Program {
Hero hero = new Hero(); hero.addLevelUpListener(new LevelUpListener() {
@Override public void onLevelUp() { System.out.println("Hjälten har nått
en ny nivå!"); } }); hero.addDiedListener(new DiedListener() { public
void onDied() { System.out.println("Hjälten har dött!");
hero.increaseXP(100); hero.die(); I detta exempel har vi en klass som
heter Hero, som representerar en spelkaraktär. Klassen har två
listor, levelUpListeners och diedListeners, som innehåller lyssnare för
händelserna LevelUp och Died. När hjältens erfarenhetspoäng (XP) ökar,
kontrolleras om XP överstiger det maximala värdet (getMaxXP()). Om så
är fallet, ökar hjältens nivå och onLevelUp()-metoden anropas för varje
lyssnare i levelUpListeners. På samma sätt, när hjältens hälsopoäng (HP)
när 0 eller mindre, anropas onDied()-metoden för varje lyssnare
i diedListeners. I Program-klassen skapar vi en instans av Hero och lägger
till anonyma klasser som implementerar LevelUpListener och DiedListener
för att definiera beteendet för varje händelse. När vi sedan ökar hjältens XP eller
simulerar dess död, kommer de tillhörande metoderna att anropas och en
meddelandetext skrivs ut i konsolen. Detta är bara ett grundläggande exempel för
att illustrera hur händelselyssnare kan användas i Java. I praktiken kan de vara
mycket kraftfulla och användas för att skapa interaktion och dynamik i programmet.
## Summering

```

Händelselyssnare i Java möjliggör reaktioner på händelser under programkörningen. Genom att separera händelsehanteringen från den del av koden som utlöser händelsen, kan vi skapa mer modulära och underhållbara program. Händelselyssnare kan användas i olika scenarier, inklusive GUI-programmering, spelutveckling och multitrådad programmering. Genom att följa principerna om ren kod (Clean Code) kan vi skriva läsbar och effektiv kod som är lätt att förstå och underhålla.

Java för Jin23

Marcus Medina

Exempel

Exempel

Vi ska skapa ett bankkonto där vi kan sätta in pengar och ta ut pengar. Ett bankkonto är en vanlig komponent i finansiella system och används för att hantera insättningar, uttag och saldo. ## Beskrivning

Vårt bankkonto kommer att ha flera events som informerar oss om olika händelser som inträffar på kontot. Dessa events inkluderar: - Insättning: När pengar sätts in på kontot. - Uttag: När pengar tas ut från kontot. - Saldo mindre än 0: När saldot på kontot blir mindre än 0. - Insättning med felaktig summa: När en insättning görs med en ogiltig summa (t.ex. 0 eller negativt belopp). - Uttag med felaktig summa: När ett uttag görs med en ogiltig summa. Vi kommer också att ha en särskild händelse för insättningar över en viss gräns, till exempel 15000 kr. Denna händelse kan användas för att varna för eventuell kriminell aktivitet. För att implementera detta bankkonto använder vi Java och skapar en klass som heter "Account". Klassen har en variabel "balance" för att hålla reda på kontots saldo, samt olika events för att rapportera händelser till intresserade lyssnare. Här är Java-koden som implementerar bankkontot:

```
import java.util.ArrayList;
import java.util.List;
public class Account {
    private int balance;
    private List<DepositListener> depositListeners;
    private List<WithdrawListener> withdrawListeners;
    private List<BalanceBelowZeroListener> balanceBelowZeroListeners;
    private List<DepositInvalidAmountListener>
        depositInvalidAmountListeners;
    private List<WithdrawInvalidAmountListener>
        withdrawInvalidAmountListeners;
    private List<DepositAboveLimitListener> depositAboveLimitListeners;
    public Account() {
        balance = 0;
        depositListeners = new ArrayList<>();
        withdrawListeners = new ArrayList<>();
        balanceBelowZeroListeners = new ArrayList<>();
        depositInvalidAmountListeners = new ArrayList<>();
        withdrawInvalidAmountListeners = new ArrayList<>();
        depositAboveLimitListeners = new ArrayList<>();
    }
    public void addDepositListener(DepositListener listener) {
        depositListeners.add(listener);
    }
    public void addWithdrawListener(WithdrawListener listener) {
        withdrawListeners.add(listener);
    }
    public void addBalanceBelowZeroListener(BalanceBelowZeroListener
        listener) {
        balanceBelowZeroListeners.add(listener);
    }
    public void addDepositInvalidAmountListener(DepositInvalidAmountListener
        listener) {
        depositInvalidAmountListeners.add(listener);
    }
    public void
        addWithdrawInvalidAmountListener(WithdrawInvalidAmountListener
        listener) {
        withdrawInvalidAmountListeners.add(listener);
    }
    public void addDepositAboveLimitListener(DepositAboveLimitListener
        listener) {
        depositAboveLimitListeners.add(listener);
    }
    public int getBalance() {
        return balance;
    }
    public void deposit(int amount) {
        if (amount <= 0) {
```

```

for (DepositInvalidAmountListener listener :
    depositInvalidAmountListeners) {
listener.onDepositInvalidAmount(amount);
}
return;
}
if (amount > 15000) {
for (DepositAboveLimitListener listener : depositAboveLimitListeners) {
listener.onDepositAboveLimit(amount);
balance += amount;
for (DepositListener listener : depositListeners) {
listener.onDeposit(amount);
public void withdraw(int amount) {
for (WithdrawInvalidAmountListener listener :
    withdrawInvalidAmountListeners) {
listener.onWithdrawInvalidAmount(amount);
if (balance - amount < 0) {
for (BalanceBelowZeroListener listener : balanceBelowZeroListeners) {
listener.onBalanceBelowZero(amount);
balance -= amount;
for (WithdrawListener listener : withdrawListeners) {
listener.onWithdraw(amount);
}
public interface DepositListener {
void onDeposit(int amount);
public interface WithdrawListener {
void onWithdraw(int amount);
public interface BalanceBelowZeroListener {
void onBalanceBelowZero(int amount);
public interface DepositInvalidAmountListener {
void onDepositInvalidAmount(int amount);
public interface WithdrawInvalidAmountListener {
void onWithdrawInvalidAmount(int amount);
public interface DepositAboveLimitListener {
void onDepositAboveLimit(int amount);

```

```

public class DepositEventArgs { private int amount; public DepositEventArgs(int
amount) { this.amount = amount; public int getAmount() { return amount; public class
WithdrawEventArgs { public WithdrawEventArgs(int amount) { public class
BalanceBelowZeroEventArgs { public BalanceBelowZeroEventArgs(int amount) {
public class DepositInvalidAmountEventArgs { public
DepositInvalidAmountEventArgs(int amount) { public class
WithdrawInvalidAmountEventArgs { public WithdrawInvalidAmountEventArgs(int
amount) { public class DepositAboveLimitEventArgs { public
DepositAboveLimitEventArgs(int amount) { Nu kan vi använda vår "Account"-klass
för att skapa bankkonton och hantera händelserna som inträffar på dem.# Java
import java.util.EventListener; if (amount > 0) { balance += amount; } else {
DepositInvalidAmountEventArgs args = new
DepositInvalidAmountEventArgs(amount); onDepositInvalidAmount(args); if
(amount > 0 && amount <= balance) { balance -= amount;
WithdrawInvalidAmountEventArgs args = new
WithdrawInvalidAmountEventArgs(amount); onWithdrawInvalidAmount(args); if
(balance < 0) { BalanceBelowZeroEventArgs args = new
BalanceBelowZeroEventArgs(balance); onBalanceBelowZero(args); public event
EventHandler Deposit; protected virtual void OnDeposit(DepositEventArgs e) {
Deposit?.Invoke(this, e); public event EventHandler Withdraw; protected virtual void
OnWithdraw(WithdrawEventArgs e) Withdraw?.Invoke(this, e); public event
EventHandler BalanceBelowZero; protected virtual void
OnBalanceBelowZero(BalanceBelowZeroEventArgs e)
BalanceBelowZero?.Invoke(this, e); public event EventHandler
DepositInvalidAmount; protected virtual void
OnDepositInvalidAmount(DepositInvalidAmountEventArgs e)
DepositInvalidAmount?.Invoke(this, e); public event EventHandler
WithdrawInvalidAmount; protected virtual void
OnWithdrawInvalidAmount(WithdrawInvalidAmountEventArgs e)
WithdrawInvalidAmount?.Invoke(this, e); public event EventHandler
DepositAboveLimit; protected virtual void
OnDepositAboveLimit(DepositAboveLimitEventArgs e)
DepositAboveLimit?.Invoke(this, e); public class DepositEventArgs extends
EventArgs { public class WithdrawEventArgs extends EventArgs { public class
BalanceBelowZeroEventArgs extends EventArgs { public class
DepositInvalidAmountEventArgs extends EventArgs { public class
WithdrawInvalidAmountEventArgs extends EventArgs { public class
DepositAboveLimitEventArgs extends EventArgs { public class Program { public
static void main(String[] args) { Account account = new Account();
account.addDepositListener(new DepositListener() { @Override public void
onDeposit(DepositEventArgs e) { System.out.println("Insättning:" + e.getAmount());
}); account.addWithdrawListener(new WithdrawListener() { public void
onWithdraw(WithdrawEventArgs e) { System.out.println("Uttag:" + e.getAmount());
account.addBalanceBelowZeroListener(new BalanceBelowZeroListener() { public
void onBalanceBelowZero(BalanceBelowZeroEventArgs e) {
System.out.println("Saldo under noll:" + e.getAmount());
account.addDepositInvalidAmountListener(new DepositInvalidAmountListener() {
public void onDepositInvalidAmount(DepositInvalidAmountEventArgs e) {
System.out.println("Ogiltig insättningsbelopp:" + e.getAmount());
account.addWithdrawInvalidAmountListener(new WithdrawInvalidAmountListener()
{ public void onWithdrawInvalidAmount(WithdrawInvalidAmountEventArgs e) {
System.out.println("Ogiltig uttagsbelopp:" + e.getAmount());
account.addDepositAboveLimitListener(new DepositAboveLimitListener() { public
void onDepositAboveLimit(DepositAboveLimitEventArgs e) {
System.out.println("Insättning över gränsen:" + e.getAmount());
account.deposit(1000); account.withdraw(500); account.withdraw(600);
account.deposit(-100); public interface DepositListener extends EventListener {
void onDeposit(DepositEventArgs e); public interface WithdrawListener extends
EventListener { void onWithdraw(WithdrawEventArgs e); public interface
BalanceBelowZeroListener extends EventListener { void
onBalanceBelowZero(BalanceBelowZeroEventArgs e); public interface
DepositInvalidAmountListener extends EventListener { void
onDepositInvalidAmount(DepositInvalidAmountEventArgs e); public interface
WithdrawInvalidAmountListener extends EventListener { void
onWithdrawInvalidAmount(WithdrawInvalidAmountEventArgs e); public interface
DepositAboveLimitListener extends EventListener { void
onDepositAboveLimit(DepositAboveLimitEventArgs e);

```

'''I det här exemplet skapas en instans av klassen "Account" och olika händelselyssnare kopplas till dess events. Därefter utförs några operationer på kontot, som insättningar och uttag, vilket resulterar i att relevanta events triggas och meddelanden skrivs ut till konsolen. Detta är bara en grundläggande implementation av ett bankkonto i Java. Beroende på användningsområdet kan det finnas ytterligare funktioner och logik som behöver läggas till.

```

public class Main {
    public void onDepositAboveLimit(DepositAboveLimitEvent event) {
        System.out.println("Insättning på" + event.getAmount() + " kr är över gränsen");
        System.out.println("Ring SÄPO!");
    }
    public void onWithdrawInvalidAmount(WithdrawInvalidAmountEvent event) {
        System.out.println("Uttag på" + event.getAmount() + " kr är inte tillåtet");
    }
    public void onDepositInvalidAmount(DepositInvalidAmountEvent event) {
        System.out.println("Insättning på" + event.getAmount() + " kr är inte tillåtet");
    }
    public void onBalanceBelowZero(BalanceBelowZeroEvent event) {
        System.out.println("Uttag på" + event.getAmount() + " kr är inte tillåtet då det skulle sätta saldo under 0");
    }
    public void onWithdraw(WithdrawEvent event) {
        System.out.println("Uttag på" + event.getAmount() + " kr");
    }
    public void onDeposit(DepositEvent event) {
        System.out.println("Insättning på" + event.getAmount() + " kr");
        account.withdraw(-100);
        account.deposit(20000);
    }
    void onDepositAboveLimit(DepositAboveLimitEvent event);
    public class DepositAboveLimitEvent {
        private double amount;
        public DepositAboveLimitEvent(double amount) {
            public double getAmount() {
                void onWithdrawInvalidAmount(WithdrawInvalidAmountEvent event);
            }
        }
        public class WithdrawInvalidAmountEvent {
            public WithdrawInvalidAmountEvent(double amount) {
                void onDepositInvalidAmount(DepositInvalidAmountEvent event);
            }
        }
        public class DepositInvalidAmountEvent {
            public DepositInvalidAmountEvent(double amount) {
                void onBalanceBelowZero(BalanceBelowZeroEvent event);
            }
        }
        public class BalanceBelowZeroEvent {
            public BalanceBelowZeroEvent(double amount) {
                void onWithdraw(WithdrawEvent event);
            }
        }
        public class WithdrawEvent {
            public WithdrawEvent(double amount) {
                void onDeposit(DepositEvent event);
            }
        }
        public class DepositEvent {
            public DepositEvent(double amount) {
                public void withdraw(double amount) {
                    if (amount < 0) {
                        System.out.println("Uttag på" + amount + " kr är inte tillåtet");
                    }
                    WithdrawEvent event = new WithdrawEvent(amount);
                    listener.onWithdraw(event);
                }
                if (amount > 10000) {
                    listener.onDepositAboveLimit(new DepositAboveLimitEvent(amount));
                }
            }
        }
        public void deposit(double amount) {
            System.out.println("Insättning på" + amount + " kr är inte tillåtet");
            DepositEvent event = new DepositEvent(amount);
            listener.onDeposit(event);
        }
    }
}

```

Java för Jin23

Marcus Medina

Filhantering

- Skapa en klass
- Läsa en textfil
- Skriva till en textfil
- Läsa och skriva till en Json-fil
- Läsa och skriva binära filer
 - Läsa och skriva binära filer med Base64
- Läsa och skriva binära filer med Base64 och GZIP
- Övrig filhantering
 - Kolla om en fil finns
 - Skapa en tom fil om den inte finns
 - Skapa en mapp om den inte finns
- Radera en fil
- Radera en mapp om den är tom
- Flytta en fil
- Lista alla filer i en mapp
- Lista alla filer i en mapp och dess undermappar
- Hitta en fil som med namn som innehåller "Katt"
- Hitta en fil som med namn som innehåller "Hemlighet" och radera den
- Hitta en fil med texten "Lösenord" i sig
- Få information om filen
- Termer och förklaringar
- Sammanfattning

Filhantering

Vi ska skapa en klass och spara den i hårddisken, sedan ska vi läsa in den.

Skapa en klass

```
class Person{
    String namn;
    int ålder;

    public Person(String namn, int ålder){
        this.namn = namn;
        this.ålder = ålder;
    }

    public String toString(){
        return "Namn: " + namn + " Ålder:" + ålder;
    }

    public String getNamn(){
        return namn;
    }

    public int getÅlder(){
        return ålder;
    }

    public void setNamn(String namn){
        this.namn = namn;
    }

    public void setÅlder(int ålder){
        this.ålder = ålder;
    }
}
```

Läsa en textfil

För att läsa en textfil, använder vi `FileReader`-klassen. Vi skapar ett nytt objekt av klassen och ger den filnamnet som argument. Sedan kan vi använda `readLine()`-metoden för att läsa in rader från filen.

```
FileReader fr = new FileReader("filnamn.txt");
String rad = fr.readLine();
while(rad != null){
    System.out.println(rad);
    rad = fr.readLine();
}
```

Skriva till en textfil

För att skriva till en textfil, använder vi `FileWriter`-klassen. Vi skapar ett nytt objekt av klassen och ger den filnamnet som argument. Sedan kan vi använda `write()`-metoden för att skriva till filen.

```
// Skapa en person
Person p = new Person("Luke Skywalker", 23);

// Serialisera personen
String person = p.toString();

// Skapa en ny fil och skriv personen till den
FileWriter fw = new FileWriter("person.txt");
fw.write(person);
fw.close();

// Läs in personen från filen
FileReader fr = new FileReader("person.txt");
String rad = fr.readLine();
ft.close();

// Skapa en ny person från strängen
// genom att dela upp den på Namn: och Ålder:
String[] delar = rad.split("Namn: | Ålder:");
Person p2 = new Person(delar[1], Integer.parseInt(delar[2]));
System.out.println(p2);
```

Läsa och skriva till en Json-fil

För att läsa och skriva till en Json-fil, använder vi `Gson`-klassen. Vi skapar ett nytt objekt av klassen och ger den filnamnet som argument. Sedan kan vi använda `toJson()`-metoden för att skriva till filen och `fromJson()`-metoden för att läsa från filen.

Lägg till `Gson`-biblioteket i Maven:

```
<dependency>
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
  <version>2.8.9</version>
</dependency>
```

```
// Skapa en person
Person p = new Person("Leia Skywalker", 23);

// Serialisera personen
Gson gson = new Gson();
String person = gson.toJson(p);

// Skapa en ny fil och skriv personen till den
FileWriter fw = new FileWriter("person.json");
fw.write(person);
fw.close();

// Läs in personen från filen
FileReader fr = new FileReader("person.json");
Person p2 = gson.fromJson(fr, Person.class);
System.out.println(p2);
```

Läsa och skriva binära filer

För att läsa och skriva binära filer, använder vi `FileInputStream` och `FileOutputStream`-klasserna. Vi skapar ett nytt objekt av klassen och ger den filnamnet som argument. Sedan kan vi använda `read()` och `write()`-metoderna för att läsa och skriva till filen.

```
// Skapa en person
Person p = new Person("Han Solo", 33);

// Serialisera personen
ByteArrayOutputStream baos = new ByteArrayOutputStream();
ObjectOutputStream oos = new ObjectOutputStream(baos);
oos.writeObject(p);

// Skapa en ny fil och skriv personen till den
FileOutputStream fos = new FileOutputStream("person.bin");
fos.write(baos.toByteArray());
fos.close();

// Läs in personen från filen
FileInputStream fis = new FileInputStream("person.bin");
ObjectInputStream ois = new ObjectInputStream(fis);
Person p2 = (Person) ois.readObject();
System.out.println(p2);
```

Läsa och skriva binära filer med Base64

För att läsa och skriva binära filer med Base64, använder vi `Base64`-klassen. Vi skapar ett nytt objekt av klassen och ger den filnamnet som argument. Sedan kan vi använda `encode()` och `decode()`-metoderna för att läsa och skriva till filen.

```
// Skapa en person
Person p = new Person("Chewbacca", 53);

// Serialisera personen
ByteArrayOutputStream baos = new ByteArrayOutputStream();
ObjectOutputStream oos = new ObjectOutputStream(baos);
oos.writeObject(p);

// Skapa en ny fil och skriv personen till den
FileOutputStream fos = new FileOutputStream("person.bin");
fos.write(Base64.getEncoder().encode(baos.toByteArray()));
fos.close();

// Läs in personen från filen
FileInputStream fis = new FileInputStream("person.bin");
ObjectInputStream ois = new ObjectInputStream(fis);
Person p2 = (Person) ois.readObject();
System.out.println(p2);
```

Läsa och skriva binära filer med Base64 och GZIP

För att läsa och skriva binära filer med Base64 och GZIP, använder vi Base64-klassen och GZIPOutputStream-klassen. Vi skapar ett nytt objekt av klassen och ger den filnamnet som argument. Sedan kan vi använda encode() och decode()-metoderna för att läsa och skriva till filen.

```
// Skapa en person
Person p = new Person("Obi-Wan Kenobi", 53);

// Serialisera personen
ByteArrayOutputStream baos = new ByteArrayOutputStream();
ObjectOutputStream oos = new ObjectOutputStream(baos);
oos.writeObject(p);

// Skapa en ny fil och skriv personen till den
FileOutputStream fos = new FileOutputStream("person.gzis");
GZIPOutputStream gzos = new GZIPOutputStream(fos);
gos.write(Base64.getEncoder().encode(baos.toByteArray()));
gos.close();

// Läs in personen från filen
FileInputStream fis = new FileInputStream("person.gzis");
GZIPInputStream gzis = new GZIPInputStream(fis);
ObjectInputStream ois = new ObjectInputStream(gzis);
Person p2 = (Person) ois.readObject();
System.out.println(p2);
```

Övrig filhantering

Kolla om en fil finns

```
File f = new File("person.txt");
if(f.exists()){
    System.out.println("Filen finns");
}
```

Skapa en tom fil om den inte finns

```
File f = new File("person.txt");
if(!f.exists()){
    f.createNewFile();
}
```

Skapa en mapp om den inte finns

```
File f = new File("mapp");
if(!f.exists()){
    f.mkdir();
}
```

Radera en fil

```
File f = new File("person.txt");
if(f.exists()){
    f.delete();
}
```

Radera en mapp om den är tom

```
File f = new File("mapp");
if(f.exists()){
    if (f.isDirectory() && f.list().length == 0)
        f.delete();
}
```

Flytta en fil

```
File f = new File("person.txt");
if(f.exists()){
    f.renameTo(new File("person2.txt"));
}
```

Lista alla filer i en mapp

```
File f = new File("mapp");
if(f.exists()){
    File[] files = f.listFiles();
    for(File file : files){
        System.out.println(file.getName());
    }
}
```

Lista alla filer i en mapp och dess undermappar

```
File f = new File("mapp");

if(f.exists()){
    File[] files = f.listFiles();
    for(File file : files){
        if(file.isDirectory()){
            File[] subFiles = file.listFiles();
            for(File subFile : subFiles){
                System.out.println(subFile.getName());
            }
        }else{
            System.out.println(file.getName());
        }
    }
}
```

Hitta en fil som med namn som innehåller “Katt”

```
File f = new File("mapp");
if(f.exists()){
    File[] files = f.listFiles();
    for(File file : files){
        if(file.getName().contains("Katt")){
            System.out.println(file.getName());
        }
    }
}
```

Hitta en fil som med namn som innehåller “Hemlighet” och radera den

```
File f = new File("mapp");
if(f.exists()){
    File[] files = f.listFiles();
    for(File file : files){
        if(file.getName().contains("Hemlighet")){
            file.delete();
        }
    }
}
```

Hitta en fil med texten “Lösenord” i sig

```
File f = new File("mapp");

if(f.exists()){
    File[] files = f.listFiles();
    for(File file : files){
        if(file.isFile()){
            FileReader fr = new FileReader(file);
            String rad = fr.readLine();
            while(rad != null){
                if(rad.contains("Lösenord")){
                    System.out.println(file.getName());
                }
                rad = fr.readLine();
            }
        }
    }
}
```

Få information om filen

```
File f = new File("person.txt");

if(f.exists()){
    System.out.println("Filnamn: " + f.getName());
    System.out.println("Filtyp: " + f.getType());
    System.out.println("Mapp: " + f.getParent());
    System.out.println("Sökväg: " + f.getAbsolutePath());
    System.out.println("Storlek: " + f.length());
    System.out.println("Senast ändrad: " + f.lastModified());
    System.out.println("Är fil: " + f.isFile());
    System.out.println("Är mapp: " + f.isDirectory());
    System.out.println("Är dold: " + f.isHidden());
    System.out.println("Är läsbar: " + f.canRead());
    System.out.println("Är skrivbar: " + f.canWrite());
    System.out.println("Är exekverbar: " + f.canExecute());
    System.out.println("Är samma fil: " + f.equals(new
        File("person.txt")));
}
```

Termer och förklaringar

Här följer termer som använts i denna artikel och deras förklaringar.

Term	Förklaring
FileReader	En klass som används för att läsa textfiler.
FileWriter	En klass som används för att skriva till textfiler.
Gson	En klass som används för att läsa och skriva till Json-filer.
Base64	En klass som används för att läsa och skriva binära filer.
GZIPOutputStream	En klass som används för att läsa och skriva binära filer med Base64 och GZIP.
ObjectOutputStream	En klass som används för att läsa och skriva binära filer.
ObjectInputStream	En klass som används för att läsa och skriva binära filer.
ByteArrayOutputStream	En klass som används för att läsa och skriva binära filer.
FileInputStream	En klass som används för att läsa och skriva binära filer.
FileOutputStream	En klass som används för att läsa och skriva binära filer.

Term	Förklaring
GZIPInputStream	En klass som används för att läsa och skriva binära filer med Base64 och GZIP.
readLine()	En metod som används för att läsa en rad från en textfil.
write()	En metod som används för att skriva till en textfil.
toJson()	En metod som används för att skriva till en Json-fil.
fromJson()	En metod som används för att läsa från en Json-fil.
read()	En metod som används för att läsa från en binär fil.
write()	En metod som används för att skriva till en binär fil.
encode()	En metod som används för att läsa från en binär fil med Base64.
decode()	En metod som används för att skriva till en binär fil med Base64.
close()	En metod som används för att stänga en fil.
split()	En metod som används för att dela upp en sträng i en array.
toString()	En metod som används för att konvertera ett objekt till en sträng.
getNamn()	En metod som används för att hämta namnet på en person.
getÅlder()	En metod som används för att hämta åldern på en person.
setNamn()	En metod som används för att sätta namnet på en person.
setÅlder()	En metod som används för att sätta åldern på en person.
Person	En klass som används för att representera en person.
namn	En variabel som används för att lagra namnet på en person.
ålder	En variabel som används för att lagra åldern på en person.
p	En variabel som används för att representera en person.
p2	En variabel som används för att representera en person.
rad	En variabel som används för att representera en rad i en textfil.
delar	En variabel som används för att representera en array av strängar.
fr	En variabel som används för att representera en textfil.
fw	En variabel som används för att representera en textfil.
gson	En variabel som används för att representera en Json-fil.
baos	En variabel som används för att representera en binär fil.
oos	En variabel som används för att representera en binär fil.

Term	Förklaring
------	------------

fos	En variabel som används för att representera en binär fil.
fis	En variabel som används för att representera en binär fil.
gzos	En variabel som används för att representera en binär fil med Base64 och GZIP.
gzis	En variabel som används för att representera en binär fil med Base64 och GZIP.
ois	En variabel som används för att representera en binär fil.
fr	En variabel som används för att representera en textfil.
Serialize	Att konvertera ett objekt till en sträng.
Deserialize	Att konvertera en sträng till ett objekt.

Sammanfattning

Det finns många olika sätt att spara information på, välj den som känns trevligast för dig helt enkelt.

Java för Jin23

Marcus Medina

CSV

- Fördelar
- Begränsningar
- Användningsområden
- Kodexempel
 - Läsning av CSV-fil
 - Skrivning till CSV-fil
- Kodförklaringar
 - Files.readAllLines
 - String.split
 - StringBuilder
 - StringBuilder.append
 - Files.write
- Slutsats
- Termer
- Länkar

CSV

Filformatet CSV (Comma Separated Values) är ett filformat som används för att lagra data i en tabell. Det är ett vanligt filformat som används för att lagra data i Excel. Det är ett textbaserat filformat som använder kommatecken för att separera värdena i en rad. CSV-filer kan enkelt läsas och skrivas med hjälp av Java-kod.

▼ Innehållsförteckning

1. TOC

Fördelar

CSV-filformatet har flera fördelar när det gäller att lagra och hantera data:

- **Enkelhet:** CSV-filer är lätta att skapa och förstå. Eftersom det är ett textbaserat format kan det enkelt redigeras med hjälp av en textredigerare eller ett kalkylbladsprogram.
- **Kompatibilitet:** CSV-filer kan läsas och skrivas av de flesta programvaror och programmeringsspråk. Detta gör det enkelt att dela och utbyta data mellan olika system.
- **Effektivitet:** CSV-filer är kompakta eftersom de använder en enkel textrepresentation för att lagra data. Detta minimerar filstorleken och gör det snabbt att läsa och skriva data.
- **Tabellstruktur:** CSV-filer behåller tabellstrukturen, vilket gör det enkelt att representera relationella data. Varje rad i filen motsvarar en post i tabellen, och varje kolumn motsvarar en attribut eller en egenskap för posten.

Begränsningar

Trots sina fördelar har CSV-filer vissa begränsningar att vara medveten om:

- **Ingen standard:** Det finns ingen officiell standard för CSV-filformatet, vilket kan leda till inkompatibiliteter mellan olika implementationer. Det är viktigt att vara medveten om vilka regler och konventioner som används för att skapa och tolka CSV-filer.
- **Begränsad datatypsstöd:** CSV-filer hanterar bara textbaserade data. Om du behöver lagra mer komplexa datatyper som datum, tid eller binära data kan det kräva extra ansträngning för att konvertera och tolka dessa värden korrekt.

- **Bristar i struktur:** CSV-filer saknar en inbyggd mekanism för att beskriva datastrukturen. Det innebär att det kan vara svårt att veta vad varje kolumn representerar om det inte finns en tydlig dokumentation eller överenskommelse om filens innehåll.

Användningsområden

CSV-filer används inom olika områden och har många användningsområden:

- **Dataimport och -export:** CSV-filer används ofta för att importera och exportera data från olika program och system. Det kan vara användbart när du behöver överföra data mellan olika databaser, kalkylblad eller applikationer.
- **Dataanalys:** CSV-filer är vanliga inom dataanalys och affärsintelligens. Genom att exportera data från olika källor till CSV-format kan du samla och analysera informationen med hjälp av specialiserade analysverktyg eller skript.
- **Testdata:** CSV-filer kan användas för att skapa testdata för programvarutestning. Genom att generera CSV-filer med olika scenarier och värden kan du testa programmet med olika dataset och se till att det fungerar korrekt i olika situationer.
- **Konfigurationsfiler:** CSV-filer kan användas som en enkel form av konfigurationsfiler. Du kan använda dem för att lagra och läsa in inställningar eller parametrar som behövs för att konfigurera en applikation eller ett system.

Kodexempel

Här är några kodexempel som visar hur du kan läsa och skriva CSV-filer i Java:

Läsning av CSV-fil

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class CSVReader {
    public static void main(String[] args) {
        String file = "myfile.csv";

        try (BufferedReader br = new BufferedReader(new
            FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                String[] values = line.split(",");
                for (String value : values) {
                    System.out.print(value + " ");
                }
                System.out.println();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

I det här exemplet används `BufferedReader` och `FileReader` för att läsa in CSV-filen rad för rad. Sedan används `String.split` för att dela upp varje rad i separata värden baserat på kommatecken. Till sist skrivs varje värde ut på konsolen.

Skrivning till CSV-fil

```

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.util.ArrayList;
import java.util.List;

public class CSVWriter {
    public static void main(String[] args) {
        List<Person> people = new ArrayList<>();
        people.add(new Person(1, "Clark Kent", "Superman", 35));
        people.add(new Person(2, "Bruce Wayne", "Batman", 40));
        people.add(new Person(3, "Barry Allen", "Flash", 25));

        try (BufferedWriter bw = new BufferedWriter(new
            FileWriter(file))) {
            // Lägg till rubriker
            bw.write("Id,Namn,Alias,Ålder");
            bw.newLine();
            // Lägg till data
            for (Person person : people) {
                bw.write(person.getId() + "," + person.getName() + "," +
                    person.getAlias() + "," + person.getAge());
                bw.newLine();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

class Person {
    private int id;
    private String name;
    private String alias;
    private int age;

    public Person(int id, String name, String alias, int age) {
        this.id = id;
        this.name = name;
        this.alias = alias;
        this.age = age;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getAlias() {
        return alias;
    }

    public int getAge() {
        return age;
    }
}

```

I det här exemplet används `BufferedWriter` och `FileWriter` för att skriva data till CSV-filen. Först läggs rubrikerna till med `bw.write` och `bw.newLine`. Sedan läggs varje person i listan till med `bw.write` och `bw.newLine`.

I detta exempel används en lista av `Person`-objekt för att hålla data. Sedan används `StringBuilder` för att bygga upp innehållet i CSV-filen. Till sist skrivs strängen till filen med `File.WriteAllText`.

Kodförklaringar

Files.readAllLines

`Files.readAllLines` läser in alla rader i en fil och lägger dem i en lista. Varje rad blir ett element i listan.

String.split

`String.split` delar upp en sträng i en lista av strängar. Den delar upp strängen vid varje komma.

StringBuilder

`StringBuilder` är en klass som används för att bygga upp en sträng. Det är en effektivare metod än att använda strängkonkatenation. Det är en klass som finns i `java.lang`.

StringBuilder.append

`StringBuilder.append` lägger till en rad till strängen. Den lägger till en radbrytning efter raden.

Files.write

`Files.write` skriver en sträng till en fil. Den skriver över allt som finns i filen.

Slutsats

CSV-filformatet är ett enkelt och flexibelt sätt att lagra och hantera tabulära data. Det är vanligt förekommande inom olika områden, inklusive datahantering, analys och testning. Med hjälp av Java kan du enkelt läsa och skriva CSV-filer och manipulera data enligt dina behov. Var medveten om dess begränsningar och regler för att få ut mesta möjliga av detta filformat.

Termer

Term	Förklaring
CSV	Comma Separated Values
Filformat	Ett sätt att lagra data i en fil
Textbaserat	Ett filformat som använder text för att representera data
Kompatibilitet	Förmågan att fungera tillsammans med andra system
Prestanda	Hur snabbt eller effektivt ett system fungerar
Datatyp	En typ av data som kan lagras i en variabel
Binär	Ett filformat som använder binära tal för att representera data
Datastruktur	Ett sätt att organisera data i en fil
Dataanalys	Att analysera data för att hitta mönster och trender
Affärsintelligens	Att använda data för att fatta beslut och förbättra verksamheten
Testning	Att testa ett program för att hitta fel och problem
Konfigurationsfil	En fil som innehåller inställningar och parametrar för ett program
Inställningar	Parametrar som används för att konfigurera ett program

Länkar

- CSV-filformatet
- CSV-filformatet i Java
- Google Kalender
- Outlook Kalender
- Google Kalender CSV

Java för Jin23

Marcus Medina

File-klassen

TL;DR

Beskrivning

Exempel

Slutsats

Termer

File-klassen

File-klassen i Java, som tillhör paketet `java.io`, erbjuder en mängd användbara metoder för att hantera filer. Du kan använda dessa metoder för att skapa, skriva, läsa och ta bort filer. Exempel på några av de vanligaste metoderna inkluderar `createNewFile`, `delete`, `exists` och `read`. Genom att använda File-klassen kan du enkelt utföra filrelaterade operationer i dina Java-program.

▼ Innehållsförteckning

TL;DR

File-klassen i Java, som tillhör paketet `java.io`, erbjuder användbara metoder för filhantering. Du kan använda dessa metoder för att skapa, skriva, läsa och ta bort filer. Några av de vanligaste metoderna inkluderar `createNewFile`, `delete`, `exists` och `renameTo`. Genom att använda File-klassen kan du enkelt utföra filrelaterade operationer i dina Java-program.

Beskrivning

File-klassen innehåller ett brett utbud av användbara metoder för filhantering. Här är några av de vanligaste metoderna och deras funktioner:

- `createNewFile`: Skapar en ny fil på den angivna sökvägen.
- `delete`: Tar bort en befintlig fil.
- `exists`: Kontrollerar om en fil existerar.
- `renameTo`: Byter namn på en fil.
- `length`: Returnerar storleken på en fil i bytes.
- `canRead`: Kontrollerar om en fil kan läsas.
- `canWrite`: Kontrollerar om en fil kan skrivas till.
- `canExecute`: Kontrollerar om en fil kan köras som ett program.
- `isFile`: Kontrollerar om en fil är en vanlig fil.
- `isDirectory`: Kontrollerar om en fil är en mapp.
- `getParent`: Returnerar sökvägen till filens överordnade mapp.
- `getName`: Returnerar filens namn.
- `getPath`: Returnerar filens sökväg.
- `lastModified`: Returnerar tidpunkten för senaste ändring av en fil.
- `list`: Returnerar en array av filer och mappar i en mapp.
- `mkdir`: Skapar en ny mapp på den angivna sökvägen.
- `listFiles`: Returnerar en array av `File`-objekt som representerar filer och mappar i en mapp.
- `FileReader`: Öppnar en befintlig textfil för läsning.
- `FileWriter`: Öppnar en fil för skrivning.
- `Files.readAllBytes`: Läser innehållet i en binär fil som en byte-array.
- `Files.readAllLines`: Läser innehållet i en textfil och returnerar det som en array av strängar.
- `Files.readAllText`: Läser innehållet i en textfil och returnerar det som en sträng.
- `Files.lines`: Läser innehållet i en textfil och returnerar det som en uppräkningsbar sekvens av strängar.

- `Files.writeAllBytes`: Skriver en byte-array till en fil.
- `Files.writeAllLines`: Skriver en array av strängar till en textfil, en sträng per rad.
- `Files.writeAllText`: Skriver en sträng till en textfil.

Dessa metoder ger oss flexibilitet att utföra olika åtgärder på filer, inklusive skapande, läsning, skrivning, flyttning och radering.

Exempel

Här är några kodexempel som visar hur man använder några av metoderna i `File`-klassen:

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

public class Main {
    public static void main(String[] args) {
        String file = "C:\\Temp\\test.txt";

        // Skapar en fil
        try {
            if (!Files.exists(Paths.get(file))) {
                Files.createFile(Paths.get(file));
            }
        } catch (IOException e) {
            e.printStackTrace();
        }

        // Skriver till en fil
        try {
            Files.write(Paths.get(file), "Hello World!".getBytes());
        } catch (IOException e) {
            e.printStackTrace();
        }

        // Läser innehållet i en fil
        try {
            String text = Files.readString(Paths.get(file));
            System.out.println(text);
        } catch (IOException e) {
            e.printStackTrace();
        }

        // Tar bort en fil
        try {
            Files.delete(Paths.get(file));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

I detta exempel skapar vi en fil på sökvägen "C:\Temp\test.txt" om den inte redan finns. Sedan använder vi metoden `Files.write` för att skriva texten "Hello World!" till filen. Vi använder `Files.readString` för att läsa in innehållet i filen och skriva ut det till konsolen. Slutligen tar vi bort filen med `Files.delete`.

Det finns många fler metoder i `File`-klassen som kan vara användbara beroende på dina specifika behov. Genom att utforska dokumentationen för `File`-klassen kan du lära dig mer om var och en av metoderna och hur de kan användas.

Slutsats

`File`-klassen i Java erbjuder enkla och kraftfulla metoder för att hantera filer. Oavsett om du behöver skapa, skriva, läsa eller ta bort filer finns det en metod i `File`-klassen som kan hjälpa dig. Genom att använda de olika metoderna kan du effektivt arbeta med filsystemet i dina Java-applikationer.

Termer

- **File:** En klass i `java.io`-paketet som innehåller metoder för att hantera filer.
- **appendText:** En metod i File-klassen som öppnar en befintlig fil för skrivning och lägger till text i slutet av filen.
- **copy:** En metod i File-klassen som kopierar en fil till en ny plats.
- **create:** En metod i File-klassen som skapar en ny fil.
- **delete:** En metod i File-klassen som tar bort en befintlig fil.
- **exists:** En metod i File-klassen som kontrollerar om en fil existerar.
- **move:** En metod i File-klassen som flyttar en fil till en ny plats.
- **open:** En metod i Java File-klassen som öppnar en fil i en `FileInputStream`.
- **openRead:** Det finns ingen direkt motsvarighet till File-klassens `OpenRead`-metod i Java. Istället kan du använda `FileInputStream`-klassen för att öppna en fil för läsning.
- **openText:** Det finns ingen direkt motsvarighet till File-klassens `OpenText`-metod i Java. Istället kan du använda `FileReader`-klassen för att öppna en befintlig textfil för läsning.
- **openWrite:** En metod i Java File-klassen som öppnar en fil för skrivning med hjälp av `FileOutputStream`.
- **readAllBytes:** En metod i Java File-klassen som läser innehållet i en binär fil som en byte-array med hjälp av `FileInputStream`.
- **readAllLines:** En metod i Java File-klassen som läser innehållet i en textfil och returnerar det som en array av strängar. Du kan använda `BufferedReader`-klassen tillsammans med `FileReader`-klassen för att uppnå detta.
- **readAllText:** En metod i Java File-klassen som läser innehållet i en textfil och returnerar det som en sträng med hjälp av `FileReader`.
- **readLines:** En metod i Java File-klassen som läser innehållet i en textfil och returnerar det som en uppräkningsbar sekvens av strängar med hjälp av `BufferedReader` och `FileReader`.
- **writeAllBytes:** En metod i Java File-klassen som skriver en byte-array till en fil med hjälp av `FileOutputStream`.
- **writeAllLines:** En metod i Java File-klassen som skriver en lista av strängar till en textfil, en sträng per rad. Du kan använda `BufferedWriter` tillsammans med `FileWriter`-klassen för att uppnå detta.
- **writeAllText:** En metod i Java File-klassen som skriver en sträng till en textfil.

Java för Jin23

Marcus Medina

Json

- Gson-biblioteket
- Skapa JSON-manuellt med StringBuilder
- Läsa JSON-data
- Fördelar och nackdelar med JSON
 - Fördelar:
 - Nackdelar:
- Varför använda Gson istället för org.json?
- Termer och förklaringar
- Obligatorisk Dad Joke

Json

JSON är ett textbaserat filformat som används för att lagra och överföra data på ett strukturerat sätt. Det är enklare att läsa och skriva än XML och används ofta inom webbapplikationer. JSON används för att representera data i form av objekt och arrayer, och det är ett populärt filformat som har stöd i olika programmeringsspråk och plattformar.

Gson-biblioteket

I Java finns det inbyggda verktyg och bibliotek för att arbeta med JSON-data. En av de mest populära biblioteken är Gson, som tillhandahåller funktioner för att serialisera och deserialisera JSON-data till Java-objekt och vice versa. Gson gör det enkelt att konvertera en JSON-sträng till ett Java-objekt och att generera en JSON-sträng från ett Java-objekt.

För att använda Gson-biblioteket behöver du lägga till beroendet i ditt projekt. Du kan göra det genom att lägga till följande kod i din pom.xml-fil om du använder Maven:

```
<dependencies>
  <dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.8.7</version>
  </dependency>
</dependencies>
```

När du har lagt till beroendet kan du använda Gson för att serialisera och deserialisera JSON-data. Här är några exempel på hur du kan använda Gson:

```

import com.google.gson.Gson;

public class Main {
    public static void main(String[] args) {
        // Skapa en instans av Gson
        Gson gson = new Gson();

        // Serialisera ett Java-objekt till en JSON-sträng
        Person person = new Person("John Doe", 30);
        String json = gson.toJson(person);
        System.out.println(json); // {"name":"John Doe","age":30}

        // Deserialisera en JSON-sträng till ett Java-objekt
        String json2 = "{\"name\":\"Jane Smith\",\"age\":25}";
        Person person2 = gson.fromJson(json2, Person.class);
        System.out.println(person2.getName()); // Jane Smith
        System.out.println(person2.getAge()); // 25
    }
}

class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }
}

```

I exemplet ovan använder vi Gson för att serialisera en Person-objekt till en JSON-sträng och deserialisera en JSON-sträng till ett Person-objekt. Gson använder Java Reflection för att konvertera objekt mellan Java och JSON och det gör det enkelt att hantera komplexa datastrukturer.

Skapa JSON-manuellt med StringBuilder

Förutom att använda bibliotek kan vi också skapa en JSON-sträng manuellt med hjälp av StringBuilder-klassen. Det här kan vara användbart om du behöver bygga en JSON-sträng dynamiskt eller om du inte vill använda ett tredjepartsbibliotek för att hantera JSON.

Här är ett exempel på hur du kan använda StringBuilder för att skapa en JSON-sträng som representerar en Star Wars-karaktär:

```

StringBuilder sb = new StringBuilder();
sb.append("{");
sb.append("\"name\": \"Luke Skywalker\",");
sb.append("\"age\": 30");
sb.append("}");
String json = sb.toString();

```

I det här exemplet skapar vi en StringBuilder-instans och använder dess append()-metod för att bygga upp JSON-strängen steg för steg. Till slut använder vi toString()-metoden för att få den färdiga JSON-strängen.

Läsa JSON-data

För att läsa JSON-data kan du använda Gson-biblioteket eller så kan du använda Java's inbyggda verktyg för JSON-hantering. Här är ett exempel på hur du kan läsa JSON-data utan att använda Gson:

```
import javax.json.Json;
import javax.json.JsonObject;
import javax.json.JsonReader;
import java.io.StringReader;

String json = "{\"name\":\"Luke Skywalker\",\"age\":30}";

JsonReader reader = Json.createReader(new StringReader(json));
JsonObject obj = reader.readObject();
String name = obj.getString("name");
int age = obj.getInt("age");

System.out.println(name); // Luke Skywalker
System.out.println(age); // 30
```

I det här exemplet använder vi `javax.json`-paketet för att läsa JSON-data. Vi skapar en `JsonReader`-instans och använder den för att läsa in JSON-data från en sträng. Sedan använder vi `JsonObject` för att få tillgång till de olika egenskaperna i JSON-objektet.

Fördelar och nackdelar med JSON

Nu när du har lärt dig grunderna i JSON, låt oss titta på några fördelar och nackdelar med att använda JSON:

Fördelar:

- JSON är lätt att läsa och skriva för både människor och maskiner.
- Det är ett populärt filformat och har stort stöd i olika programmeringsspråk och plattformar.
- JSON-data kan enkelt konverteras till objekt och vice versa, vilket gör det lätt att arbeta med.

Nackdelar:

- JSON saknar några av de mer avancerade funktionerna som XML erbjuder.
- Det kan vara mindre lämpligt för strukturerade dokument med hierarkiska datastrukturer.
- JSON kan bli svårt att hantera om filerna blir mycket stora.

Varför använda Gson istället för org.json?

Du kanske undrar varför jag rekommenderar Gson istället för `org.json`. Det finns flera skäl till detta:

- Gson har en mer intuitiv och enkel API-design.
- Gson har bättre prestanda och lägre minnesanvändning än `org.json`.
- Gson har stöd för avancerad typomvandling och anpassning av serialisering/deserialisering.
- Gson har en aktiv och stor användarbas och har mer dokumentation och exempel att använda som referens.

Termer och förklaringar

Här är en tabell över termer som använts i denna artikel och deras förklaringar:

Term	Förklaring
JSON	JavaScript Object Notation, ett textbaserat filformat för att lagra och transportera data
Serialisera	Konvertera Java-objekt till JSON-format
Deserialisera	Konvertera JSON till Java-objekt
Gson	Ett populärt bibliotek för JSON-hantering i Java
StringBuilder	En klass i Java för att bygga upp textsträngar dynamiskt
javax.json	Ett paket i Java för JSON-hantering
org.json	Ett bibliotek för JSON-hantering i Java

För att fördjupa dina kunskaper och utforska mer om JSON och Java-programmering rekommenderar jag följande resurser:

- [JSON.org](https://www.json.org/): Den officiella webbplatsen för JSON med massor av information och exempel.
- [Gson Dokumentation](#): Officiell dokumentation för Gson-biblioteket, ett populärt bibliotek för att hantera JSON i Java.
- [Oracle Java Tutorials](#): Officiella Java-tutorials från Oracle för att lära dig mer om Java-programmering.

Obligatorisk Dad Joke

Varför gick JSON till terapeuten? För att det hade problem med att parsas sina känslor!

Java för Jin23

Marcus Medina

Mappar

Sammanfattning

Obligatorisk Dad-joke

Mappar

I detta kodexempel använder vi klassen `File` från `java.io`-paketet för att kontrollera, skapa och radera mappar i Java-programmet.

```
import java.io.File;

public class MapparExempel {
    public static void main(String[] args) {
        // Kontrollera om en mapp finns
        String mappSokvag = "C:\\exempel\\mapp";
        File mapp = new File(mappSokvag);
        if (mapp.exists()) {
            System.out.println("Mappen finns.");
        } else {
            System.out.println("Mappen finns inte.");
        }

        // Skapa en mapp
        String nyMappSokvag = "C:\\exempel\\ny_mapp";
        File nyMapp = new File(nyMappSokvag);
        if (!nyMapp.exists()) {
            if (nyMapp.mkdir()) {
                System.out.println("Mappen har skapats.");
            } else {
                System.out.println("Kunde inte skapa mappen.");
            }
        } else {
            System.out.println("Mappen finns redan.");
        }

        // Radera en mapp
        String raderaMappSokvag = "C:\\exempel\\att_radera";
        File raderaMapp = new File(raderaMappSokvag);
        if (raderaMapp.exists()) {
            if (raderaMapp.delete()) {
                System.out.println("Mappen har raderats.");
            } else {
                System.out.println("Kunde inte radera mappen.");
            }
        }
    }
}
```

I detta kodexempel utför vi följande operationer:

1. Kontrollerar om en mapp existerar genom att skapa en `File`-instans med sökvägen till mappen och använda `exists()`-metoden.
2. Skapar en ny mapp genom att skapa en `File`-instans med sökvägen till den nya mappen och använda `mkdir()`-metoden. Vi kontrollerar först om mappen redan finns med hjälp av `exists()`-metoden.
3. Raderar en mapp genom att skapa en `File`-instans med sökvägen till mappen och använda `delete()`-metoden. Vi kontrollerar först om mappen existerar med `exists()`-metoden.

Det är viktigt att vara försiktig när man arbetar med mappar och filer och att hantera eventuella fel eller undantag som kan uppstå. Det kan vara användbart att använda try-catch-block för att fånga och hantera eventuella undantag som genereras vid arbete med mappar och filer.

Sammanfattning

I detta kodexempel har vi sett hur man kan arbeta med mappar i Java genom att använda `File`-klassen från `java.io`-paketet. Vi har utfört operationer som att kontrollera om en mapp existerar, skapa en ny mapp och radera en mapp. Genom att använda dessa metoder kan vi enkelt hantera mappar i våra Java-program.

Det är viktigt att vara försiktig när man arbetar med mappar och filer och att hantera eventuella fel eller undantag som kan uppstå. Det kan vara användbart att använda try-catch-block för att fånga och hantera eventuella undantag som genereras vid arbete med mappar och filer.

Jag hoppas att denna artikel har varit användbar och gett dig en bättre förståelse för hur man arbetar med mappar i Java. Om du har några frågor eller vill lära dig mer, tveka inte att utforska dokumentationen och resurserna från Oracle som jag har länkat till ovan. Ha kul med att organisera dina filer och mappar i dina Java-projekt!

Källor:

- Oracle Docs - File Class

Obligatorisk Dad-joke

Varför älskar mappar att gå på festivaler?

För att de alltid får chansen att "mappa" upp sig!

Java för Jin23

Marcus Medina

Text

- Skapa en textfil
- Läs in en textfil
- Spara en lista i en textfil
- Läs in en lista från en textfil
- Lägg till text i en textfil
- Skriv över en textfil
- Radera en textfil
- Läs in en textfil från en URL
- Spara en textfil från en URL
- Lista på termer i artikeln
- Obligatorisk Dad-joke

Text

Textfiler är en vanlig filtyp som används för att lagra text. Det är ett textbaserat filformat som används för att lagra data av olika slag.

▼ Innehållsförteckning

1. Innehållsförteckning

Skapa en textfil

För att skapa en textfil i Java kan följande kod användas:

```
String contents = "God morgon Mr Bond! Jag har ett meddelande till dig.";
String fileName = "Message.txt";

try {
    FileWriter fileWriter = new FileWriter(fileName);
    BufferedWriter bufferedWriter = new BufferedWriter(fileWriter);
    bufferedWriter.write(contents);
    bufferedWriter.close();
    System.out.println("Textfilen har skapats.");
} catch (IOException e) {
    System.out.println("Ett fel inträffade vid skapandet av textfilen.");
}
```

Läs in en textfil

För att läsa innehållet i en textfil kan följande kod användas:

```
try {
    File file = new File("Message.txt");
    Scanner scanner = new Scanner(file);
    while (scanner.hasNextLine()) {
        String contents = scanner.nextLine();
        System.out.println(contents);
    }
    scanner.close();
} catch (FileNotFoundException e) {
    System.out.println("Textfilen kunde inte hittas.");
}
```

Spara en lista i en textfil

För att spara en lista av strängar i en textfil kan följande kod användas:

```
List<String> names = new ArrayList<>();
names.add("Picard");
names.add("Janeway");
names.add("Kirk");
names.add("Sisko");
names.add("Archer");

try {
    FileWriter fileWriter = new FileWriter("names.txt");
    BufferedWriter bufferedWriter = new BufferedWriter(fileWriter);
    for (String name : names) {
        bufferedWriter.write(name);
        bufferedWriter.newLine();
    }
    bufferedWriter.close();
    System.out.println("Listan har sparats i textfilen.");
} catch (IOException e) {
    System.out.println("Ett fel inträffade vid sparandet av listan i textfilen.");
}
```

Läs in en lista från en textfil

För att läsa innehållet från en textfil och spara det i en lista kan följande kod användas:

```
List<String> names = new ArrayList<>();

try {
    File file = new File("names.txt");
    Scanner scanner = new Scanner(file);
    while (scanner.hasNextLine()) {
        String name = scanner.nextLine();
        names.add(name);
    }
    scanner.close();
} catch (FileNotFoundException e) {
    System.out.println("Textfilen kunde inte hittas.");
}
```

Detta exempel visar hur man kan skapa, läsa, spara och läsa in textfiler i Java. Genom att använda `FileWriter`, `BufferedWriter`, `FileReader`, `BufferedReader` och `Scanner` kan vi utföra olika operationer på textfiler. Listor kan också enkelt sparas och läsas från textfiler genom att använda dessa metoder.

Det är viktigt att hantera eventuella undantag (exceptions) som kan uppstå vid hantering av filer, till exempel om filen inte hittas eller om det uppstår problem med filåtkomst. Genom att använda try-catch-block kan vi fånga och hantera dessa undantag på ett säkert sätt.

Det är också viktigt att stänga filanslutningar när de inte längre behövs för att frigöra resurser och undvika minnesläckor. I kodexemplen ovan används `close()`-metoden för att stänga anslutningarna efter att de har använts.

Lägg till text i en textfil

För att lägga till text i en textfil kan följande kod användas:

```
String contents = "James Bond, din uppgift är enkel men avgörande: Sök
    upp den fiktiva elakingen och eliminera honom utan nåd. Låt
    inget stå i vägen för rättvisa och säkerhet. Var din vanliga
    självsäkra och eleganta själv. Tiden är knapp, agera snabbt och
    precist. Världen litar på dig, 007. Gör det som behöver
    göras.";
String fileName = "Message.txt";

try {
    FileWriter fileWriter = new FileWriter(fileName, true);
    BufferedWriter bufferedWriter = new BufferedWriter(fileWriter);
    bufferedWriter.newLine();
    bufferedWriter.write(contents);
    bufferedWriter.close();
    System.out.println("Texten har lagts till i textfilen.");
} catch (IOException e) {
    System.out.println("Ett fel inträffade vid tillägg av texten i
        textfilen.");
}
```

I FileWriter-konstruktorn kan vi ange true som andra argument för att lägga till text i en befintlig textfil. Detta gör att vi kan lägga till text i en befintlig textfil utan att skriva över den befintliga texten. Hade vi skrivit false istället för true hade den befintliga texten skrivits över.

Skriv över en textfil

För att skriva över en textfil kan följande kod användas:

```
String message = "Message deducted.";
String fileName = "Message.txt";

try {
    FileWriter fileWriter = new FileWriter(fileName);
    BufferedWriter bufferedWriter = new BufferedWriter(fileWriter);
    bufferedWriter.write(message);
    bufferedWriter.close();
    System.out.println("Textfilen har skrivits över.");
} catch (IOException e) {
    System.out.println("Ett fel inträffade vid skrivning över
        textfilen.");
}
```

Radera en textfil

För att radera en textfil kan följande kod användas:

```
String fileName = "Message.txt";

try {
    File file = new File(fileName);
    if (file.delete()) {
        System.out.println("Textfilen har raderats.");
    } else {
        System.out.println
            ("Kunde inte radera textfilen.");
    }
} catch (Exception e) {
    System.out.println("Ett fel inträffade vid radering av textfilen.");
}
```

Läs in en textfil från en URL

För att läsa innehållet i en textfil från en URL kan följande kod användas:

```
try {
    URL url = new URL("https://minwebbsida/text.txt");
    Scanner scanner = new Scanner(url.openStream());
    while (scanner.hasNextLine()) {
        String contents = scanner.nextLine();
        System.out.println(contents);
    }
    scanner.close();
} catch (IOException e) {
    System.out.println("Ett fel inträffade vid läsning av textfilen.");
}
```

Spara en textfil från en URL

För att spara en textfil från en URL kan följande kod användas:

```
try {
    URL url = new URL("https://minwebbsida/text.txt");
    InputStream inputStream = url.openStream();
    Files.copy(inputStream, Paths.get("text.txt"),
        StandardCopyOption.REPLACE_EXISTING);
    System.out.println("Textfilen har sparats.");
} catch (IOException e) {
    System.out.println("Ett fel inträffade vid sparande av textfilen.");
}
```

Lista på termer i artikeln

Ord	Förklaring
Buffer	En buffert är en tillfällig lagringsplats för data.
Buffering	Att använda en buffert för att lagra data.
Byte	En byte är en enhet för digital information som består av åtta bitar.
Byte array	En byte array är en samling av bytes.
Byte stream	En byte stream är en sekvens av bytes.
Character stream	En character stream är en sekvens av tecken.
Close	Att stänga en filanslutning.
Copy	Att kopiera en fil.
File	En fil är en samling av data som lagras på en enhet.
File path	En filväg är en sträng som identifierar en fil.
File writer	En filskrivare är en klass som används för att skriva till filer.
File reader	En filskrivare är en klass som används för att läsa från filer.
File input stream	En filinmatningsström är en ström som används för att läsa från filer.
File output stream	En filutmatningsström är en ström som används för att skriva till filer.
File writer	En filskrivare är en klass som används för att skriva till filer.
File reader	En filskrivare är en klass som används för att läsa från filer.

Obligatorisk Dad-joke

Varför gillar textfiler att gå på fester?

För att de älskar att dra skämt om "line"-dans!

Java för Jin23

Marcus Medina

Path-klassen

Path-klassen i Java erbjuder verktyg för att hantera sökvägar till filer och mappar. Genom att använda Path-klassens olika metoder kan vi enkelt utföra operationer som att slå ihop sökvägar, extrahera information om filer och mappar, ändra filändelser och mycket mer. Detta gör det lättare att arbeta med filsystemet inom våra program och skapa bättre och mer robusta applikationer.

```
import java.nio.file.Path;
import java.nio.file.Paths;

public class PathExample {
    public static void main(String[] args) {
        // Skapa en sökväg till en fil
        Path filePath =
            Paths.get("C:\\Users\\User\\Documents\\file.txt");

        // Slå ihop sökvägar
        Path combinedPath = filePath.resolve("subfolder\\file2.txt");

        // Extrahera filnamn
        String fileName = filePath.getFileName().toString();

        // Byta filändelse
        Path newFilePath =
            filePath.resolveSibling(fileName.replace(".txt", ".csv"));

        // Kontrollera om sökvägen har en filändelse
        boolean hasExtension = filePath.toString().contains(".");

        // Skriv ut sökvägarna
        System.out.println("File Path: " + filePath);
        System.out.println("Combined Path: " + combinedPath);
        System.out.println("File Name: " + fileName);
        System.out.println("New File Path: " + newFilePath);
        System.out.println("Has Extension: " + hasExtension);
    }
}
```

I det här exemplet använder vi Path-klassen för att utföra olika operationer på sökvägar. Vi skapar en sökväg till en fil och visar hur man slår ihop sökvägar, extraherar filnamn, ändrar filändelser och kontrollerar om en sökväg har en filändelse. Vi använder också metoden `resolveSibling` för att byta filändelse och `resolve` för att slå ihop sökvägar.

Resultatet av körningen av detta program är:

```
File Path: C:\Users\User\Documents\file.txt
Combined Path: C:\Users\User\Documents\subfolder\file2.txt
File Name: file.txt
New File Path: C:\Users\User\Documents\file.csv
Has Extension: true
```

Som du kan se ger Path-klassen oss enkla och effektiva verktyg för att hantera sökvägar till filer och mappar i Java-program.

Oavsett om du behöver slå ihop sökvägar, extrahera information om filer och mappar eller ändra filändelser, kan Path-klassen vara till stor hjälp. Genom att utforska dokumentationen för Path-klassen kan du lära dig mer om de olika metoderna och hur de kan användas för att hantera sökvägar i dina Java-program.

Path-klassen

- [Java Path Documentation](#)

- Java Path Tutorial
-

Java för Jin23

Marcus Medina

Asynkron

TL;DR

När du läst detta ska du kunna

Introduktion

Beskrivning

När ska man använda asynkrona metoder?

Sammanfattning

Java

Fördelar

Begränsningar

Användningsområden

Exempelkod - Asynkrona anrop (Java)

Slutsats

Termtabell

Obligatorisk Dad-joke

Asynkron

Asynkrona metoder är metoder som inte blockerar tråden som kör metoden. Detta gör det möjligt för oss att köra flera metoder samtidigt och förbättra applikationens responsivitet.

TL;DR

Asynkrona metoder gör det möjligt att köra flera metoder samtidigt och förbättra applikationens responsivitet genom att inte blockera tråden.

När du läst detta ska du kunna

- Förstå vad asynkrona metoder är och hur de fungerar.
- Veta när det är lämpligt att använda asynkrona metoder.
- Kunna använda asynkrona metoder i Java.

Introduktion

I vissa situationer kan det vara nödvändigt att köra flera metoder samtidigt i en applikation. Det kan handla om att kommunicera med andra system och API:er eller att hantera tunga beräkningar. Traditionellt sett skulle sådana metoder blockera huvudtråden i applikationen och göra den otillgänglig för andra operationer tills den aktuella metoden är klar. Detta kan resultera i en långsam och ineffektiv användarupplevelse. För att undvika detta och göra applikationen mer responsiv kan man använda sig av asynkrona metoder.

Beskrivning

Asynkrona metoder i Java gör det möjligt att utföra operationer utan att blockera tråden som kör metoden. Istället för att vänta på att en operation är klar innan man går vidare till nästa, kan man använda nyckelordet `Runnable` tillsammans med `Thread` för att ange att en metod ska köras asynkront. När en metod är markerad som `Runnable` kommer den att returnera ett `Thread`-objekt som representerar den asynkrona operationen. När man använder `Thread` för att vänta på att en asynkron operation är klar, kommer tråden som kör metoden att släppas och andra operationer kan köras samtidigt.

När ska man använda asynkrona metoder?

Asynkrona metoder är användbara när man vill förbättra applikationens responsivitet och prestanda. De är också användbara när man vill köra flera operationer samtidigt, som att kommunicera med andra system eller hantera tunga beräkningar.

Sammanfattning

Asynkrona metoder är ett användbart verktyg för att förbättra applikationens responsivitet och prestanda. De gör det möjligt att köra flera metoder samtidigt och blockera inte tråden som kör metoden. I Java kan man använda nyckelordet `Runnable` tillsammans med `Thread` för att ange att en metod ska köras asynkront.³
Kodförenkling: Asynkrona metoder kan göra koden mer läsbar och förenkla kodstrukturen.

Java

Vanligtvis används asynkrona metoder när man inte vet hur lång tid en operation kommer att ta, till exempel vid kommunikation över nätverk eller när man arbetar med stora datamängder. Genom att köra sådana operationer asynkront kan man göra applikationen mer responsiv och tillåta att andra operationer utförs parallellt.

Det är viktigt att notera några viktiga punkter när man använder asynkrona metoder:

- En asynkron metod måste alltid returnera antingen en `Future` eller en `Future<T>`. Detta är för att kunna köra flera metoder samtidigt.
- Asynkrona metoder kan inte ha en `void`-returtyp. Istället bör de returnera en `Future` om inget värde behövs eller en `Future<T>` om det finns ett returvärde.
- Asynkrona metoder kan inte ha `out`- eller `ref`-parametrar.
- Asynkrona metoder kan inte ha parametrar som är av typen `varargs`.

Fördelar

Användningen av asynkrona metoder kan ge flera fördelar i en applikation:

1. **Responsivitet:** Genom att använda asynkrona metoder kan man undvika att blockera huvudtråden i applikationen och ge en mer responsiv användarupplevelse.
2. **Parallellism:** Genom att köra flera metoder samtidigt kan man utnyttja flera processorkärnor och därmed förbättra prestanda och genomströmning.
3. **Kodförenkling:** Asynkrona metoder kan göra koden mer läsbar och förenkla kodstrukturen.³ **Förbättrad skalbarhet:** Asynkrona metoder möjliggör parallell exekvering av operationer, vilket kan förbättra skalbarheten i en applikation och möjliggöra hantering av hög trafik eller arbetsbelastning.

Begränsningar

Trots de fördelar som asynkrona metoder erbjuder finns det vissa begränsningar och överväganden att tänka på:

1. **Komplexitet:** Asynkron programmering kan vara mer komplext än synkron programmering, särskilt när det gäller att hantera fel och synkronisering mellan olika operationer.
2. **Ökad CPU-användning:** Asynkrona metoder kan resultera i en ökad CPU-användning på grund av overhead för att starta och hantera nya trådar eller arbetsenheter.
3. **Felhantering:** Det kan vara utmanande att hantera fel i asynkrona metoder, särskilt när flera operationer körs samtidigt. Korrekt felhantering och återställning är viktigt för att undvika buggar och felaktigt beteende.

Användningsområden

Asynkrona metoder används i olika situationer och applikationsområden:

1. **Webbapplikationer:** Vid kommunikation med webbtjänster och API:er kan asynkrona metoder användas för att göra anrop och hämta data på ett effektivt sätt utan att blockera huvudtråden.
2. **Databashantering:** Asynkrona metoder kan användas vid databashantering för att parallellt hämta och uppdatera data från flera källor samtidigt. Asynkrona metoder är ett användbart verktyg för att förbättra prestanda och responsivitet i applikationer som hanterar flera anrop samtidigt.

Exempelkod - Asynkrona anrop (Java)

Här är ett exempel på hur man kan använda asynkrona metoder i Java:

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.URL;
import java.net.URLConnection;
import java.util.concurrent.CompletableFuture;

public class Main {
    public static CompletableFuture<String> getAsync(String url) {
        // Starta en ny tråd och exekvera GET-begäran
        return CompletableFuture.supplyAsync(() -> {
            try {
                // Skapa en URL-anslutning till webbplatsen
                URL website = new URL(url);
                // Öppna anslutningen
                URLConnection connection = website.openConnection();
                // Skapa en BufferedReader för att läsa svaret
                BufferedReader in = new BufferedReader(new
InputStreamReader(connection.getInputStream()));
                // Läs svaret från anslutningen
                StringBuilder response = new StringBuilder();

                String inputLine;
                // Läs varje rad från svaret
                while ((inputLine = in.readLine()) != null)
                    response.append(inputLine);
                in.close();
                // Returnera svaret som en sträng
                return response.toString();
            } catch (IOException e) {
                // Hantera eventuella IOExceptions
                e.printStackTrace();
                return null;
            }
        });
    }

    public static void main(String[] args) {
        // Utför en asynkron GET-begäran till "https://www.google.com"
        CompletableFuture<String> result =
getAsync("https://www.google.com");
        try {
            // Vänta på att resultatet ska bli tillgängligt
            System.out.println(result.get());
        } catch (Exception e) {
            // Hantera eventuella undantag
            e.printStackTrace();
        }
    }
}
```

Här är förklaringar till vad varje del av koden gör:

1. Importera nödvändiga klasser från Java API.
2. Skapa en klass Main.

3. Definiera `getAsync`-metoden som returnerar en `CompletableFuture` som innehåller resultatet av en asynkron HTTP GET-begäran.
4. Anropa `CompletableFuture.supplyAsync` för att starta en ny tråd och exekvera GET-begäran.
5. Inuti den nya tråden, försök att utföra GET-begäran.
6. Skapa en `URL`-instans baserat på den angivna URL-strängen.
7. Öppna en anslutning (`URLConnection`) till webbplatsen.
8. Skapa en `BufferedReader` för att läsa svaret från anslutningen.
9. Skapa en `StringBuilder` för att bygga upp hela svaret rad för rad.
10. Läs varje rad från svaret och lägg till den i `response`-objektet.
11. Stäng `BufferedReader`.
12. Returnera det kompletta svaret som en sträng.
13. Hantera eventuella `IOExceptions` genom att skriva ut felmeddelandet och returnera `null`.
14. Definiera `main`-metoden som startpunkt för programmet.
15. Anropa `getAsync`-metoden för att utföra en asynkron GET-begäran till "https://www.google.com".
16. Anropa `result.get()` för att vänta på att resultatet av GET-begäran ska vara tillgängligt.
17. Skriv ut resultatet till konsolen.
18. Hantera eventuella undantag genom att skriva ut felmeddelandet.``

I exemplet ovan definieras en asynkron metod `getAsync` som hämtar data från en given URL. Metoden använder `CompletableFuture` för att göra ett asynkront HTTP-anrop och returnerar innehållet som en sträng. I `main`-metoden anropas `getAsync` och resultatet skrivs ut på konsolen.

Slutsats

Asynkrona metoder är ett användbart verktyg för att förbättra prestanda och responsivitet i applikationer som hanterar flera anrop samtidigt. Asynkrona metoder är ett kraftfullt verktyg för att göra applikationer mer responsiva och effektiva. Genom att köra flera metoder samtidigt kan man förbättra prestanda och användarupplevelse. Det är viktigt att använda asynkrona metoder på rätt sätt och vara medveten om deras begränsningar och överväganden. Med rätt användning kan asynkron programmering vara ett värdefullt verktyg i utvecklingen av moderna applikationer.

Termtabell

- **Asynkrona metoder:** Metoder som inte blockerar tråden och tillåter körsel av flera metoder samtidigt.
- **Responsivitet:** Förmågan hos en applikation att snabbt svara på användarinteraktioner och andra händelser.
- **Parallellism:** Exekvering av flera operationer samtidigt för att utnyttja flera processorkärnor och förbättra prestanda.
- **Skalbarhet:** Förmågan hos en applikation att hantera en ökad arbetsbelastning och trafik utan att försämra prestanda och responsivitet.

Obligatorisk Dad-joke

Varför gillar programmerare att arbeta med asynkrona metoder?

För att de älskar att "vänta" på spänning!

Java för Jin23

Marcus Medina

Exempel

Kod

Slutsats

Termtabell

Obligatorisk Dad-joke

Exempel

Yay! Nu ska vi titta på ett exempel där vi ska skapa en metod som söker igenom alla filer i en mapp och returnerar en lista med filer som innehåller en specifik text. Vi kommer att använda asynkrona metoder för att kunna köra flera metoder samtidigt och därigenom effektivisera vår sökning. Hur coolt låter inte det?

▼ Innehållsförteckning

1. Innehållsförteckning

Kod

```
import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class Main {
    public static void main(String[] args) throws Exception {
        // Skapa en ny instans av DirReader-klassen
        DirReader dir = new DirReader();

        // Hämta användarens hemkatalog
        String profile = System.getProperty("user.home");

        // Definiera sökvägen till källmappen
        String source = Paths.get(profile, "source").toString();

        // Sök efter alla .java-filer i källmappen asynkront
        List<String> result = dir.searchFilesAsync(source, "*.java",
            "asynkron").get();

        // Skriv ut resultaten till konsolen
        System.out.println(String.join(System.lineSeparator(), result));
    }
}

class DirReader {
    // Sök efter alla filer i en mapp som matchar ett angivet mönster
    // och innehåller en viss text
    public CompletableFuture<List<String>> searchFilesAsync(String path,
        String searchPattern, String searchText) {
        try (Stream<Path> paths = Files.walk(Paths.get(path))) {
            // Skapa en lista med uppgifter för att söka varje fil
            // asynkront
            List<CompletableFuture<String>> tasks = paths
                .filter(Files::isRegularFile) // Endast vanliga
                // filer beaktas
                .filter(p ->
                    p.getFileName().toString().endsWith(searchPattern)) // Endast
                // filer som matchar sökmönstret beaktas
        }
    }
}
```

```

        .map(p -> searchFileAsync(p.toString(), searchText))
// Sök i varje fil asynkront
        .collect(Collectors.toList());

// Vänta på att alla uppgifter ska slutföras och samla
// resultaten
        CompletableFuture<Void> allTasks =
        CompletableFuture.allOf(tasks.toArray(new
        CompletableFuture[tasks.size()]));
        return allTasks.thenApply(v -> tasks.stream()
        .map(CompletableFuture::join)
        .filter(x -> x != null)
        .collect(Collectors.toList()));
    } catch (IOException e) {
        // Om ett fel inträffar, returnera ett misslyckat
        // CompletableFuture med undantaget
        e.printStackTrace();
        return CompletableFuture.failedFuture(e);
    }
}

// Sök efter en given text i en enskild fil och returnera resultatet
public CompletableFuture<String> searchFileAsync(String path, String
searchText) {
    return CompletableFuture.supplyAsync(() -> {
        try {
            // Läs in alla rader i filen
            List<String> lines =
            Files.readAllLines(Paths.get(path));

            // Sök efter texten i filen
            int index = lines.indexOf(searchText);

            // Om texten hittas, returnera filens sökväg och
            // radnummer
            return index >= 0 ? path + " - " + index : null;
        } catch (IOException e) {
            // Om ett fel inträffar, returnera null
            e.printStackTrace();
            return null;
        }
    });
}
}

```

1. Vi börjar med att skapa en metod som heter SearchFilesAsync.
2. Metoden tar emot en sökväg, sökmönster och söktext.
3. Vi använder Files.walk för att hämta en ström med alla filer i den angivna sökvägen.
4. Därefter skapar vi en lista med tasks som kommer att köra metoden SearchFileAsync.
5. Vi loopar igenom varje fil och lägger till en task för varje fil i vår lista.
6. Genom att använda CompletableFuture.allOf väntar vi på att alla tasks ska bli klara.
7. Vi returnerar en lista med resultatet från varje task som inte är null.
8. Därefter skapar vi en metod som heter SearchFileAsync.
9. Metoden tar emot en sökväg och en söktext.
10. Vi använder Files.readAllLines för att läsa innehållet i filen och sparar det i en lista.
11. Vi skapar en variabel som håller koll på vilken rad vi är på i filen.
12. Sedan loopar vi igenom varje rad i filen.
13. Om raden innehåller den sökta texten så returnerar vi sökvägen till filen och vilken rad det var.
14. Vi läser in nästa rad från filen.
15. Vi ökar radnumret med 1.
16. Om vi inte hittar någon rad som innehåller den sökta texten så returnerar vi null.
17. Slutligen har vi vår Main-metod där vi skapar en instans av DirReader och definierar sökvägen till vår mapp.
18. Vi kör metoden SearchFilesAsync och sparar resultatet i en variabel.
19. Till sist skriver vi ut alla resultat i konsolen.

Wow! Nu kan vi söka igenom filer asynkront och få en lista med de filer som innehåller vår sökta text. Det är riktigt coolt att kunna köra flera metoder samtidigt och på så sätt förbättra vår effektivitet. Fortsätt det fantastiska arbetet!

Slutsats

Att kunna utföra asynkrona operationer är en kraftfull teknik som gör att våra applikationer kan vara mer responsiva och effektiva. Genom att använda asynkrona metoder kan vi undvika att blockera huvudtråden och istället köra flera operationer samtidigt. Detta är särskilt användbart vid uppgifter som tar tid, som att söka igenom filer eller kommunicera med externa system. Genom att utnyttja asynkron programmering kan vi skapa mer responsiva och skalbara applikationer. Fortsätt kodning med glädje!

Termtabell

- **Asynkrona metoder:** Metoder som inte blockerar tråden och tillåter parallell exekvering av flera metoder samtidigt.
- **Responsivitet:** Förmågan hos en applikation att snabbt svara på användarinteraktioner och andra händelser.
- **Parallellism:** Exekvering av flera operationer samtidigt för att utnyttja flera processorkärnor och förbättra prestanda.
- **Skalbarhet:** Förmågan hos en applikation att hantera en ökad arbetsbelastning och trafik utan att försämra prestanda och responsivitet.

Obligatorisk Dad-joke

Varför kallas asynkrona metoder för "coola"?

För att de jobbar i sin egen "takt"!

Java för Jin23

Marcus Medina

API

- TL; DR
- Fördjupning
- Exempel
- Referenser
- Obligatorisk Dad-joke

API

Nu ska vi prata om API, eller Application Programming Interface, som är ett viktigt gränssnitt för att underlätta kommunikationen med webbapplikationer. API:er fungerar som en slags mellanhand mellan två applikationer, vilket gör det möjligt för oss att ställa frågor och få svar från webbapplikationer samt skicka och ta emot data. Genom API:er blir klienten och servern oberoende av varandra och kan enkelt utbyta information.

▼ Innehållsförteckning

TL; DR

API:er är ett viktigt gränssnitt för att underlätta kommunikationen med webbapplikationer. De gör det möjligt för oss att ställa frågor och få svar från webbapplikationer samt skicka och ta emot data. Genom API:er blir klienten och servern oberoende av varandra och kan enkelt utbyta information.

Fördjupning

API:er gör det möjligt för oss att använda webbapplikationer utan att behöva interagera med deras grafiska gränssnitt (GUI). Istället kan vi kommunicera med applikationen genom att ställa frågor och begära data via en URL. Svaren från API:et kommer vanligtvis i formatet XML eller JSON, vilket gör det lättare att läsa och tolka informationen.

Genom att använda API:er kan vi skapa mer flexibla och dynamiska applikationer. Vi kan anpassa datautbytet mellan klienten och servern och se till att endast de delar av servern som API:et tillåter blir synliga för klienten. Detta underlättar kommunikationen och ger oss möjlighet att skapa mer effektiva och modulära system.

Exempel

Här är ett exempel på hur vi kan använda ett API i Java för att hämta data från The Movie Database (TMDb):

```

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;

public class TmdbApiExample {
    public static void main(String[] args) {
        try {
            // API-nyckel från The Movie Database (TMDb)
            String apiKey = "DIN_API_NYCKEL_HÄR";

            // URL för att fråga efter filmen "Happy Death Day"
            String url = "https://api.themoviedb.org/3/search/movie?
            api_key=" + apiKey + "&query=Happy+death+day";

            // Skapa en URL-objekt från strängen
            URL apiURL = new URL(url);

            // Öppna en anslutning till URL:en
            HttpURLConnection connection = (HttpURLConnection)
            apiURL.openConnection();

            // Ange att vi vill göra en GET-begäran
            connection.setRequestMethod("GET");

            // Hämta svarskod från anslutningen
            int responseCode = connection.getResponseCode();
            System.out.println("Svarskod: " + responseCode);

            // Läs svaret från anslutningen
            BufferedReader reader = new BufferedReader(new
            InputStreamReader(connection.getInputStream()));
            String inputLine;
            StringBuilder response = new StringBuilder();
            while ((inputLine = reader.readLine()) != null) {
                response.append(inputLine);
            }
            reader.close();

            // Visa svaret
            System.out.println(response.toString());

            // Stäng anslutningen
            connection.disconnect();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Koden ovan gör följande:

1. Den skapar en URL för att fråga The Movie Database API efter filmen "Happy Death Day" med hjälp av en API-nyckel.
2. Den skapar en HTTP-anslutning (HttpURLConnection) till URL:en och specificerar att vi vill göra en GET-begäran.
3. Den hämtar svarskoden från anslutningen och visar den.
4. Den läser svaret från anslutningen och sparar det i en sträng.
5. Den visar svaret på konsolen.
6. Den stänger anslutningen.

Referenser

- The Movie Database API
- Oracle - HttpURLConnection

Jag hoppas att du har lärt er något nytt om API:er och hur de kan underlätta kommunikationen mellan webbapplikationer. För mer information och djupgående dokumentation rekommenderar jag er att besöka referenserna ovan. Ha en fortsatt bra dag och lycka till med era programmeringsprojekt!

Obligatorisk Dad-joke

Varför älskar utvecklare att arbeta med APIer?

För att de ger dem en chans att "PUT" sina skämt "GET" ersättningar! ☐

Java för Jin23

Marcus Medina

Övrigt

Övrigt

Här finns några andra ämnen som inte riktigt passar någon annanstans. Detta kommer att fyllas på med tiden...

Java för Jin23

Marcus Medina

SQL

SQL

SQL står för Structured Query Language och är ett språk som används för att kommunicera med databaser. Det är ett av de mest använda språken i världen.

Filen editerades senast 2022-12-09 ## Referenser

- [SQL](#)
- [SQL Cheat Sheet](#)
- [SQL Zoo](#)
- [SQL Fiddle](#)

Java för Jin23

Marcus Medina

Tabeller

- Lägg till en kolumn

- Ta bort en kolumn

- Referenser

 - Lägg till en kolumn och gör den till en ForeignKey

- Ta bort en ForeignKey

- Skapa en tabell med en ForeignKey

- Skapa en tabell med en ForeignKey och en primärnyckel

- Referenser

Tabeller

Tabeller är som ett rutnät av data som lagras i en databas. Varje tabell har en eller flera kolumner, och varje rad i tabellen är en post. Varje post har en värde för varje kolumn. En tabell kan ha en primärnyckel, som är en kolumn som är unik för varje post. En tabell kan också ha en eller flera främmande nycklar, som är en kolumn som är kopplad till en annan tabell. Exempelen här nedanför är för MySQL. Det kan skilja på andra servrar

▼ Innehållsförteckning

Filen editades senast 2022-12-09 ### Skapa en tabell om den inte finns

Här skapar vi en tabell med namnet `users` och tre kolumner: `id`, `name` och `email`.

```
CREATE TABLE IF NOT EXISTS users (  
  id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(30) NOT NULL,  
  email VARCHAR(50)  
);
```

Lägg till en kolumn

```
ALTER TABLE users ADD COLUMN age INT(3);
```

Ta bort en kolumn

```
ALTER TABLE users DROP COLUMN age;
```

Referenser

Lägg till en kolumn och gör den till en ForeignKey

Vi skapar en länk till en annan tabell. I detta fall till tabellen `cars` och kolumnen `CarId`.

```
ALTER TABLE users ADD COLUMN CarId INT(6) UNSIGNED;  
ALTER TABLE users ADD FOREIGN KEY (CarId) REFERENCES cars(CarId);
```

Ta bort en ForeignKey

Vi tar bort referensen till `Cars.CarId`

```
ALTER TABLE users DROP FOREIGN KEY users_ibfk_1;
```

`users_ibfk_1` är den första ForeignKey som finns i tabellen `users`.

Skapa en tabell med en ForeignKey

```
CREATE TABLE IF NOT EXISTS cars (  
  CarId INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  Model VARCHAR(30) NOT NULL,  
  Brand VARCHAR(50),  
  UserId INT(6) UNSIGNED,  
  FOREIGN KEY (UserId) REFERENCES users(id)  
);
```

Skapa en tabell med en ForeignKey och en primärnyckel

```
CREATE TABLE IF NOT EXISTS cars (  
  CarId INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  Model VARCHAR(30) NOT NULL,  
  Brand VARCHAR(50),  
  UserId INT(6) UNSIGNED,  
  PRIMARY KEY (CarId, UserId),  
  FOREIGN KEY (UserId) REFERENCES users(id)  
);
```

Referenser

- W3Schools
- MySQL

Java för Jin23

Marcus Medina

Databaser

- Skapa en databas om den inte finns
- Ta bort en databas
- Ändra namn på databasen

Databaser

▼ Innehållsförteckning

Filen editerades senast 2022-12-09 ### Skapa en databas

CREATE DATABASE databasnamn;

Skapa en databas om den inte finns

CREATE DATABASE IF NOT EXISTS databasnamn;

Ta bort en databas

Tänk på att backuppa först, annars är allt borta.

DROP DATABASE databasnamn;

Ändra namn på databasen

Farligt! Detta kan skapa problem om du har kopplat dig till databasen med ett program.

ALTER DATABASE databasnamn **RENAME TO** nytt_databasnamn;

Java för Jin23

Marcus Medina

Constraints

Foreign Key

Exempel

Exempel på hur man kopplar ihop tabellerna

Referenser

Constraints

SQL-Constraints är ett sätt att begränsa datan i en tabell. Det finns två typer av constraints, primary key och foreign key. Primary key är en constraint som gör att en kolumn i en tabell kan ha unika värden. Foreign key är en constraint som gör att en kolumn i en tabell kan ha värden som finns i en annan tabell.

▼ Innehållsförteckning

Filen editades senast 2022-12-09 ## Primary Key

Primary Key är det värde som är unikt för varje rad i en tabell. Det är det värde som används för att identifiera en rad i en tabell. Det är också det värde som används för att koppla ihop tabeller med varandra. Det är inte möjligt att ha fler än en primary key i en tabell. Det är också inte möjligt att ha en primary key som är null. Det är också möjligt att ha en primary key som är en kombination av flera kolumner.

Foreign Key

Foreign key är ett värde som finns i en tabell som är kopplat till ett värde i en annan tabell. Det är det värde som används för att koppla ihop tabeller med varandra. Det är inte möjligt att ha fler än en foreign key i en tabell. Det är också inte möjligt att ha en foreign key som är null. Det är också möjligt att ha en foreign key som är en kombination av flera kolumner.

Exempel

```
CREATE TABLE Person (  
  PersonID int NOT NULL PRIMARY KEY,  
  LastName varchar(255) NOT NULL,  
  FirstName varchar(255),  
  Address varchar(255),  
  City varchar(255),  
  PRIMARY KEY (PersonID)  
);  
CREATE TABLE Orders (  
  OrderID int NOT NULL PRIMARY KEY,  
  OrderDate date NOT NULL,  
  PersonID int NOT NULL,  
  PRIMARY KEY (OrderID),  
  FOREIGN KEY (PersonID) REFERENCES Person(PersonID)  
);
```

Man använder Primary key för att identifiera en rad i en tabell, och Foreign key för att koppla ihop tabeller med varandra.

Exempel på hur man kopplar ihop tabellerna

```
INSERT INTO Person (PersonID, LastName, FirstName, Address, City)  
VALUES (1, 'Peter', 'Petersson', 'St Petersgatan 1', 'Peterstad');  
INSERT INTO Orders (OrderID, OrderDate, PersonID)  
VALUES (1, '2021-01-01', 1);
```

och vi kan sedan hämta ut informationen från båda tabellerna med följande kod:

```
SELECT * FROM Person  
INNER JOIN Orders ON Person.PersonID = Orders.PersonID;
```

Referenser

SQL Constraints

Java för Jin23

Marcus Medina

Ordlista

Ordlista

Här finns några ordlistor med vanliga ord och uttryck som används inom IT världen, med korta förklaringar.

Java för Jin23

Marcus Medina

Git

Git

Ord	Förklaring
Repository	En mapp som innehåller alla filer och historiken för en projekt.
Branch	En gren i repot. Det är en ögonblickskopia av repot.
Commit	En commit är en punkt i historiken. Det är en punkt där du sparar dina ändringar.
Push	Att skicka upp ändringar till servern.
Pull	Att hämta ner ändringar från servern.
Merge	Att slå ihop två grenar.
Rebase	Att flytta en gren till en annan gren.
Stash	Att spara undan ändringar som inte är klara.
Checkout	Att byta gren.
Clone	Att kopiera ett repo till din dator.
Fork	Att kopiera ett repo till din github.
Pull request	Att be servern merga två brancher.
Merge conflict	När två grenar har ändrat samma fil på samma ställe.
Remote	En koppling till ett annat repo.
Origin	Det vanligaste namnet på en remote.
Upstream	Det vanligaste namnet på en remote som inte är din.
Fork	Att kopiera ett repo till din github.
Pull request	Att be servern merga två brancher.

Java för Jin23

Marcus Medina

Programmering

Programmering

Ord	Förklaring
Algorithm	En algoritm är lista av kommandon i steg för steg- beskrivning, för att lösa ett problem.
Attribut	En variabel som tillhör ett objekt.
Deserialisera	Att göra om en sträng till ett objekt.
Get	En metod som returnerar en variabel.
Instansvariabel	En variabel som tillhör ett objekt.
Interface	En mall för en klass.
Klass	En mall för ett objekt.
Klassvariabel	En variabel som tillhör en klass.
Konstruktor	En metod som körs när ett objekt skapas.
Metod	En funktion som tillhör ett objekt.
Modifierare	En modifierare är en modifierare som ändrar hur en variabel eller metod fungerar.
Namespace	En grupp av klasser.
Namespace alias	Att ge en namespace ett alias.
Namespace import	Att importera en namespace.
Namespace using	Att använda en namespace.
Objekt	En instans av en klass.
Property	En metod som sätter och returnerar en variabel.
Serialisera	Att göra om ett objekt till en sträng.
Set	En metod som sätter en variabel.
Static	En modifierare som gör att en variabel tillhör klassen och inte objektet.
using static	Att använda en klass som namespace och dess statiska metoder.

Java för Jin23

Marcus Medina

Programmeringsspråk

Programmeringsspråk

Språk	Beskrivning
Angular	Ett JavaScript-bibliotek.
Composer	Ett pakethanteringsverktyg.
CSS	Cascading Style Sheets.
Docker	Ett program som gör det enkelt att köra program i en container.
Docker Compose	Ett verktyg för att skapa och köra flera Docker-containers.
Docker Desktop	Ett program som gör det enkelt att köra Docker.
Docker Hub	Ett webbaserat verktyg för att lagra Docker-containers.
Docker Machine	Ett verktyg för att skapa och köra flera Docker-containers.
Docker Registry	Ett webbaserat verktyg för att lagra Docker-containers.
Docker Swarm	Ett verktyg för att skapa och köra flera Docker-containers.
Docker Toolbox	Ett program som gör det enkelt att köra Docker.
Gatsby	Ett statiskt webbplatsbyggverktyg.
Git	Ett versionshanteringssystem.
GitHub	Ett webbaserat versionshanteringssystem.
GitLab	Ett webbaserat versionshanteringssystem.
HTML	Hypertext Markup Language.
Hugo	Ett statiskt webbplatsbyggverktyg.
JavaScript	Ett programmeringsspråk.
Jekyll	Ett statiskt webbplatsbyggverktyg.
JSON	JavaScript Object Notation.
Markdown	Ett textformat.
Node	Ett JavaScript-bibliotek.
NPM	Ett pakethanteringsverktyg.
Ord	Förklaring
PHP	Ett programmeringsspråk.
React	Ett JavaScript-bibliotek.
SQL	Structured Query Language.
Vue	Ett JavaScript-bibliotek.
XML	Extensible Markup Language.
YAML	Yet Another Markup Language.
Yarn	Ett pakethanteringsverktyg.

Java för Jin23

Marcus Medina

Webb

Webb

Ord	Förklaring
CDN	Content Delivery Network.
DNS	Domain Name System.
DNS	Domain Name System.
FTP	File Transfer Protocol.
FTPS	File Transfer Protocol Secure.
HTTP	Hypertext Transfer Protocol.
HTTPS	Hypertext Transfer Protocol Secure.
IMAP	Internet Message Access Protocol.
IP	Internet Protocol.
POP	Post Office Protocol.
SFTP	Secure File Transfer Protocol.
SMTP	Simple Mail Transfer Protocol.
SSH	Secure Shell.
SSL	Secure Sockets Layer.
TLS	Transport Layer Security.
URL	Uniform Resource Locator.
VPN	Virtual Private Network.
VPS	Virtual Private Server.

Java för Jin23

Marcus Medina

Yrken

Yrken

Ord	Förklaring
Backend	Logiken bakom ett program.
Backend utvecklare	En person som hjälper företag med backend.
Cloud	En molntjänst.
Cloud Architect	En person som planerar och utvecklar molntjänster.
Cloud Consultant	En person som hjälper företag med molntjänster.
Cloud Engineer	En person som hjälper företag med molntjänster.
Cloud Security	En person som hjälper företag med molntjänster.
Cloud utvecklare	En person som hjälper företag med molntjänster.
DevOps	En metod för att utveckla och driftsätta program.
DevOps Architect	En person som planerar och utvecklar DevOps.
DevOps Consultant	En person som hjälper företag med DevOps.
DevOps Engineer	En person som hjälper företag med DevOps.
Frontend	Det som användaren ser. Användargränssnittet.
Frontend utvecklare	En person som hjälper företag med frontend.
Fullstack	En person som kan utveckla både frontend och backend.
IT	Informationsteknik.
IT-arkitekt	En person som planerar och utvecklar IT-system.
IT-konsult	En person som hjälper företag med IT.
IT-säkerhet	En person som hjälper företag med IT-säkerhet.
IT-support	En person som hjälper användare med IT.
IT-tekniker	En person som hjälper användare med IT.
Systemarkitekt	En person som planerar och utvecklar system.
Systemutvecklare	En person som hjälper företag med systemutveckling.

References